

รายงานการศึกษาอัลกอริทึมสำหรับการสร้างกรณีทดสอบอัตโนมัติ
และการออกแบบพรมพืปัญญาประดิษฐ์เชิงสร้างสรรค์
เพื่อการทดสอบซอฟต์แวร์

สมาชิก

663380507-9 นางสาวนิภาดา ญาเย็นนันท์	Section 1
673380584-2 นางสาวทัตพิชา ะสาร	Section 2
673380602-6 นางสาวศศิวิตรา วงษ์รุ่งอรุณเลิศ	Section 2

เสนอ

ผศ.ดร.ชิตสุธา สุ่มเล็ก

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353201 การประกันคุณภาพซอฟต์แวร์
ภาคเรียนที่ 1 ปีการศึกษา 2569
สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์
มหาวิทยาลัยขอนแก่น
(เดือน ตุลาคม พ.ศ. 2569)

นิภาดา ญาเย็นนันท์, ทัดพิชา ะสาร และ ศศิวิตรา วงษ์รุ่งอรุณเลิศ. 2569. รายงานการศึกษอัลกอริทึมสำหรับการสร้างกรณีทดสอบอัตโนมัติ และการออกแบบพรมพปัญหาประดิษฐ์เชิงสร้างสรรค์ เพื่อการทดสอบซอฟต์แวร์. รายงานประกอบการเรียน. รายวิชาการประกันคุณภาพซอฟต์แวร์ รหัสวิชา CP353201

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น.

เสนอ: ผศ.ดร.ชิตสุธา สุ่มเล็ก.

บทคัดย่อ

รายงานฉบับนี้จัดทำขึ้นเพื่อเป็นส่วนหนึ่งของรายวิชาการประกันคุณภาพซอฟต์แวร์ (Software Quality Assurance) รหัสวิชา CP353201 ภาคการศึกษาที่ 1 ปีการศึกษา 2569 โดยมีวัตถุประสงค์เพื่อศึกษาและประเมินแนวทางการสร้างกรณีทดสอบอัตโนมัติ โดยเปรียบเทียบการใช้ขั้นตอนวิธีสำหรับการสร้างกรณีทดสอบอัตโนมัติ (Automatic Test Case Generation) กับการประยุกต์ใช้ปัญญาประดิษฐ์เชิงสร้างสรรค์ (Generative AI) สำหรับการสร้างชุดทดสอบและกรณีทดสอบในระดับ Unit Testing

การดำเนินงานประกอบด้วยการศึกษาและประยุกต์ใช้ขั้นตอนวิธีสำหรับการสร้างกรณีทดสอบอัตโนมัติจำนวน 2 อัลกอริทึม ได้แก่ Botsing และ CATG รวมถึงการศึกษการประยุกต์ใช้ปัญญาประดิษฐ์เชิงสร้างสรรค์จำนวน 2 เครื่องมือ ได้แก่ Deepseek และ Gemini โดยใช้โปรเจกต์ภาษา Java จากชุดข้อมูล Defects4J เป็นกรณีศึกษา การทดลองครอบคลุมกระบวนการสร้างชุดทดสอบ การนำชุดทดสอบไปดำเนินการกับโปรเจกต์ที่มีข้อบกพร่อง และการประเมินผลด้วยตัวชี้วัดที่เกี่ยวข้อง เช่น จำนวนกรณีทดสอบที่สร้างได้ ความสามารถในการตรวจพบข้อบกพร่อง (Fault Detection) ความครอบคลุมของโค้ด (Code Coverage) และเวลาในการสร้างชุดทดสอบ เพื่อใช้ในการวิเคราะห์และเปรียบเทียบลักษณะการทำงานของแต่ละแนวทางอย่างเป็นระบบ

ผลการดำเนินงานที่ได้ถูกนำมาวิเคราะห์ในด้านความสามารถในการสร้างชุดทดสอบ การตรวจพบข้อบกพร่อง ความครอบคลุมของโค้ด และประสิทธิภาพด้านเวลา รวมถึงพิจารณาข้อจำกัดและปัจจัยที่ส่งผลต่อผลการทดลองของแต่ละแนวทาง ผลการศึกษาสามารถนำมาใช้เป็นข้อมูลประกอบการทำความเข้าใจความแตกต่างระหว่างการสร้างกรณีทดสอบด้วยอัลกอริทึมแบบอัตโนมัติและการประยุกต์ใช้ปัญญาประดิษฐ์เชิงสร้างสรรค์ ตลอดจนเป็นแนวทางในการเลือกใช้และออกแบบกระบวนการสร้างชุดทดสอบให้เหมาะสมกับลักษณะของงานทดสอบซอฟต์แวร์

Niphada Yayanan, Sasiwitra Wongrungarunlert, **Report on the Study of Automatic Test Case Generation Algorithms and Generative AI Prompt Design for Software Testing.**

Computer Project, Bachelor of Science, Program in Computer Science, College of Computing, Khon Kaen University.

Presented to: Asst. Prof. Chitsutha Soomlek, Ph.D.

ABSTRACT

This report was prepared as part of the Software Quality Assurance course (CP353201) during the first semester of the 2026 academic year. The objective of this report is to study and evaluate approaches to automated test case generation by comparing the use of Automatic Test Case Generation algorithms with the application of Generative Artificial Intelligence (Generative AI) for generating test suites and test cases for unit testing.

The study consists of the investigation and application of two Automatic Test Case Generation algorithms, namely Botsing and CATG, as well as two Generative AI tools, namely Deepseek and Gemini. Java projects from the Defects4J dataset were used as the case study. The experiments covered the generation of test suites, execution of the generated test suites against defective projects, and evaluation using relevant metrics, including the number of generated test cases, fault detection capability, code coverage, and test generation time. These metrics were used to systematically analyze and compare the characteristics and performance of each approach.

The results were analyzed in terms of test generation capability, fault detection, code coverage, and time performance, as well as the limitations and factors affecting the experimental results of each approach. The findings provide information for understanding the differences between algorithm-based automatic test case generation and the application of Generative AI. Furthermore, the results can serve as a guideline for designing and applying test generation processes appropriately according to the characteristics of software testing tasks.

กิตติกรรมประกาศ

รายงานฉบับนี้สำเร็จลุล่วงได้ด้วยความรู้และคำแนะนำจาก ผศ.ดร.จิตสุธา สุ่มเล็ก อาจารย์ผู้สอนรายวิชาการประกันคุณภาพซอฟต์แวร์ รหัสวิชา CP353201 ที่ได้ให้ความรู้ที่เป็นประโยชน์ในการศึกษาและการทำงานของรายงานฉบับนี้

คณะผู้จัดทำขอขอบคุณรายวิชาการประกันคุณภาพซอฟต์แวร์ที่เปิดโอกาสให้ได้ศึกษาและประยุกต์ใช้ความรู้เกี่ยวกับการสร้างกรณีทดสอบอัตโนมัติ การทดสอบซอฟต์แวร์ และการประยุกต์ใช้ปัญญาประดิษฐ์เชิงสร้างสรรค์ในการสร้างชุดทดสอบ ซึ่งช่วยให้คณะผู้จัดทำได้พัฒนาความรู้และทักษะด้านการประกันคุณภาพซอฟต์แวร์มากยิ่งขึ้น

คณะผู้จัดทำขอขอบคุณสมาชิกในชั้นเรียนที่ให้คำแนะนำที่เป็นประโยชน์ และขอขอบคุณสมาชิกในกลุ่มที่ได้ให้ความร่วมมือในการศึกษา ดำเนินการทดลอง จัดเก็บและวิเคราะห์ผล ตลอดจนจัดทำรายงานฉบับนี้จนสำเร็จลุล่วง หากรายงานฉบับนี้มีข้อผิดพลาดหรือข้อบกพร่องประการใด คณะผู้จัดทำขอน้อมรับคำแนะนำและข้อเสนอแนะเพื่อนำไปปรับปรุงให้มีความถูกต้องและสมบูรณ์ยิ่งขึ้น

คณะผู้จัดทำ

สารบัญ

บทคัดย่อ	ก
ABSTRACT	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญตาราง	ฉ
สารบัญภาพ	ช
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของโครงการ	2
1.4 เครื่องมือและเทคโนโลยีที่ใช้	2
1.5 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2 ทฤษฎีและงานที่เกี่ยวข้อง	4
2.1 Defects4J	4
2.2 การทดสอบซอฟต์แวร์ (Software Testing)	5
2.3 การทดสอบระดับหน่วย (Unit Testing)	7
2.4 JUnit	8
2.5 การสร้างกรณีทดสอบโดยอัตโนมัติ (Automatic Test Case Generation: ATCG)	10
2.6 Botsing Algorithm	11
2.7 CATG (Combined Automated Test Generation) Algorithm	16
2.8 Gemini	20
2.9 DeepSeek	21
บทที่ 3 วิธีการดำเนินงาน	22
3.1 ภาพรวมการทดลอง	22
3.2 Experimental Environment	22
3.3 Dataset: Defects4J	23

3.4 Botsing	24
3.5 CATG	29
3.6 DeepSeek	40
3.7 Gemini	41
3.8 Prompt Design	42
3.9 Experimental Procedure	44
3.10 Experimental Configuration	44
3.11 Evaluation Metrics	45
บทที่ 4 การวิเคราะห์ ออกแบบและพัฒนาระบบ	46
4.1 ผลการทดลอง Botsing	46
4.2 ผลการทดลอง CATG	50
4.3 ผลการทดลอง DeepSeek	72
4.4 ผลการทดลอง Gemini	75
4.5 ผลการวัด Test Coverage	78
4.6 ผลการวัด Fault Detection	78
4.7 ผลการวัดประสิทธิภาพ	78
4.8 สรุปผลการทดลอง	79
บทที่ 5 สรุปผลการดำเนินงาน	81
5.1 สรุปผลการดำเนินโครงการ	81
5.2 สรุปผลการทดลอง	81
5.3 การเปรียบเทียบผลการทดลอง	82
5.4 ข้อจำกัดของโครงการ	83
5.5 ข้อเสนอแนะและแนวทางการพัฒนาต่อ	84
เอกสารอ้างอิง	85

สารบัญตาราง

ตารางที่ 1	สรุปงานวิจัยและแนวทางที่เกี่ยวข้องกับ Defects4J	4
ตารางที่ 2	สรุป JUnit Annotation และหน้าที่ในการทดสอบ	8
ตารางที่ 3	สรุป JUnit Assertion และหน้าที่ในการตรวจสอบผลการทดสอบ	10
ตารางที่ 4	สภาพแวดล้อมและเครื่องมือที่ใช้ในการทดลอง	23
ตารางที่ 5	สรุปการกำหนดค่าพารามิเตอร์สำหรับ Botsing	25
ตารางที่ 6	สรุปเงื่อนไขและแผนการดำเนินการทดลองของ Botsing	28
ตารางที่ 7	ค่าการตั้งค่าสภาพแวดล้อมและพารามิเตอร์ที่ใช้ในการทดลองด้วย CATG	30
ตารางที่ 8	คำศัพท์และความหมายที่ใช้ในการสร้าง Test Case ด้วย CATG	32
ตารางที่ 9	ไฟล์และโฟลเดอร์ที่เกี่ยวข้องกับกระบวนการสร้าง Test Case ด้วย CATG	33
ตารางที่ 10	สาเหตุที่ไม่สามารถสร้าง Test Case ด้วย CATG ได้	34
ตารางที่ 11	ตัวชี้วัดที่ใช้ในการประเมินกรณีทดสอบที่สร้างขึ้น	36
ตารางที่ 12	ขั้นตอนการดำเนินการทดลองและการจัดการผลการทดลองด้วย CATG	39
ตารางที่ 13	ขั้นตอนการทดลอง	44
ตารางที่ 14	การกำหนดค่าและสภาพแวดล้อมในการทดลอง	44
ตารางที่ 15	ตัวชี้วัดและเกณฑ์การประเมินผลการทดลอง	45
ตารางที่ 16	ผลการทดลองและตัวชี้วัดประสิทธิภาพของ Botsing บนบีก Lang_1b	49
ตารางที่ 17	ขอบเขตและเงื่อนไขการทดลองด้วย CATG	51
ตารางที่ 18	รายละเอียดข้อมูลผลการทดลองที่บันทึกในไฟล์ benchmark_results.csv ของ CATG	53
ตารางที่ 19	ผลสถานะการประมวลผลของ CATG	55
ตารางที่ 20	สถิติการสร้าง Test Case ของ CATG จาก Bug ที่มีสถานะ Completed	56
ตารางที่ 21	การกระจายจำนวนกรณีทดสอบที่ CATG สร้างได้ต่อ Bug	57
ตารางที่ 22	ผลการตรวจจับ Fault ของ CATG	60
ตารางที่ 23	ผลการวัด Code Coverage ของ CATG	61
ตารางที่ 24	สถิติเวลาในการสร้าง Test Case ของ CATG	63
ตารางที่ 25	เวลาในการสร้างกรณีทดสอบจำแนกตามจำนวน กรณีทดสอบ	64
ตารางที่ 26	ผลการทดลองจำแนกตามโปรเจกต์	65
ตารางที่ 27	ผลการตรวจจับบักและตัวชี้วัดการสร้างกรณีทดสอบจำแนกตามโปรเจกต์	66
ตารางที่ 28	สาเหตุของการประเมินผลไม่สำเร็จ	67

ตารางที่ 29 การจำแนกสาเหตุของผลการทดลองที่ไม่สำเร็จ	69
ตารางที่ 30 การจำแนกสาเหตุของสถานะ Unsupported	70
ตารางที่ 31 การจำแนกสาเหตุของสถานะ Error	71
ตารางที่ 32 ผลการรัน Test Case ที่สร้างโดย DeepSeek จำแนกตามสถานะ	73
ตารางที่ 33 สรุปสถานะการรัน Test Case จำแนกตามโปรเจกต์	77
ตารางที่ 34 เปรียบเทียบประสิทธิภาพของ Gemini และ DeepSeek	79

สารบัญภาพ

ภาพที่ 1 สถาปัตยกรรมและกระบวนการทำงาน	13
ภาพที่ 2 สถาปัตยกรรมและการไหลของข้อมูลของอัลกอริทึม CATG	17
ภาพที่ 3 การออกแบบ Prompt	43
ภาพที่ 4 ผลการคอมไพล์และการรันชุดทดสอบ Botsing รอบที่ 1	47
ภาพที่ 5 ผลการประมวลผล Botsing รอบที่ 2 และการรันยืนยันผลด้วย JUnit 4.12 บน Terminal	48
ภาพที่ 6 โครงสร้างชุดทดสอบ JUnit (NumberUtils_ESTest.java) ที่สร้างโดย Botsing	49
ภาพที่ 7 ตัวอย่างข้อมูลผลการทดลองที่บันทึกในไฟล์ benchmark_results.csv	52
ภาพที่ 8 ตัวอย่างข้อมูลผลการทดลองของ Bug จากไฟล์ benchmark_results.csv ของ CATG	53
ภาพที่ 9 สถานะการประมวลผลของ CATG จาก benchmark_results.csv	55
ภาพที่ 10 ผลการคำนวณสถิติจำนวนกรณีทดสอบต่อ Bug ด้วย PowerShell	56
ภาพที่ 11 ผลการจัดกลุ่มจำนวนกรณีทดสอบที่สร้างได้ต่อ Bug	57

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

การประกันคุณภาพซอฟต์แวร์ (Software Quality Assurance: SQA) เป็นกระบวนการสำคัญที่ช่วยให้ซอฟต์แวร์มีความถูกต้อง น่าเชื่อถือ และสามารถทำงานได้ตามข้อกำหนดที่กำหนดไว้ หนึ่งในกระบวนการสำคัญของการประกันคุณภาพซอฟต์แวร์คือ การทดสอบระดับหน่วย (Unit Testing) ซึ่งมุ่งเน้นตรวจสอบความถูกต้องของส่วนประกอบย่อยของโปรแกรม เช่น เมธอดหรือคลาส อย่างไรก็ตาม การออกแบบกรณีทดสอบด้วยตนเอง (Manual) โดยนักพัฒนาหรือผู้ทดสอบอาจต้องใช้เวลาและทรัพยากรจำนวนมาก โดยเฉพาะซอฟต์แวร์ที่มีความซับซ้อน

จากข้อจำกัดดังกล่าว จึงเกิดแนวคิดการสร้างกรณีทดสอบอัตโนมัติ ซึ่งเป็นแนวทางในการสร้างชุดทดสอบและกรณีทดสอบโดยอัตโนมัติ เพื่อลดภาระของผู้ทดสอบและเพิ่มประสิทธิภาพของกระบวนการทดสอบซอฟต์แวร์ โดยเทคนิคการสร้างกรณีทดสอบอัตโนมัติสามารถดำเนินการได้หลายแนวทาง เช่น การทดสอบซอฟต์แวร์โดยอาศัยการค้นหา (Search-Based Software Testing: SBST) นอกจากแนวทางเชิงอัลกอริทึมแล้ว ปัจจุบันยังมีการนำปัญญาประดิษฐ์เชิงสร้างสรรค์ (Generative AI) และเครื่องมือช่วยพัฒนาซอฟต์แวร์ (AI-Assisting Tools) มาใช้เพื่อช่วยในการสร้างกรณีทดสอบ โดยสามารถอาศัยคำสั่งหรือพรอมพ์ (Prompt) เพื่อให้เครื่องมือวิเคราะห์โค้ดและสร้างชุดทดสอบที่เหมาะสมกับโปรแกรมที่ต้องการทดสอบ แนวทางดังกล่าวช่วยเพิ่มทางเลือกในการสร้างกรณีทดสอบ และสามารถนำมาเปรียบเทียบกับวิธีการสร้างกรณีทดสอบแบบอัตโนมัติที่อาศัยอัลกอริทึมได้

รายงานฉบับนี้มุ่งศึกษาและประเมินแนวทางการสร้างกรณีทดสอบระดับหน่วย โดยพิจารณาแนวทางเชิงอัลกอริทึม ได้แก่ Botsing และ CATG ร่วมกับการใช้ปัญญาประดิษฐ์เชิงสร้างสรรค์ ได้แก่ Deepseek และ Gemini โดยใช้ชุดข้อมูล Defects4J ซึ่งเป็นชุดข้อมูลสำหรับการศึกษาข้อบกพร่องของซอฟต์แวร์ภาษา Java การศึกษาครั้งนี้ครอบคลุมหลักการทำงานของอัลกอริทึม แนวทางการสร้างชุดทดสอบและกรณีทดสอบ การออกแบบพรอมพ์สำหรับเครื่องมือปัญญาประดิษฐ์เชิงสร้างสรรค์ ตลอดจนการทดลองและประเมินผลโดยใช้ตัวชี้วัดที่เกี่ยวข้องกับการสร้างชุดทดสอบ การตรวจพบข้อบกพร่อง และความครอบคลุมของโค้ด

1.2 วัตถุประสงค์

- 1) เพื่อศึกษาและทำความเข้าใจหลักการทำงานของ Botsing Algorithm และ CATG Algorithm สำหรับการสร้างชุดทดสอบ (Test Suite) และกรณีทดสอบ (Test Case) ระดับ Unit Testing สำหรับโครงการภาษา Java ในชุดข้อมูล Defects4J Dataset
- 2) เพื่อศึกษาแนวทางการประยุกต์ใช้ Deepseek และ Gemini สำหรับการสร้างชุดทดสอบและกรณีทดสอบระดับ Unit Testing สำหรับโครงการภาษา Java ในชุดข้อมูล Defects4J Dataset รวมถึงศึกษาและออกแบบ Prompt ที่เหมาะสมสำหรับการสร้างกรณีทดสอบ
- 3) เพื่อศึกษาแนวทางและเกณฑ์ที่ใช้ในการประเมินคุณภาพและประสิทธิภาพของชุดทดสอบและกรณีทดสอบที่สร้างขึ้น เช่น Fault Detection Rate, Branch Coverage และ Code Coverage Ratio เพื่อเตรียมความพร้อมสำหรับการประเมินผลในขั้นตอนการทดลอง

- 4) เพื่อศึกษาแนวทางการเปรียบเทียบ Automatic Test Case Generation Algorithms กับ Generative AI/AI-Assisting Tools ในด้านความสามารถในการสร้างชุดทดสอบและกรณีทดสอบ รวมถึงวิเคราะห์ข้อดีข้อจำกัด และความเหมาะสมของแต่ละแนวทาง เพื่อใช้เป็นแนวทางในการดำเนินการทดลองในขั้นตอนต่อไป

1.3 ขอบเขตของโครงการ

- 1) ศึกษาและทดลองใช้ Automatic Test Case Generation Algorithms จำนวน 2 อัลกอริทึม ได้แก่ Botsing และ CATG
- 2) ศึกษาและทดลองใช้ Generative AI จำนวน 2 เครื่องมือ ได้แก่ Deepseek และ Gemini สำหรับการสร้างชุดทดสอบและกรณีทดสอบระดับ Unit Testing
- 3) ใช้ชุดข้อมูล Defects4J ซึ่งประกอบด้วยโครงการซอฟต์แวร์ภาษา Java ที่มีข้อบกพร่องที่กำหนดไว้เป็นกรณีศึกษาในการทดลอง
- 4) มุ่งเน้นการสร้างและประเมินชุดทดสอบระดับ Unit Testing โดยใช้ JUnit ตามข้อกำหนดและสภาพแวดล้อมที่เหมาะสม
- 5) สำหรับ Generative AI มีการออกแบบ Prompt เพื่อกำหนดบริบทของโปรแกรม ข้อกำหนดของการสร้าง Test Case รูปแบบผลลัพธ์ และเงื่อนไขที่เกี่ยวข้องกับการทดสอบ
- 6) ประเมินผลการทดลองโดยพิจารณาตัวชี้วัด เช่น จำนวน Test Case ที่สร้างได้ ความสามารถในการตรวจพบข้อบกพร่อง (Fault Detection) ความครอบคลุมของโค้ด (Code Coverage) และเวลาในการสร้างชุดทดสอบ
- 7) เปรียบเทียบผลการทดลองของ Automatic Test Case Generation Algorithms และ Generative AI/AI-Assisting Tools ตามตัวชี้วัดและเงื่อนไขที่กำหนดในการทดลอง

1.4 เครื่องมือและเทคโนโลยีที่ใช้

ในการดำเนินการศึกษาและทดลอง คณะผู้จัดทำได้เลือกใช้เครื่องมือและเทคโนโลยีที่เกี่ยวข้องกับการสร้างกรณีทดสอบอัตโนมัติ การทดสอบซอฟต์แวร์ และการประเมินผลการทดลอง โดยครอบคลุมทั้งเครื่องมือสำหรับการสร้างชุดทดสอบด้วยอัลกอริทึม เครื่องมือปัญญาประดิษฐ์เชิงสร้างสรรค์ ตลอดจนเครื่องมือสำหรับการพัฒนา การดำเนินการทดสอบ การวัดความครอบคลุมของโค้ด และการจัดการโครงการ ดังต่อไปนี้

- 1) Defects4J เป็นชุดข้อมูลสำหรับการทดลองและประเมินการทดสอบซอฟต์แวร์ภาษา Java ที่มีโครงการและข้อบกพร่องที่กำหนดไว้
- 2) Botsing เครื่องมือสำหรับการสร้างกรณีทดสอบอัตโนมัติโดยใช้แนวทางการค้นหาเชิงวิวัฒนาการ (Evolutionary Search)
- 3) CATG เครื่องมือสำหรับการสร้างกรณีทดสอบอัตโนมัติโดยใช้แนวทาง Concolic Testing
- 4) Deepseek เครื่องมือปัญญาประดิษฐ์เชิงสร้างสรรค์ที่ใช้สำหรับช่วยสร้างชุดทดสอบและกรณีทดสอบจากบริบทและ Prompt ที่กำหนด
- 5) Gemini เครื่องมือปัญญาประดิษฐ์เชิงสร้างสรรค์ที่ใช้สำหรับช่วยสร้างชุดทดสอบและกรณีทดสอบจากบริบทและ Prompt ที่กำหนด

- 6) Java ภาษาโปรแกรมที่ใช้ในการศึกษาและทดลองกับโครงการในชุดข้อมูล Defects4J
- 7) JUnit เฟรมเวิร์กสำหรับการสร้างและดำเนินการทดสอบระดับหน่วยของโปรแกรมภาษา Java
- 8) Git และ GitHub ใช้สำหรับจัดการเวอร์ชันของซอร์สโค้ด จัดเก็บผลการทดลอง และบันทึกการดำเนินงานของโครงการ

1.5 ประโยชน์ที่คาดว่าจะได้รับ

จากการศึกษาและดำเนินการทดลองสร้างกรณีทดสอบอัตโนมัติในการสร้างชุดทดสอบและกรณีทดสอบ คณะผู้จัดทำคาดหวังว่าจะได้รับความรู้และประสบการณ์ทั้งในด้านการสร้างกรณีทดสอบ การประเมินผลการทดสอบ และการวิเคราะห์ผลการทดลอง โดยมีประโยชน์ที่คาดว่าจะได้รับดังต่อไปนี้

- 1) ได้ความรู้และความเข้าใจเกี่ยวกับหลักการทำงานของ Automatic Test Case Generation Algorithms และ Generative AI สำหรับการสร้างกรณีทดสอบระดับ Unit Testing
- 2) สามารถประยุกต์ใช้ Botsing และ CATG ในการสร้างชุดทดสอบสำหรับโครงการภาษา Java ในชุดข้อมูล Defects4J
- 3) สามารถประยุกต์ใช้ Deepseek และ Gemini รวมถึงการออกแบบ Prompt เพื่อช่วยสร้างชุดทดสอบและกรณีทดสอบสำหรับการทดสอบซอฟต์แวร์
- 4) สามารถประเมินชุดทดสอบที่สร้างขึ้นโดยพิจารณาจากความสามารถในการตรวจพบข้อบกพร่อง ครอบคลุมของโค้ด และประสิทธิภาพด้านเวลา

บทที่ 2

ทฤษฎีและงานที่เกี่ยวข้อง

2.1 Defects4J

Defects4J ถูกสร้างขึ้นเพื่อเป็นชุดข้อมูลมาตรฐานของข้อผิดพลาดจริงในโปรแกรม Java ซึ่งงานวิจัยที่เกี่ยวข้องครอบคลุมสามประเด็นหลัก ได้แก่ สมมติฐานพื้นฐานทางทฤษฎี งานวิจัยการทดสอบและซ่อมแซมโปรแกรมอัตโนมัติ และข้อจำกัดด้านความถูกต้องของข้อมูล

สมมติฐานพื้นฐานและการออกแบบชุดข้อมูล

ชุดข้อมูล Defects4J ถูกออกแบบมาเพื่อแก้ปัญหาข้อจำกัดของการทดสอบด้วยโค้ดจำลองหรือ artificial mutants โดยรวบรวมบักจริงที่ผ่านการคัดแยกอย่างเป็นระบบ โดยมีหลักการและการค้นพบเชิงโครงสร้างดังนี้: การแยกข้อผิดพลาดเดี่ยว (Single Fault Assumption) ถูกกำหนดขึ้นโดยการตัดโค้ดที่ไม่เกี่ยวข้องออกเพื่อให้แต่ละบักเกิดจากสาเหตุรากฐานที่แยกจากกันอย่างชัดเจน

โครงสร้างของแพตช์ส่วนใหญ่มีขนาดกะทัดรัด โดยมี **ค่ามัธยฐานขนาด 4 บรรทัด** และ 92% ของแพตช์มีการแก้ไขเพียงไฟล์เดียวเท่านั้น

การซ่อมแซมส่วนใหญ่อิงสมมติฐาน Redundancy Assumption ซึ่งอาศัยโค้ดหรือรูปแบบที่มีอยู่เดิม แต่พบว่าครึ่งหนึ่งของบักไม่สามารถแก้ไขได้เพราะขาดชิ้นส่วนบริจาค (donor code)

งานวิจัยการประเมินและการซ่อมแซมโปรแกรมอัตโนมัติ Defects4J ทำหน้าที่เป็นเกณฑ์มาตรฐานหลักสำหรับ Automated Program Repair (APR) และ Fault Localization (FL) ในภาษา Java โดยถูกใช้เพื่อวัดประสิทธิภาพของเครื่องมือค้นหาแพตช์และจัดอันดับตำแหน่งข้อผิดพลาด

ตารางที่ 1 สรุปงานวิจัยและแนวทางที่เกี่ยวข้องกับ Defects4J

ขอบเขตงานวิจัย	แนวทางและเครื่องมือหลัก	ข้อค้นพบสำคัญบน Defects4J
Traditional APR	jGenProg, jKali, Nopol	ระบบสามารถสร้างแพตช์ที่ผ่านชุดทดสอบได้ แต่มีเพียงส่วนน้อยที่เป็นแพตช์ถูกต้องแท้จริง
Patch Feature Analysis	การวิเคราะห์ Syntax และ Pattern	แพตช์ที่แก้บรรทัดเดียวมักสร้างแพตช์ถูกต้องได้ดีกว่าการเพิ่ม try/catch หรือ method call

Learning-based & LLM APR	Large Language Models	LLM แก้ไขบั๊กได้จำนวนมากและลด overfitting ใน Defects4J แต่ความสามารถลดลงเมื่อเจอโค้ดที่ผ่านการแปลงรูป
Fault Localization	Spectrum-based Fault Localization (SBFL)	ใช้ชุดทดสอบที่ระบุความล้มเหลวเพื่อคำนวณอันดับความน่าจะเป็นของตำแหน่งข้อผิดพลาด
Benchmark Generation	BugBuilder, GrowingBugs	นำวิธีดัดแปลงบั๊กแบบ Defects4J มาทำเป็นระบบอัตโนมัติเพื่อลดแรงงานคน

ข้อจำกัด ความสะอาดของข้อมูล และความสามารถในการทำซ้ำ

ชุดทดสอบที่มาพร้อมกับบั๊กก็มีข้อมูลของนักพัฒนาแฝงอยู่ (developer knowledge) โดยพบว่า 55% ของชุดทดสอบที่กระตุ้นบั๊กถูกเขียนขึ้นมาใหม่เพื่อจำลองความผิดพลาด และ 22% ถูกดัดแปลงหลังมีการรายงานบั๊ก เมื่อนำความรู้ของนักพัฒนาเหล่านี้ออก ประสิทธิภาพของเทคนิค Spectrum-based Fault Localization (SBFL) ลดลงอย่างมากถึง 90% ถึง 415% ในแง่ Mean First Rank นอกจากนี้ การคัดแยกบั๊กให้เป็นข้อผิดพลาดเดี่ยวอาจไม่สะท้อนสภาพแวดล้อมจริง เนื่องจากการทดสอบข้ามเวอร์ชันพบว่า 95.4% ของโปรแกรมใน Defects4J แท้จริงแล้วมีหลายบั๊กอยู่ร่วมกัน (multi-faults) อีกทั้งยังพบปัญหาชุดทดสอบไม่รัดกุมพอ (under-specified test suites) ทำให้แพตช์ที่ไม่สมบูรณ์สามารถผ่านการทดสอบได้ ตลอดจนปัญหาความสามารถในการทำซ้ำ (reproducibility) จากการเปลี่ยนแปลงของสภาพแวดล้อมตามกาลเวลา

2.2 การทดสอบซอฟต์แวร์ (Software Testing)

การทดสอบซอฟต์แวร์ (Software Testing) เป็นกระบวนการประเมินและตรวจสอบซอฟต์แวร์หรือแอปพลิเคชัน เพื่อตรวจสอบว่าซอฟต์แวร์นั้น ๆ สามารถทำงานได้อย่างถูกต้อง ปลอดภัย และมีประสิทธิภาพ ซึ่งสามารถเปรียบเทียบได้กับข้อกำหนดที่กำหนดไว้ ในการทดสอบมีบทบาทสำคัญในการค้นหาข้อผิดพลาดและช่วยปรับปรุงประสิทธิภาพของซอฟต์แวร์ ซึ่งส่งผลต่อการพัฒนาซอฟต์แวร์ที่มีคุณภาพและสามารถตอบสนองต่อความต้องการของผู้ใช้งานได้ (Susnjara & Smalley, 2026)

ปัจจุบันการทดสอบซอฟต์แวร์เป็นส่วนหนึ่งของกระบวนการพัฒนาซอฟต์แวร์สมัยใหม่ โดยมีความเกี่ยวข้องกับแนวทางการพัฒนา เช่น Agile, DevOps และ Continuous Integration/Continuous Delivery (CI/CD) การทดสอบซอฟต์แวร์จึงไม่ได้ถูกจำกัดเฉพาะขั้นตอนสุดท้ายก่อนการเผยแพร่ซอฟต์แวร์เนื่องจากรูปแบบการทำงานที่แตกต่างต่างข้างต้นนั้นสามารถเริ่มต้นตั้งแต่ขั้นตอนการวางแผนและออกแบบ และดำเนินต่อเนื่องหลังจากนำซอฟต์แวร์ไปใช้งานจริงได้ โดยแนวทางกระบวนการทำงานนี้ส่งผลให้สามารถค้นพบปัญหาได้เร็วขึ้น ลดความเสี่ยง และเพื่อให้สามารถส่งมอบผลิตภัณฑ์ที่มีประสิทธิภาพได้

นอกจากนี้ แนวทางการทดสอบซอฟต์แวร์ในปัจจุบันมีการพัฒนาไปพร้อมกับเทคโนโลยีด้านระบบอัตโนมัติ (Automation) , ปัญญาประดิษฐ์ (Artificial Intelligence (AI)) และ สถาปัตยกรรมแบบคลาวด์เนทีฟ (Cloud-native Architecture) ส่งผลให้มีการนำระบบอัตโนมัติและเทคโนโลยีปัญญาประดิษฐ์เข้ามาช่วยในกระบวนการทดสอบมากขึ้น โดยเฉพาะในระบบซอฟต์แวร์ที่มีความซับซ้อนและมีรอบการพัฒนาที่รวดเร็ว (Susnjara & Smalley, 2026)

สำหรับการศึกษาและทดลองในครั้งนี้ การทดสอบซอฟต์แวร์จะมุ่งเน้นการทดสอบในระดับหน่วย (Unit Testing) ซึ่งเป็นการทดสอบส่วนย่อยของโปรแกรม เช่น เมธอดหรือคลาส จากชุดข้อมูล Defects4J เพื่อตรวจสอบการทำงานของแต่ละส่วนว่ามีผลลัพธ์เป็นไปตามที่คาดหวังหรือไม่ โดยการทดสอบในระดับหน่วยสามารถดำเนินการผ่าน Test Case และ Test Suite ซึ่งเป็นองค์ประกอบสำคัญในการตรวจสอบพฤติกรรมของซอฟต์แวร์และค้นหาข้อผิดพลาดที่เกิดขึ้นภายในโปรแกรม ดังนั้น การสร้าง Test Case ที่มีประสิทธิภาพจึงมีส่วนสำคัญต่อความสามารถในการตรวจจับข้อผิดพลาดของซอฟต์แวร์

2.2.1 กรณีทดสอบ (Test Case) คือ ชุดของขั้นตอนหรือคำสั่งที่ใช้กำหนดขึ้นเพื่อตรวจสอบว่าซอฟต์แวร์สามารถทำงานได้ตามที่คาดหวังหรือไม่ โดยทั่วไปของกรณีทดสอบจะประกอบด้วย

- Test Case ID คือ รหัสประจำกรณีทดสอบ
- Description คือ รายละเอียดสิ่งที่ต้องการทดสอบ
- Pre-conditions คือ เงื่อนไขหรือสถานะก่อนเริ่มทำการทดสอบ
- Test Steps คือ ขั้นตอนการปฏิบัติงานทีละลำดับ
- Test Data คือ ข้อมูลที่ใช้ป้อนเข้าระบบ
- Expected Result คือ ผลลัพธ์ที่คาดว่าจะเกิดขึ้น
- Actual Result คือ ผลลัพธ์จริงที่เกิดขึ้นหลังทดสอบ
- Status คือ ผลสรุปของกรณีทดสอบว่าผ่าน (Pass) ไม่ผ่าน (Fail) หรือยังไม่ทดสอบ (No run)

ซึ่งใช้เป็นเกณฑ์ในการเปรียบเทียบกับผลลัพธ์ที่เกิดขึ้นจริงจากการทำงานของซอฟต์แวร์ กรณีทดสอบส่งผลกระทบสำคัญสำหรับการค้นหาข้อผิดพลาดและปัญหาที่เกิดขึ้นในซอฟต์แวร์ รวมถึงช่วยให้สามารถตรวจสอบการทำงานของซอฟต์แวร์ได้อย่างเป็นระบบขั้นตอน (Devaraj, 2026)

2.2.2 ชุดทดสอบ (Test Suite)

ชุดทดสอบ คือ ชุดของกรณีทดสอบ (Test Cases) หรือสคริปต์ทดสอบที่ถูกรวบรวมไว้เข้าด้วยกัน เพื่อใช้สำหรับรันและตรวจสอบว่าระบบหรือซอฟต์แวร์ยังคงทำงานได้ถูกต้องตามข้อกำหนด (Requirements) ทั้งหมดหรือไม่ (Gupta, 2026) โดยชุดทดสอบมักใช้ในการจัดกลุ่มการทดสอบตามเป้าหมาย เช่น Smoke / Sanity Test Suite ใช้รวม Test Cases สำคัญเพื่อเช็กเบื้องต้นว่าระบบหลักไม่พังหลังการ Deploy, Regression Test Suite เป็นการรวม Test Cases ทั้งหมดเพื่อตรวจว่าการแก้ไขใหม่ไม่ได้ไปส่งผลกระทบต่อฟีเจอร์เดิมเสียหาย และ Functional Test Suite รวมกรณีทดสอบที่เน้นฟังก์ชันการทำงานของระบบตามธุรกิจ (Business Logic)

2.2.3 สถานการณ์การทดสอบ (Test Scenario)

สถานการณ์การทดสอบ คือ สถานการณ์หรือเงื่อนไขในภาพรวมที่กำหนดขึ้นเพื่อระบุว่าซอฟต์แวร์หรือฟังก์ชันใดควรถูกทดสอบ โดยมุ่งเน้นการระบุสิ่งที่ต้องการทดสอบมากกว่าการระบุขั้นตอนการทดสอบโดยละเอียด (Understand Test Case and Scenario, 2025) สถานการณ์ทดสอบสามารถใช้เป็นแนวทางในการกำหนดกรณี

ทดสอบที่เหมาะสมสำหรับตรวจสอบการทำงานของซอฟต์แวร์ในสถานการณ์ต่าง ๆ ได้ เช่น หากต้องการทดสอบระบบเข้าสู่ระบบ (Login) สามารถกำหนด สถานการณ์ทดสอบว่า “ตรวจสอบการเข้าสู่ระบบของผู้ใช้งาน” จากสถานการณ์ดังกล่าวสามารถสร้างกรณีทดสอบได้หลายกรณี เช่น การเข้าสู่ระบบด้วยชื่อผู้ใช้และรหัสผ่านที่ถูกต้อง การเข้าสู่ระบบด้วยรหัสผ่านไม่ถูกต้อง การไม่กรอกข้อมูล และการกรอกข้อมูลไม่ครบถ้วน

ดังนั้น Test Scenario จึงเป็นแนวคิดในระดับภาพรวมที่ช่วยกำหนดขอบเขตของการทดสอบ ขณะที่ Test Case จะกำหนดรายละเอียดของวิธีการทดสอบและผลลัพธ์ที่คาดหวัง และ Test Suite จะเป็นกลุ่มของ Test Case ที่รวบรวมไว้สำหรับดำเนินการทดสอบ

2.2.4 การตรวจจับข้อผิดพลาด (Fault Detection / Bug Detection)

การตรวจจับข้อผิดพลาด (Fault Detection) คือ ความสามารถของกรณีทดสอบหรือชุดทดสอบในการค้นพบข้อผิดพลาดที่มีอยู่ภายในซอฟต์แวร์ โดยเมื่อโปรแกรมทำงานภายใต้เงื่อนไขที่กำหนดและให้ผลลัพธ์แตกต่างจากผลลัพธ์ที่คาดหวัง จะสามารถระบุได้ว่าการทดสอบดังกล่าวสามารถตรวจพบพฤติกรรมที่ผิดปกติของโปรแกรมได้ (Singh & Sharma, 2018) การตรวจจับข้อผิดพลาดจึงเป็นองค์ประกอบสำคัญในการประเมินประสิทธิภาพของชุดทดสอบ เนื่องจากการที่ชุดทดสอบสามารถทำงานได้สำเร็จเพียงอย่างเดียวไม่ได้หมายความว่าสามารถค้นพบข้อผิดพลาดที่มีอยู่ในโปรแกรมได้ทั้งหมด

โดยในการศึกษานี้ การตรวจจับข้อผิดพลาดพิจารณาจากการนำชุดทดสอบไปดำเนินการกับโปรแกรมที่มีข้อผิดพลาด (Buggy Version) และโปรแกรมที่ได้รับการแก้ไขข้อผิดพลาดแล้ว (Fixed Version) หากชุดทดสอบสามารถทำให้การทดสอบของ Buggy Version เกิดความล้มเหลว ในขณะที่การทดสอบเดียวกันสามารถทำงานผ่านบน Fixed Version จะถือว่าชุดทดสอบดังกล่าวสามารถตรวจจับข้อผิดพลาดของโปรแกรมนั้นได้ แนวทางดังกล่าวช่วยให้สามารถประเมินความสามารถของ Test Suite ที่สร้างขึ้นจากวิธีการหรือเครื่องมือที่แตกต่างกันได้อย่างเป็นระบบ

2.3 การทดสอบระดับหน่วย (Unit Testing)

การทดสอบระดับหน่วยเป็นการทดสอบซอฟต์แวร์ในระดับหน่วยย่อยของโปรแกรม เช่น เมธอด (Method) ฟังก์ชัน (Function) หรือคลาส (Class) โดยมีวัตถุประสงค์เพื่อตรวจสอบว่าหน่วยของโปรแกรมแต่ละส่วนสามารถทำงานได้ถูกต้องตามพฤติกรรมหรือผลลัพธ์ที่คาดหวัง (Powell & Smalley, n.d.) การทดสอบในระดับหน่วยช่วยให้สามารถค้นหาข้อผิดพลาดของโปรแกรมได้ตั้งแต่ระยะเริ่มต้นของกระบวนการพัฒนา และช่วยแยกตรวจสอบปัญหาที่เกิดขึ้นในแต่ละส่วนของโปรแกรมได้อย่างเป็นระบบทั้งยังสามารถค้นหาข้อผิดพลาดได้โดยเร็ว

นอกจากนี้การทดสอบระดับหน่วยยังมีความเกี่ยวข้องกับแนวคิด Shift-Left Testing ซึ่งเป็นแนวทางที่นำกิจกรรมการทดสอบเข้ามาดำเนินการตั้งแต่ระยะแรกของ Software Development Life Cycle (SDLC) เพื่อช่วยตรวจพบและแก้ไขข้อผิดพลาดก่อนที่ซอฟต์แวร์จะเข้าสู่ขั้นตอนที่ซับซ้อนมากขึ้น นอกจากนี้ การแยกหน่วยที่ต้องการทดสอบออกจากส่วนอื่นของระบบยังช่วยให้สามารถตรวจสอบพฤติกรรมของแต่ละหน่วยได้อย่างเฉพาะเจาะจง (Powell & Smalley, n.d.)

สำหรับการศึกษานี้ Unit Testing เป็นรูปแบบการทดสอบหลักที่ใช้ในการทดลอง เนื่องจาก Test Case ที่สร้างขึ้นจากเครื่องมือ Automated Test Case Generation และเครื่องมือ AI จะถูกนำไปใช้ทดสอบหน่วยของโปรแกรม Java ในชุดข้อมูล Defects4J เพื่อประเมินความสามารถของชุดทดสอบในการตรวจจับข้อผิดพลาดและครอบคลุมโค้ดของโปรแกรม

2.4 JUnit

JUnit เป็นเฟรมเวิร์กสำหรับการเขียนและดำเนินการทดสอบโปรแกรมภาษา Java ซึ่ง JUnit รองรับ Java ตั้งแต่เวอร์ชันที่ 17 เป็นต้นไป โดยสนับสนุนการสร้าง Unit Test เพื่อใช้ตรวจสอบการทำงานของเมธอดหรือส่วนต่าง ๆ ของโปรแกรม ซึ่งผู้พัฒนาสามารถกำหนด Test Case และใช้ Assertion เพื่อตรวจสอบว่าผลลัพธ์ที่เกิดขึ้นตรงกับผลลัพธ์ที่คาดหวังหรือไม่ (JUnit, n.d.) ในการศึกษาครั้งนี้ JUnit ถูกใช้เป็น Framework สำหรับดำเนินการ Test Case ที่สร้างขึ้นกับโปรแกรม Java ใน Defects4J โดยผลจากการ Execute Test Case สามารถนำมาใช้ประกอบการประเมิน เช่น ผลการทดสอบผ่านหรือไม่ผ่าน และความสามารถในการตรวจจับ Fault

2.4.1 JUnit 5

JUnit 5 เป็นเวอร์ชันของ JUnit ที่ใช้ Annotation เพื่อกำหนดบทบาทของเมธอดภายใน Test Class ทำให้สามารถสร้างและจัดการ Test Case ได้อย่างเป็นระบบ โดยโครงสร้างพื้นฐานของ JUnit 5 ประกอบด้วย Test Class ซึ่งภายในประกอบด้วย Test Method อย่างน้อยหนึ่งเมธอด และสามารถกำหนดเมธอดสำหรับเตรียมสภาพแวดล้อมก่อนการทดสอบหรือทำความสะอาดหลังการทดสอบได้ ตัวอย่างโครงสร้าง Test Case ของ JUnit 5 ดังนี้

```
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest {

    @Test
    public void testAddition() {
        int result = 2 + 3;
        assertEquals(5, result);
    }

}
```

2.4.3 Annotation

Annotation คือ คำกำกับหรือตัวระบุ ที่เขียนไว้เหนือเมธอด คลาส หรือส่วนต่าง ๆ ของโค้ด เพื่อบอกให้ Java หรือ Framework ทราบว่าโค้ดส่วนนั้นมีหน้าที่หรือพฤติกรรมอย่างไร สำหรับ JUnit ใช้มีการ Annotation เพื่อระบุบทบาทของเมธอดภายใน Test Class (GeeksforGeeks, 2026) โดย Annotation ที่สำคัญ ดังตารางที่ 2

ตารางที่ 2 สรุป JUnit Annotation และหน้าที่ในการทดสอบ

Annotation	หน้าที่
@Test	ระบุว่าเมธอดนั้นเป็น Test Method ที่ต้องการให้ JUnit ดำเนินการทดสอบ
@Before	ดำเนินการก่อน Test Method แต่ละรายการ ใช้สำหรับเตรียมข้อมูลหรือสภาพแวดล้อม
@After	ดำเนินการหลัง Test Method แต่ละรายการ ใช้สำหรับทำความสะอาดทรัพยากร
@BeforeClass	ดำเนินการหนึ่งครั้งก่อน Test Method ทั้งหมดใน Test Class
@AfterClass	ดำเนินการหนึ่งครั้งหลัง Test Method ทั้งหมดใน Test Class

สำหรับโครงงานนี้ Annotation ที่พบโดยตรงใน Test Case ที่สร้างด้วย JUnit 5 โดยเฉพาะ @Test มีความสำคัญ เนื่องจากใช้กำหนด Test Method ที่ JUnit จะนำไป Execute

2.4.4 Assertion

Assertion คือ เมธอดที่ใช้ตรวจสอบว่าผลลัพธ์ที่ได้จากการทดสอบตรงตามผลลัพธ์ที่คาดหวังหรือเงื่อนไขที่กำหนดไว้หรือไม่ โดย JUnit มีเมธอดสำหรับตรวจสอบผลการทดสอบหลายรูปแบบ เช่น assertEquals() สำหรับเปรียบเทียบค่าที่คาดหวังกับค่าที่เกิดขึ้นจริง และ assertTrue() สำหรับตรวจสอบว่าเงื่อนไขที่กำหนดเป็นจริง ใน JUnit Jupiter ซึ่งเป็นส่วนหนึ่งของ JUnit 5 เมธอดสำหรับ Assertion จะอยู่ในคลาส org.junit.jupiter.api.Assertions และเป็น Static Method ทำให้สามารถเรียกใช้งานได้โดยตรงโดยไม่ต้องสร้าง Object ของคลาสดังกล่าว นอกจากนี้ Assertion ยังสามารถกำหนดข้อความเพื่อแสดงเมื่อการตรวจสอบไม่ผ่านได้ เพื่อช่วยในการวิเคราะห์สาเหตุของความล้มเหลวของกรณีทดสอบ ตัวอย่างการใช้งาน assertEquals() ดังนี้

@Test

```
public void testAddition() {
    int result = calculator.add(2, 3);
    assertEquals(5, result);
}
```

จากตัวอย่าง 5 คือค่าที่คาดหวัง (Expected Value) และ result คือค่าที่เกิดขึ้นจริง (Actual Value) หากค่าทั้งสองเท่ากัน การทดสอบจะผ่าน แต่หากไม่เท่ากัน Assertion จะล้มเหลวและ Test Case จะถูกระบุว่าไม่ผ่าน ตัวอย่าง Assertion ที่สำคัญ ดังตารางที่ ซึ่งแสดงตัวอย่างคำสั่ง Assertion หน้าที่ และตัวอย่าง

ตารางที่ 3 สรุป JUnit Assertion และหน้าที่ในการตรวจสอบผลการทดสอบ

Assertion	หน้าที่	ตัวอย่าง
assertEquals	ตรวจสอบว่าค่าที่คาดหวังและค่าที่เกิดขึ้นจริงเท่ากัน	assertEquals(5, result);
assertTrue	ตรวจสอบว่าเงื่อนไขเป็นจริง	assertTrue(result > 0);
assertFalse	ตรวจสอบว่าเงื่อนไขเป็นเท็จ	assertFalse(result < 0);
assertNull	ตรวจสอบว่าค่าเป็น null	assertNull(result);
assertNotNull	ตรวจสอบว่าค่าไม่เป็น null	assertNotNull(result);

2.5 การสร้างกรณีทดสอบโดยอัตโนมัติ (Automatic Test Case Generation: ATCG)

การสร้างกรณีทดสอบโดยอัตโนมัติ (Automatic Test Case Generation: ATCG) เป็นกระบวนการที่ใช้เทคนิคหรืออัลกอริทึมในการสร้างกรณีทดสอบโดยอัตโนมัติจากข้อมูลของซอฟต์แวร์ เช่น Source Code โครงสร้างของโปรแกรม ข้อกำหนด หรือเงื่อนไขในการทำงาน โดยมีเป้าหมายเพื่อสร้างกรณีทดสอบที่สามารถนำไปตรวจสอบพฤติกรรมของโปรแกรมและค้นหาข้อผิดพลาดได้ กระบวนการนี้ช่วยลดงานที่ต้องสร้างกรณีทดสอบด้วยตนเอง และสามารถสร้างกรณีทดสอบจำนวนมากเพื่อสำรวจพฤติกรรมหรือเส้นทางการทำงานของโปรแกรมที่หลากหลาย (PractiTest Team, 2024)

2.5.1 แนวคิดในการสร้างกรณีทดสอบอัตโนมัติ

กระบวนการสร้างกรณีทดสอบอัตโนมัติเริ่มจากการกำหนดเป้าหมายของการทดสอบ เช่น การเพิ่ม Statement Coverage หรือ Decision Coverage จากนั้นเครื่องมือจะนำข้อมูลของซอฟต์แวร์ เช่น Source Code โครงสร้างการควบคุมการทำงาน และข้อมูลที่เกี่ยวข้องกับข้อกำหนด มาใช้เป็นข้อมูลสำหรับสร้างกรณีทดสอบเมื่อสร้างกรณีทดสอบแล้ว จะสามารถนำไปดำเนินการกับโปรแกรมและประเมินผลตามเป้าหมายที่กำหนดไว้ เช่น Code Coverage หรือความสามารถในการตรวจพบ Fault (Shaikh et al., 2025)

โดยทั่วไปกระบวนการ ATCG สามารถสรุปได้ดังนี้ Source Code หรือข้อมูลของโปรแกรม จากนั้นนำไปวิเคราะห์โปรแกรมและกำหนดเป้าหมาย สร้าง Test Input และ Test Case แล้วนำไป Execute Test Case สุดท้ายจะเป็นการประเมินผลลัพธ์ โดยแต่ละเครื่องมืออาจใช้เทคนิคและอัลกอริทึมที่แตกต่างกันในการสร้างกรณีทดสอบ

2.5.2 เทคนิคในการสร้างกรณีทดสอบอัตโนมัติ

การสร้างกรณีทดสอบอัตโนมัติสามารถใช้เทคนิคได้หลายรูปแบบ โดยแต่ละเทคนิคมีแนวทางในการค้นหา Test Input และเส้นทางการทำงานของโปรแกรมแตกต่างกัน เช่น

- Random Testing เป็นการสร้าง Input หรือ Test Case แบบสุ่ม เพื่อสำรวจการทำงานของโปรแกรมในหลายรูปแบบ วิธีการนี้สามารถสร้าง Test Case ได้จำนวนมาก แต่ไม่สามารถรับประกันว่าจะครอบคลุมเงื่อนไขหรือเส้นทางที่สำคัญทั้งหมด

- Search-based Testing เป็นการใช้กระบวนการค้นหาเพื่อหา Test Case ที่เหมาะสมกับเป้าหมายที่กำหนด เช่น การเพิ่ม Code Coverage โดยอาจใช้แนวคิดของ Genetic Algorithm หรือ Evolutionary Algorithm เพื่อปรับปรุง Test Case ให้เข้าใกล้เป้าหมายมากขึ้น
- Symbolic Execution เป็นเทคนิคที่วิเคราะห์โปรแกรมโดยใช้ Symbolic Value แทนค่าจริงของตัวแปร และสร้างเงื่อนไขตามเส้นทางการทำงานของโปรแกรม เพื่อนำเงื่อนไขดังกล่าวไปใช้สร้าง Test Input ที่ทำให้โปรแกรมสามารถเข้าสู่เส้นทางที่ต้องการได้

โดยเทคนิคเหล่านี้สามารถนำมาใช้ร่วมกับวิธีการอื่นเพื่อเพิ่มความสามารถในการค้นหา Test Case ที่มีประสิทธิภาพ ทั้งนี้วิธีการที่เหมาะสมขึ้นอยู่กับลักษณะของโปรแกรม เป้าหมายของการทดสอบ และข้อจำกัดของเครื่องมือที่ใช้

2.5.3 ข้อดีและข้อจำกัดของการสร้างกรณีทดสอบอัตโนมัติ

การสร้างกรณีทดสอบอัตโนมัติมีข้อดีคือสามารถลดเวลาและแรงงานในการสร้าง Test Case จำนวนมาก ช่วยเพิ่มความสามารถในการสำรวจสถานการณ์การทำงานของโปรแกรม และสามารถกำหนดเป้าหมายด้าน Code Coverage เพื่อช่วยให้การทดสอบครอบคลุมได้มากขึ้น นอกจากนี้ Test Case ที่สร้างขึ้นสามารถนำไปดำเนินการซ้ำได้โดยอัตโนมัติ ทำให้เหมาะกับกระบวนการทดสอบที่ต้องดำเนินการอย่างต่อเนื่อง (PractiTest, 2024)

อย่างไรก็ตาม ATCG ยังมีข้อจำกัด เช่น Test Case ที่สร้างขึ้นอาจไม่สามารถครอบคลุมทุกสถานการณ์ที่สำคัญได้ เครื่องมือบางประเภทอาจมีข้อจำกัดในการจัดการกับโครงสร้างโปรแกรมหรือเงื่อนไขที่มีความซับซ้อน และ Test Case ที่มี Code Coverage สูงไม่ได้หมายความว่าจะสามารถตรวจพบ Fault ได้ทั้งหมด นอกจากนี้ การสร้าง Test Case อาจต้องใช้เวลาและทรัพยากรในการประมวลผล และอาจจำเป็นต้องตรวจสอบหรือปรับปรุง Test Case ที่สร้างขึ้นก่อนนำไปใช้งานจริง

2.5.4 การสร้างกรณีทดสอบอัตโนมัติและการสร้างกรณีทดสอบด้วยปัญญาประดิษฐ์

การสร้างกรณีทดสอบอัตโนมัติและการสร้างกรณีทดสอบด้วยปัญญาประดิษฐ์ (Automatic Test Case Generation and AI-assisted Test Generation) มีเป้าหมายคือการลดการสร้างกรณีทดสอบด้วยตนเอง แต่มีแนวทางในการสร้างกรณีทดสอบที่แตกต่างกัน โดย ATCG แบบที่ใช้ระบบอัลกอริทึมเป็นพื้นฐาน (Algorithm-based) มักอาศัยเทคนิคและอัลกอริทึมที่กำหนดกระบวนการค้นหา Test Input หรือ Test Case เช่น Search-based Testing และ Symbolic Execution ขณะที่การสร้างกรณีทดสอบด้วยปัญญาประดิษฐ์ (AI-assisted Test Generation) ใช้แบบจำลองปัญญาประดิษฐ์หรือ Generative AI เพื่อช่วยสร้าง Test Case จากข้อมูลที่ป้อนให้แบบจำลอง เช่น Source Code และ Prompt

ความแตกต่างดังกล่าวทำให้การเปรียบเทียบทั้งสองแนวทางสามารถพิจารณาได้จากผลลัพธ์ของ Test Case ที่สร้างขึ้น โดยใช้ตัวชี้วัดเดียวกัน เช่น Line Coverage, Branch Coverage, Fault Detection และ Test Generation Time เพื่อประเมินความสามารถของแต่ละแนวทางอย่างเป็นระบบ

2.6 Botsing Algorithm

Botsing ได้ใช้แนวทาง Search-based Software Testing (SBST) ซึ่งมีเป้าหมายในการสร้าง Test Case ที่สามารถทำให้เกิด Crash ตาม Stack Trace ที่กำหนดไว้ โดย Botsing รับ Stack Trace และโปรแกรมที่ต้องการทดสอบเป็นข้อมูลนำเข้า จากนั้นใช้กระบวนการค้นหาแบบ Single-objective หรือ Multi-objective เพื่อค้นหา Test Case ที่สามารถทำให้เกิด Crash ดังกล่าว (Derakhshanfar et al., 2020)

แนวทางของ Botsing ช่วยลดภาระในการสร้าง Test Case เพื่อทำซ้ำ Crash ด้วยตนเอง ซึ่งเป็นกระบวนการที่อาจใช้เวลานานและต้องอาศัยความเข้าใจเกี่ยวกับระบบของนักพัฒนา นอกจากนี้ Test Case ที่สามารถทำซ้ำ Crash ได้ยังสามารถนำไปใช้ในการวิเคราะห์และแก้ไขข้อผิดพลาด รวมถึงสามารถนำไปเพิ่มใน Test Suite เพื่อช่วยป้องกันการเกิด Regression ในอนาคต

2.6.1 แนวคิดหลักและวัตถุประสงค์

1) วัตถุประสงค์หลัก

มุ่งเน้นการสร้าง crash-reproducing test case จาก stack trace โดยอัตโนมัติ เพื่อช่วยสนับสนุนงานด้าน debugging และ fixing ของนักพัฒนาในภาคปฏิบัติ

2) การต่อยอดแนวคิด

สืบทอดแนวคิดจากสายงาน EvoCrash โดยใช้ evolutionary search ร่วมกับ Guided Genetic Algorithm แทนการเน้นเพิ่ม code coverage เพียงอย่างเดียว เนื่องจากกระบวนการ reproduce crash ต้องใช้วิธีการนำทาง (guidance) ที่เฉพาะเจาะจงต่อ exception และ stack trace เป้าหมาย

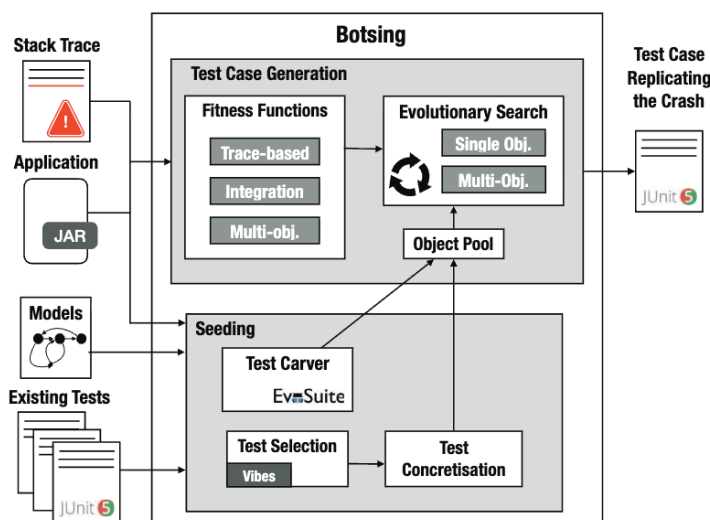
3) เป้าหมายพฤติกรรม

เน้นการสร้าง test case ที่จับพฤติกรรม crash ให้ตรงกับสถานะจริง (match crash behavior) มากกว่าการเพิ่มสัดส่วนการทดสอบโค้ดทั่วไป

4) ขอบเขตปัญหา

มุ่งเน้นการแก้ปัญหาคราซ (crashes) เป็นหลัก เนื่องจากกระบวนการย้อนรอยข้อผิดพลาดทั่วไป (general bug reproduction) จากข้อความรายงานมีความซับซ้อนและยากกว่า crash reproduction มาก

2.6.2 สถาปัตยกรรมและกระบวนการทำงาน (Architecture & Process Workflow)



ภาพที่ 1 สถาปัตยกรรมและกระบวนการทำงาน

1) สถาปัตยกรรมระบบ

Botsing ถูกออกแบบเป็น Extensible Framework บนระบบ EvoSuite โดยใช้ evosuite-client ทำหน้าที่ด้าน Code Instrumentation และจัดการโครงสร้างภายในของ Test Case สถาปัตยกรรมของ Botsing ประกอบด้วย 2 โมดูลหลัก ได้แก่

- 1.1) โมดูลสร้างเคสทดสอบ (Test Case Generation Module): ทำหน้าที่ขับเคลื่อนกระบวนการ Evolutionary Search (รองรับทั้งแบบ Single-Objective และ Multi-Objective) โดยประเมินคุณภาพของเคสทดสอบผ่าน Fitness Functions รูปแบบต่างๆ (เช่น Trace-based หรือ Integration)
- 1.2) โมดูลเสริมข้อมูลตั้งต้น (Seeding Module): ทำหน้าที่เตรียมวัตถุและลำดับการเรียกใช้เมธอดที่มีคุณภาพใส่ไว้ใน Object Pool โดยอาศัยการสกัดวัตถุจาก Test Case เดิมด้วย Test Carver (EvoSuite) ร่วมกับการคัดเลือกพฤติกรรมจากแบบจำลอง (Models) ด้วย Vibes แล้วนำมาแปลงเป็นวัตถุจริง (Test Concretisation)

2) กระบวนการทำงานหลัก (Process Workflow)

- 2.1) Input: รับข้อมูล Stack Trace, โปรแกรมที่อยู่ระหว่างการทดสอบ (Software Under Test: SUT ในรูปแบบ JAR File) รวมถึงข้อมูลเสริมอย่าง Models และ Existing Tests (หากเปิดใช้งาน Seeding) เข้าสู่ระบบ
 - 2.2) Initialization & Seeding: สุ่มสร้างประชากร Test Case เริ่มต้นเพื่อสำรวจ Search Space โดยหากเปิดใช้ระบบ Seeding ตัว Botsing จะดึงวัตถุและลำดับเมธอดจาก Object Pool มาช่วยประกอบเป็น Test Case เริ่มต้นที่มีบริบทสมจริงยิ่งขึ้น
 - 2.3) Evaluate Fitness: คำนวณประเมินค่า Fitness Function ของแต่ละ Test Case ในประชากร เพื่อวัดระดับความใกล้เคียงของการรันเทียบกับ Stack Trace เป้าหมาย
 - 2.4) Decision Check: ตรวจสอบว่ามี Test Case ใดสามารถทำให้เกิด Exception ชนิดเดียวกัน และมี Stack Trace ตรงตามเป้าหมายแล้วหรือยัง
 - 2.5) Genetic Operators Loop: หากยังไม่ประสบความสำเร็จ จะประยุกต์ใช้ Genetic Operators เช่น Crossover (ผสมสายพันธุ์) และ Guided Mutation (กลายพันธุ์) ร่วมกับกระบวนการซ่อมแซมเคสทดสอบ (Repair Operator) แล้ววนกลับไปประเมินค่า Fitness ใหม่จนกว่าจะสำเร็จหรือหมดเวลาที่กำหนด
 - 2.6) Output: ส่งออกชุดทดสอบ JUnit (เช่น JUnit 5) ที่สามารถจำลองการพัง (Crash Reproduction) ได้สำเร็จ เพื่อให้นักพัฒนานำไปใช้ตรวจสอบสาเหตุในขั้นตอนต่อไป
- 3) คุณสมบัติการขยายระบบ (Extensibility)

สถาปัตยกรรมมีความยืดหยุ่นสูง เปิดให้ผู้พัฒนาสามารถสับเปลี่ยนหรือเพิ่ม fitness function, genetic operators และ evolutionary algorithms ใหม่ ๆ เข้าไปในระบบได้ง่าย (Derakhshanfar et al., 2020; Derakhshanfar, 2021)

2.6.3 ขั้นตอนวิธีในการสร้าง Test Case อัตโนมัติ

- 1) **Search Strategy** อาศัย fitness-guided evolutionary search ในการประเมินระยะห่างระหว่างการทำงานของ test case กับจุดเกิด crash โดยรองรับทั้งรูปแบบ single-objective และ multi-objective
- 2) **Fitness Functions** WeightedSum: ค่า fitness เริ่มต้นแบบ single-objective ที่สืบทอดมาจาก EvoCrash Multi-objective Fitness: รูปแบบการประเมินที่แยกองค์ประกอบย่อยออกจากกันเพื่อเพิ่มมิติในการนำทาง

STDistance: ฟังก์ชันประเมินทางเลือกที่ทำการไล่ตรวจสอบความถูกต้องของเฟรมต่างๆ ใน stack trace ก่อน จากนั้นจึงนำชนิดของ exception มาพิจารณา

3) Seeding Strategies

Test Seeding: ทำการรันชุดทดสอบเดิมที่มีอยู่ในระบบเพื่อสกัดและบันทึกวัตถุ (objects) แล้วนำกลับมาใช้ใหม่ในขั้นตอนสร้าง test case

Behavioural Model Seeding: สร้างโครงสร้าง state-machine จากการใช้งานคลาส เพื่อสร้างลำดับการเรียก method ที่สมจริงและสอดคล้องกับพฤติกรรมระบบจริงในการขึ้นำการค้นหา

Secondary Objective (Basic Block Coverage - BBC): เพิ่มมิติการตรวจสอบ relevant basic blocks เข้ามาช่วยแยกแยะกรณี ที่ค่า approach level และ branch distance เท่ากัน ทำให้การค้นหาเงื่อนไขซับซ้อนแม่นยำขึ้น

2.6.4 ข้อดี และ ข้อจำกัด (Advantages & Limitations)

ข้อดี (Advantages):

- 1) อัตราความสำเร็จสูง: สามารถสร้าง test case เพื่อ reproduce crash ระดับที่แก้ยาก (hard-to-reproduce crashes) ได้สำเร็จ 66 จาก 124 กรณีในสภาวะปกติ และเพิ่มเป็น 70 กรณีเมื่อเปิดใช้งาน behavioural model seeding
- 2) ลดเวลาในการค้นหา: การประยุกต์ใช้เป้าหมายรองอย่าง BBC ช่วยเพิ่มจำนวน crash ที่สร้างได้สำเร็จ และลดเวลาในกระบวนการค้นหาอย่างมีนัยสำคัญ
- 3) ยืดหยุ่นต่อการวิจัย: สถาปัตยกรรมแบบเปิดเอื้อต่อการทดลองปรับแต่งตัวแปรและฟังก์ชันนำทางใหม่ๆ ได้สะดวก

ข้อจำกัด (Limitations):

- 1) ข้อจำกัดเชิงโครงสร้างโค้ด: ประสบความยากลำบากเมื่อเจอ search space ขนาดใหญ่, เงื่อนไขแบบ implicit branching, โครงสร้างข้อมูลที่สร้างยาก, หรือคลาสประเภท abstract และ anonymous classes
- 2) ปัญหาด้าน Runtime Environment: การรันบนระบบที่มีความซับซ้อนสูงหรือมีสภาพแวดล้อมจำเพาะ เช่น Elasticsearch ทำได้ยาก
- 3) ความไม่แน่นอนของ Seeding: ในบางกรณี การใช้ test seeding อาจให้ผลลัพธ์การค้นหาที่แย่กว่าการไม่ใช้ seeding

- 4) ขอบเขตการใช้งานจำกัด: เหมาะสมกับ crash ที่ระบุ stack trace ชัดเจน ไม่รองรับการแก้ปัญหาจากรายงาน bug ทั่วไป และไม่เหมาะสมกับสภาพแวดล้อมที่จัดการด้วย genetic operators ได้ยาก เช่น การจัดการสถานะใน Android Apps

2.7 CATG (Combined Automated Test Generation) Algorithm

2.7.1 แนวคิดหลักและวัตถุประสงค์

CATG (Combined Automated Test Generation) เป็นอัลกอริทึมที่ใช้แนวทาง Concolic Testing หรือ Dynamic Symbolic Execution ในการสำรวจเส้นทางการทำงานของโปรแกรม โดยแนวทางดังกล่าวเป็นการทำงานร่วมกันระหว่าง Concrete Execution ซึ่งเป็นการประมวลผลโปรแกรมด้วยค่าอินพุตจริง และ Symbolic Execution ซึ่งเป็นการติดตามค่าของตัวแปรในรูปแบบเชิงสัญลักษณ์และสร้างเงื่อนไขของเส้นทางการทำงาน เพื่อนำข้อมูลดังกล่าวไปสร้าง Test Input ที่สามารถทำให้โปรแกรมเข้าสู่เส้นทางการทำงานที่แตกต่างกันได้ (Tanno et al., 2015; Pham et al., 2019)

ในการทำงานของ Concolic Testing โปรแกรมจะเริ่มต้นจาก Test Input ชุดหนึ่ง จากนั้นระบบจะทำการประมวลผลโปรแกรมด้วยค่าจริงและติดตามเส้นทางที่โปรแกรมเลือกผ่านเงื่อนไขต่าง ๆ ระหว่างการทำงาน ระบบจะบันทึกเงื่อนไขของเส้นทางดังกล่าวเป็น Path Constraint จากนั้นจะเปลี่ยนแปลงเงื่อนไขบางส่วนของเส้นทาง เช่น การทำ Negation ของเงื่อนไข ณ จุด Branch และส่ง Constraint ที่ได้ไปยัง Constraint Solver เพื่อค้นหา Test Input ใหม่ที่สามารถทำให้โปรแกรมเข้าสู่เส้นทางอื่นได้ (Olsthoom, 2022)

กระบวนการดังกล่าวทำให้ระบบสามารถสร้าง Test Case เพิ่มเติมจากเส้นทางที่โปรแกรมเคยทำงาน และนำ Test Input ที่สร้างขึ้นกลับไปประมวลผลเพื่อสำรวจเส้นทางใหม่ กระบวนการจะดำเนินซ้ำจนกว่าจะไม่สามารถค้นหาเส้นทางใหม่ได้ หรือบรรลุเกณฑ์การทดสอบที่กำหนดไว้ แนวทางนี้ช่วยให้การสร้าง Test Case สามารถดำเนินการอย่างเป็นระบบ และสามารถนำไปใช้เพื่อเพิ่ม Code Coverage และตรวจสอบเส้นทางการทำงานของโปรแกรมได้

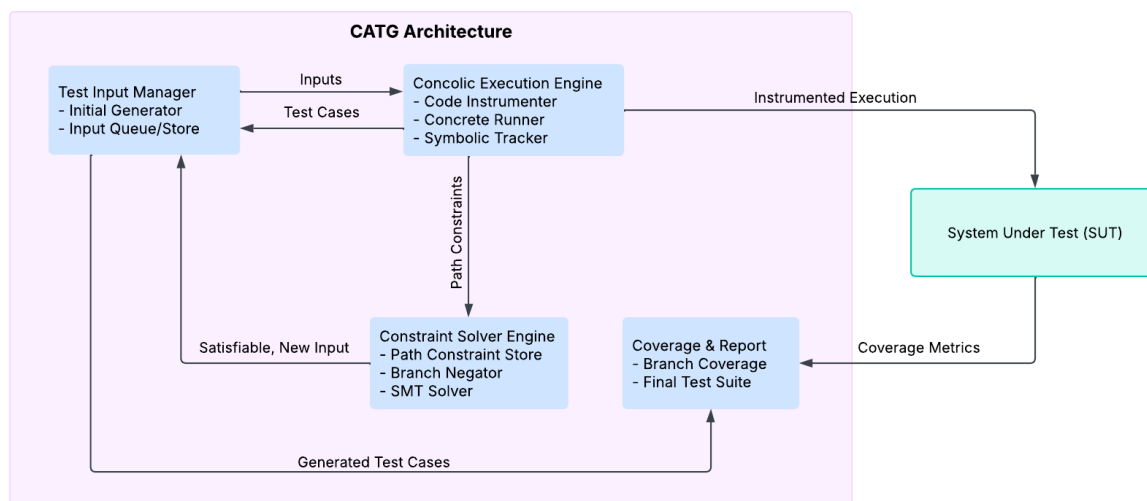
1. วัตถุประสงค์หลักของ CATG ได้แก่

- 1) สร้าง Test Case โดยอัตโนมัติ เพื่อลดความจำเป็นในการกำหนด Test Input ด้วยตนเอง
- 2) สำรวจเส้นทางการทำงานของโปรแกรม โดยสร้าง Test Input ที่ทำให้โปรแกรมเข้าสู่ Branch หรือเส้นทางการทำงานที่แตกต่างกัน
- 3) เพิ่ม Code Coverage โดยเฉพาะ Statement Coverage และ Branch Coverage ตามความสามารถและข้อจำกัดของโปรแกรมที่นำมาทดสอบ
- 4) ค้นหาเส้นทางการทำงานที่เข้าถึงได้ยาก ซึ่งอาจต้องใช้ Test Input ที่มีเงื่อนไขเฉพาะในการเข้าถึง
- 5) ลดเวลาและแรงงานในการสร้าง Test Input สำหรับการทดสอบโปรแกรม

ดังนั้น แนวคิดสำคัญของ CATG คือการใช้ข้อมูลจากการประมวลผลโปรแกรมจริงร่วมกับข้อมูลเชิงสัญลักษณ์ และ Constraint Solver เพื่อค้นหา Test Input ใหม่สำหรับสำรวจเส้นทางการทำงานของโปรแกรมอย่างเป็นระบบ (Sen et al., 2005)

2.7.2 สถาปัตยกรรมและกระบวนการทำงาน

สถาปัตยกรรมและการไหลของข้อมูลของอัลกอริทึม CATG (Combined/Concolic Automated Test Generation) ประกอบด้วยโมดูลการทำงานสำคัญ 4 ส่วนหลัก ได้แก่ Test Input Manager, Concolic Execution Engine, Constraint Solver Engine และ Coverage & Report ซึ่งทำงานร่วมกันแบบวงจรมุ่งเป้า (Closed-loop Architecture) เพื่อสร้างกรณีทดสอบอัตโนมัติสำหรับนำโปรแกรมเป้าหมาย (System Under Test: SUT) ไปสู่เส้นทางการทำงานที่หลากหลาย (Pradel, 2021)



ภาพที่ 2 สถาปัตยกรรมและการไหลของข้อมูลของอัลกอริทึม CATG

จากภาพที่ 2 กระบวนการทำงานเริ่มจาก Test Input Manager ทำหน้าที่สุ่มกำหนดค่าอินพุตให้แก่โปรแกรม จากนั้น Concolic Execution Engine จะทำการแทรกตัวดักจับสัญญาณ (Instrumentation) ลงใน SUT และสังเคราะห์โปรแกรมจริงผ่าน Concrete Execution ควบคู่ไปกับการรันจำลองเชิงสัญลักษณ์ผ่าน Symbolic Execution เพื่อติดตามเส้นทางการทำงานและบันทึกเงื่อนไขทางแยกที่เกิดขึ้นเป็นสมการข้อจำกัด (Path Constraints)

โดยสมการข้อจำกัดนี้จะถูกส่งต่อไปยัง Constraint Solver Engine ซึ่งทำหน้าที่สลับทิศทางเงื่อนไขทางแยก (Branch Negation) และใช้ SMT Solver ในการคำนวณแก้สมการเพื่อหาค่าอินพุตชุดใหม่ ที่จะบังคับให้โปรแกรมวิ่งไปยังเส้นทางที่ไม่เคยทดสอบ เมื่อแก้สมการสำเร็จ (Satisfiable) ค่าอินพุตใหม่จะถูกส่งกลับเข้าสู่ Test Input Manager เพื่อเข้าคิวรอรันในรอบถัดไป กระบวนการนี้จะดำเนินต่อเนื่องจนครอบคลุมของการทดสอบบรรลุ

เป้าหมาย โดยมี Coverage & Report คอยเก็บรวบรวมข้อมูลสถิติความครอบคลุม (Coverage Metrics) และสรุปชุดกรณีทดสอบทั้งหมด (Final Test Suite) เป็นขั้นตอนสุดท้าย

โดยภายในสถาปัตยกรรมดังกล่าว มีหัวใจสำคัญในการขับเคลื่อนด้วย 3 กระบวนการหลักทางอัลกอริทึม ได้แก่ Concrete Execution, Symbolic Execution และ Constraint Solving ซึ่งทำงานประสานกันผ่านโมดูลต่างๆ ดังรายละเอียดต่อไปนี้

- 1) Concrete Execution เป็นขั้นตอนที่โปรแกรมถูกประมวลผลด้วยค่า Input จริง (Concrete Input) เพื่อให้ทราบว่าโปรแกรมเลือกเส้นทางการทำงานใดในแต่ละเงื่อนไข
- 2) Symbolic Execution เป็นการติดตามค่าของตัวแปรในรูปแบบ Symbolic Value แทนการใช้ค่าจริงเพียงอย่างเดียว พร้อมทั้งบันทึกเงื่อนไขที่โปรแกรมพบระหว่างการทำงาน โดยเงื่อนไขเหล่านี้จะถูกนำมาสร้างเป็น Path Constraint ซึ่งแสดงข้อจำกัดของ Input ที่จำเป็นสำหรับการเข้าสู่เส้นทางนั้น
- 3) Constraint Solving เป็นขั้นตอนการนำ Path Constraint ที่ได้ไปยังตัวแก้สมการหรือ SMT Solver (Satisfiability Modulo Theories Solver) เพื่อค้นหาค่าของ Input ที่ทำให้เงื่อนไขที่กำหนดเป็นจริง หาก Solver สามารถหาคำตอบได้ ค่าที่ได้จะถูกนำไปสร้างเป็น Test Case ใหม่โดยกระบวนการดังกล่าวจะทำให้ทำงานกว่าจะไม่สามารถค้นหาเส้นทางใหม่หรือบรรลุเกณฑ์การทดสอบที่กำหนดไว้ได้

2.7.3 ขั้นตอนวิธีในการสร้าง Test Case อัตโนมัติ

CATG Algorithm สร้าง Test Case โดยการสำรวจเส้นทางการทำงานของโปรแกรมและสร้าง Input ใหม่จากการเปลี่ยนแปลงเงื่อนไขของเส้นทางเดิม กระบวนการหลักสามารถแบ่งออกเป็นขั้นตอนดังนี้

- 1) กำหนด Initial Test Input คือ การกำหนดค่า Input ชุดแรกสำหรับโปรแกรม เพื่อใช้เป็นจุดเริ่มต้นในการสำรวจเส้นทาง
- 2) ทำการ Concrete Execution นำ Input ดังกล่าวไปประมวลผลจริง และติดตามลำดับของ Branch หรือเงื่อนไขที่โปรแกรมเลือกในระหว่างการทำงาน
- 3) บันทึก Path Constraint ในแต่ละ Branch ระบบจะบันทึกเงื่อนไขที่เกี่ยวข้องกับเส้นทางที่โปรแกรมเลือก จากนั้นนำเงื่อนไขเหล่านั้นมารวมกันเป็น Path Constraint ของเส้นทางนั้น
- 4) เลือก Branch สำหรับสร้างเส้นทางใหม่ ระบบเลือกเงื่อนไขบางส่วนของ Path Constraint ที่ต้องการเปลี่ยนเส้นทาง โดยพิจารณาจาก Branch ที่ยังไม่เคยถูกสำรวจ
- 5) ทำการ Negation ของเงื่อนไข เงื่อนไขที่เลือกจะถูกกลับค่าความจริง การ Negate เงื่อนไขช่วยให้ระบบสามารถค้นหา Input ที่นำโปรแกรมเข้าสู่ Branch ฝั่งตรงข้ามได้

- 6) ส่ง Constraint ไปยัง SMT Solver โดย Path Constraint ที่ถูกปรับเปลี่ยนจะถูกส่งไปยัง SMT Solver เพื่อค้นหาค่าของตัวแปรที่ทำให้ Constraint ทั้งหมดสามารถเป็นจริงได้
- 7) สร้าง Test Case ใหม่ หาก Solver พบคำตอบ (Satisfiable) ค่าที่ได้จะถูกนำมาใช้เป็น Input สำหรับ Test Case ใหม่ หากไม่สามารถหาคำตอบได้ (Unsatisfiable) เส้นทางดังกล่าวจะไม่สามารถสร้าง Test Case ใหม่จาก Constraint ชุดนั้นได้
- 8) ทำซ้ำเพื่อสำรวจเส้นทางอื่น Test Case ใหม่จะถูกนำกลับไป Execute และสร้าง Path Constraint ชุดใหม่ กระบวนการจะดำเนินต่อไปจนกว่าจะครบตามเกณฑ์ Coverage หรือไม่สามารถค้นหาเส้นทางใหม่ได้

2.7.4 ข้อดีและข้อจำกัด

จากกระบวนการทำงานของ CATG Algorithm ที่อาศัยเทคนิค Concolic Testing (Dynamic Symbolic Execution) ร่วมกับการประมวลผลด้วยค่าจริง (Concrete Execution) และการวิเคราะห์เชิงสัญลักษณ์ (Symbolic Execution) ร่วมกับการใช้ Constraint Solver เพื่อสร้าง Test Input ใหม่ ทำให้ CATG สามารถสำรวจเส้นทางการทำงานของโปรแกรมและสร้าง Test Case ได้อย่างเป็นระบบ อย่างไรก็ตาม การดำเนินการดังกล่าวยังมีข้อจำกัดบางประการ โดยเฉพาะเมื่อโปรแกรมมีโครงสร้างซับซ้อนหรือมีจำนวนเส้นทางการทำงานเป็นจำนวนมาก ดังนั้นจึงสามารถสรุปข้อดีและข้อจำกัดของ CATG Algorithm ได้ดังนี้

ข้อดี

- 1) สามารถสร้าง Test Case แบบอัตโนมัติ ช่วยลดเวลาและแรงงานในการออกแบบ Test Case
- 2) เพิ่ม Code Coverage สามารถค้นหา Input ที่ทำให้โปรแกรมเข้าสู่ Branch หรือ Path ที่แตกต่างกันได้
- 3) ค้นหาเส้นทางที่เข้าถึงได้ยาก เนื่องจากใช้ Constraint Solver ในการคำนวณ Input ที่เหมาะสม
- 4) ลดการพึ่งพา Test Input ที่กำหนดโดยมนุษย์ โดยระบบสามารถสร้าง Input ใหม่จากเงื่อนไขของโปรแกรม
- 5) ตรวจสอบเงื่อนไขของโปรแกรมอย่างเป็นระบบ โดยเฉพาะโปรแกรมที่มี Conditional Branch จำนวนมาก
- 6) สามารถใช้ร่วมกับเทคนิค Software Testing อื่น ๆ เช่น White-Box Testing และ Code Coverage Analysis

ข้อจำกัด

- 1) สามารถสร้าง Test Case แบบอัตโนมัติ ช่วยลดเวลาและแรงงานในการออกแบบ Test Case
- 2) Path Explosion Problem เมื่อโปรแกรมมี Branch จำนวนมาก จำนวนเส้นทางที่เป็นไปได้สามารถเพิ่มขึ้นอย่างรวดเร็ว ทำให้การสำรวจทุกเส้นทางทำได้ยาก

- 3) ข้อจำกัดของ Constraint Solver หาก Constraint มีความซับซ้อนมาก Solver อาจใช้เวลานานในการหาคำตอบ หรือไม่สามารถแก้ปัญหาได้
- 4) ใช้ทรัพยากรในการประมวลผลสูง เนื่องจากต้อง Execute โปรแกรมหลายครั้งและเรียกใช้ Solver เพื่อสร้าง Input ใหม่
- 5) จัดการกับโปรแกรมที่มี External Dependencies ได้ยาก เช่น การติดต่อฐานข้อมูล ระบบเครือข่าย หรือบริการภายนอก เนื่องจากพฤติกรรมของระบบเหล่านี้อาจไม่สามารถแทนด้วย Constraint ได้โดยตรง
- 6) มีข้อจำกัดกับข้อมูลที่ไม่สามารถวิเคราะห์เชิงสัญลักษณ์ได้ง่าย เช่น String ที่มีการประมวลผลซับซ้อน หรือข้อมูลจากระบบภายนอก
- 7) ไม่สามารถรับประกันการครอบคลุมทุก Path ในโปรแกรมขนาดใหญ่ได้ เนื่องจากจำนวนเส้นทางอาจมีจำนวนมหาศาล

2.8 Gemini

Gemini เป็นระบบปัญญาประดิษฐ์เชิงสร้างสรรค์ (Generative Artificial Intelligence) ที่สามารถประมวลผลและสร้างเนื้อหาจากคำสั่งหรือพรอมต์ (Prompt) ที่ผู้ใช้กำหนด โดยสามารถนำมาประยุกต์ใช้กับงานด้านการเขียนและวิเคราะห์โปรแกรมคอมพิวเตอร์ได้ เช่น การสร้างโค้ด การอธิบายการทำงานของโค้ด การตรวจสอบข้อผิดพลาด และการปรับปรุงโค้ดให้เหมาะสมกับความต้องการของผู้ใช้ (Google AI for Developers, n.d.)

2.8.1 ความสามารถด้านการสร้างและอธิบายโค้ด

Gemini สามารถรับคำสั่งที่เกี่ยวข้องกับการเขียนโปรแกรมและสร้างโค้ดตามข้อกำหนดที่ระบุใน Prompt รวมถึงสามารถอธิบายโครงสร้างและลำดับการทำงานของโปรแกรม วิเคราะห์เงื่อนไขของเมธอด และเสนอแนวทางในการแก้ไขข้อผิดพลาดของโค้ด ความสามารถดังกล่าวทำให้สามารถนำ Gemini มาประยุกต์ใช้เป็นเครื่องมือช่วยในการพัฒนาและทดสอบซอฟต์แวร์ได้

2.8.2 การประยุกต์ใช้ในการสร้าง JUnit Test Case

ในการทดสอบซอฟต์แวร์ Gemini สามารถนำข้อมูลของคลาสหรือเมธอดเป้าหมาย รวมถึงข้อกำหนดที่ระบุใน Prompt มาใช้ในการสร้าง JUnit Test Case โดยสามารถสร้างโครงสร้างของ Test Method กำหนด Test Input และสร้าง Assertion เพื่อใช้ตรวจสอบผลลัพธ์ที่ได้จากการทำงานของโปรแกรม การนำ Gemini มาใช้ในลักษณะนี้ช่วยลดภาระในการเขียน Test Case ด้วยตนเอง และสามารถนำ Test Case ที่สร้างขึ้นไปตรวจสอบและประเมินผลด้วยเกณฑ์ต่าง ๆ (JaCoCo, n.d.) เช่น ความสามารถในการ Compile การทำงานของ Test Case และ Code Coverage

2.8.3 ข้อดีและข้อจำกัดในการสร้าง Test Case

ข้อดีของการใช้ Gemini ในการสร้าง Test Case คือ สามารถสร้างโค้ดทดสอบได้อย่างรวดเร็ว และสามารถปรับรูปแบบของ Test Case ตามคำสั่งหรือข้อกำหนดที่กำหนดใน Prompt นอกจากนี้ยังสามารถช่วยอธิบายเหตุผลของการสร้าง Test Case และเสนอ Test Input สำหรับกรณีต่าง ๆ ได้

อย่างไรก็ตาม Test Case ที่สร้างโดย Gemini อาจไม่สามารถ Compile หรือทำงานได้ถูกต้องทุกกรณี เนื่องจากผลลัพธ์ขึ้นอยู่กับข้อมูลที่ให้แก่ระบบและรายละเอียดของ Prompt รวมถึงความเข้าใจบริบทของโค้ดเป้าหมาย ดังนั้น Test Case ที่สร้างขึ้นจึงจำเป็นต้องผ่านการตรวจสอบและประเมินผลก่อนนำไปใช้

2.9 DeepSeek

DeepSeek เป็นระบบปัญญาประดิษฐ์เชิงสร้างสรรค์ที่สามารถประมวลผลคำสั่งภาษาธรรมชาติและสร้างเนื้อหาที่เกี่ยวข้องกับการเขียนโปรแกรมได้ โดยสามารถนำมาใช้ในการสร้าง วิเคราะห์ และอธิบายโค้ด รวมถึงประยุกต์ใช้ในงานด้าน Software Testing เช่น การสร้าง Test Case จากโค้ดหรือข้อกำหนดของโปรแกรม (Guo et al., 2024)

2.9.1 ความสามารถด้านการสร้างและอธิบายโค้ด

DeepSeek สามารถรับคำสั่งผ่าน Prompt และสร้างผลลัพธ์ที่เกี่ยวข้องกับงานเขียนโปรแกรมได้ โดยงานวิจัย DeepSeek-Coder ศึกษาความสามารถของโมเดลในงานด้านโค้ด (Guo et al., 2024) ในการใช้งาน ผู้ใช้อาจป้อนโค้ดและขอให้ระบบอธิบายหรือเสนอแนวทางแก้ไขได้ ทั้งนี้ ความถูกต้องและความเหมาะสมของคำตอบขึ้นกับโมเดล Prompt และบริบทที่ให้ไว้ จึงควรตรวจสอบผลลัพธ์กับโค้ดและข้อกำหนดจริง

2.9.2 การประยุกต์ใช้ในการสร้าง JUnit Test Case

DeepSeek สามารถนำข้อมูลของคลาสและเมธอดเป้าหมายมาใช้ในการสร้าง JUnit Test Case (JUnit Team, n.d.) โดยผู้ใช้งานสามารถกำหนดเงื่อนไขของการทดสอบ เช่น Test Input ผลลัพธ์ที่คาดหวัง และรูปแบบของ Assertion ผ่าน Prompt จากนั้นระบบจะสร้างโค้ด Test Case ตามข้อกำหนดที่ระบุ ซึ่ง Test Case ที่ได้สามารถนำไปตรวจสอบการ Compile และการทำงาน รวมถึงประเมินประสิทธิภาพด้วยตัวชี้วัด เช่น Line Coverage และ Branch Coverage

2.9.3 ข้อดีและข้อจำกัดในการสร้าง Test Case

ข้อดีของ DeepSeek คือสามารถช่วยสร้าง Test Case จากคำอธิบายหรือโค้ดที่กำหนดได้อย่างรวดเร็ว และสามารถสร้างโค้ดทดสอบที่มีรูปแบบแตกต่างกันตามคำสั่งที่ผู้ใช้กำหนด นอกจากนี้ยังสามารถช่วยวิเคราะห์เงื่อนไขของโปรแกรมเพื่อเสนอ Test Input สำหรับการทดสอบได้ (Autio et al., 2024)

อย่างไรก็ตาม ผลลัพธ์ที่สร้างโดย DeepSeek อาจมีความถูกต้องแตกต่างกันตามรายละเอียดของ Prompt และข้อมูลบริบทที่ป้อนให้ระบบ Test Case ที่สร้างขึ้นอาจมีข้อผิดพลาด เช่น อ้างอิงคลาสหรือเมธอดไม่ถูกต้อง กำหนด Assertion ไม่ตรงกับพฤติกรรมของโปรแกรม หรือไม่สามารถ Compile ได้ ดังนั้นจึงจำเป็นต้องตรวจสอบและประเมิน Test Case ที่สร้างขึ้นก่อนนำไปใช้ในการทดลอง

บทที่ 3

วิธีการดำเนินงาน

3.1 ภาพรวมการทดลอง

การทดลองนี้มีวัตถุประสงค์เพื่อศึกษาและเปรียบเทียบความสามารถในการสร้าง Test Case สำหรับตรวจสอบข้อผิดพลาดของซอฟต์แวร์ โดยแบ่งแนวทางการสร้าง Test Case ออกเป็น 2 กลุ่ม ได้แก่ Automatic Test Case Generation (ATCG) ซึ่งประกอบด้วย Botsing และ CATG และแนวทางที่ใช้ Generative AI เพื่อช่วยในการสร้าง Test Case ซึ่งประกอบด้วย DeepSeek และ Gemini โดยทุกแนวทางใช้ชุดข้อมูลข้อผิดพลาดจาก Defects4J เป็นข้อมูลสำหรับการทดลอง

กระบวนการทดลองเริ่มจากการเตรียม Dataset และข้อมูลของแต่ละ Bug รวมถึง Target Class และ Target Method ที่ใช้ในการทดสอบ จากนั้นดำเนินการสร้าง Test Case ด้วยเครื่องมือแต่ละประเภทตามวิธีการทำงานของเครื่องมือเหล่านั้น โดย Botsing และ CATG ใช้กระบวนการอัตโนมัติในการค้นหาและสร้าง Test Case ขณะที่ DeepSeek และ Gemini ใช้ Prompt ที่กำหนดขึ้นเพื่อให้ปัญญาประดิษฐ์ช่วยสร้าง JUnit Test Case จากข้อมูลของ Bug และ Target Method

หลังจากได้ Test Case จากแต่ละแนวทางแล้ว Test Case จะถูกนำไปทดสอบกับ Buggy Version และ Fixed Version ของ Bug เดียวกัน เพื่อประเมินความสามารถในการตรวจจับข้อผิดพลาด โดยพิจารณาจากผลการทำงานของ Test Case นอกจากนี้ยังประเมินตัวชี้วัดที่เกี่ยวข้อง ได้แก่ Fault Detection, Line Coverage, Branch/Condition Coverage และเวลาในการสร้าง Test Case (Generation Time) เพื่อใช้ในการวิเคราะห์และเปรียบเทียบผลการทดลอง

ผลการทดลองของแต่ละเครื่องมือจะถูกจัดเก็บและนำมาวิเคราะห์ทั้งในภาพรวมและจำแนกตามโปรเจกต์ของ Defects4J รวมถึงวิเคราะห์กรณีที่ไม่สามารถสร้างหรือประเมิน Test Case ได้ เพื่อระบุข้อจำกัดและสาเหตุที่เกิดขึ้นในแต่ละแนวทาง

3.2 Experimental Environment

การทดลองดำเนินการโดยสมาชิกในกลุ่มผ่านเครื่องคอมพิวเตอร์ที่มีสภาพแวดล้อมแตกต่างกันตามเครื่องมือที่ใช้ในการทดลอง โดยส่วนของ Automatic Test Case Generation ได้แก่ Botsing และ CATG ดำเนินการบนระบบปฏิบัติการ Windows ขณะที่ส่วนของ AI-Assisted Test Case Generation ได้แก่ DeepSeek และ Gemini ดำเนินการผ่านระบบปฏิบัติการ macOS ทั้งนี้ การประเมินผลของ Test Case ที่สร้างขึ้นใช้เกณฑ์และกระบวนการเดียวกัน ได้แก่ การทดสอบกับ Buggy Version และ Fixed Version การตรวจสอบ Fault Detection การวัด Code Coverage และการบันทึก Generation Time

เครื่องมือและซอฟต์แวร์หลักที่ใช้ในการทดลองประกอบด้วย Defects4J v3.0.1 สำหรับชุดข้อมูลข้อผิดพลาดจริง, JUnit สำหรับการสร้างและทดสอบ Test Case, Botsing และ CATG สำหรับ Automatic Test Case

Generation, DeepSeek และ Gemini สำหรับ AI-Assisted Test Case Generation, CVC4 1.8 สำหรับการแก้ Path Constraint ในการทำงานของ CATG, Gradle สำหรับการจัดการและคอมไพล์โปรเจกต์ และ Defects4J/Cobertura สำหรับการวัด Code Coverage

เนื่องจากเครื่องมือแต่ละประเภทมีข้อกำหนดด้านสภาพแวดล้อมแตกต่างกัน จึงมีการกำหนด Java Version และซอฟต์แวร์ที่เกี่ยวข้องตามความต้องการของแต่ละเครื่องมือ โดยรายละเอียดของสภาพแวดล้อมและเครื่องมือที่ใช้แสดงในตารางที่

ตารางที่ 4 สภาพแวดล้อมและเครื่องมือที่ใช้ในการทดลอง

รายการ	รายละเอียด
Operating System	Windows และ macOS
Programming Language	Java, Python
Dataset	Defects4J v3.0.1
Testing Framework	JUnit
ATCG Tools	Botsing, CATG
AI-Assisted Tools	DeepSeek, Gemini
Solver	CVC4 1.8
Build Tool	Gradle
Code Coverage	Defects4J/Cobertura

3.3 Dataset: Defects4J

ในการทดลองนี้เลือกใช้ Defects4J v3.0.1 เป็นชุดข้อมูลสำหรับการประเมินการสร้าง Test Case เนื่องจากเป็นชุดข้อมูลที่รวบรวมข้อผิดพลาดจริงจากโครงการซอฟต์แวร์โอเพนซอร์สที่พัฒนาด้วยภาษา Java โดยแต่ละ Bug ประกอบด้วย Buggy Version ซึ่งเป็นเวอร์ชันที่มีข้อผิดพลาด และ Fixed Version ซึ่งเป็นเวอร์ชันที่ได้รับการแก้ไขแล้ว ทำให้สามารถนำผลการทดสอบของทั้งสองเวอร์ชันมาเปรียบเทียบเพื่อประเมินความสามารถในการตรวจจับข้อผิดพลาดได้

ชุดข้อมูลที่ใช้ในการทดลองประกอบด้วย Bug จากโครงการ Java จำนวน 17 โครงการ โดยในการเตรียมข้อมูลได้จัดทำ Bug Manifest เพื่อระบุข้อมูลของแต่ละ Bug ที่จำเป็นต่อกระบวนการทดลอง ได้แก่ ชื่อโครงการ รหัส Bug, Target Class และ Target Method รวมถึงข้อมูลที่เกี่ยวข้องกับ Buggy Version และ Fixed Version

ในการสร้าง Test Case จะใช้ Target Method ของแต่ละ Bug เป็นเป้าหมายในการทดสอบ โดยข้อมูล Target Method ถูกนำไปใช้เป็นข้อมูลเริ่มต้นสำหรับเครื่องมือสร้าง Test Case ทั้ง Botsing, CATG, DeepSeek และ Gemini

ทั้งนี้ เครื่องมือแต่ละประเภทอาจมีวิธีการเตรียมข้อมูลและสร้าง Test Case แตกต่างกันไปตามลักษณะการทำงานของเครื่องมือ

หลังจากได้ Test Case แล้ว จะนำ Test Case ที่สร้างขึ้นไปทดสอบกับ Buggy Version และ Fixed Version ของ Bug เดียวกัน เพื่อใช้เปรียบเทียบพฤติกรรมของโปรแกรมและประเมินว่า Test Case สามารถตรวจจับข้อผิดพลาดได้หรือไม่ นอกจากนี้ Defects4J ยังถูกใช้ในขั้นตอนการทดสอบและการวัด Code Coverage เพื่อให้สามารถประเมินผลของ Test Case ภายใต้สภาพแวดล้อมของชุดข้อมูลเดียวกัน

3.4 Botsing

Botsing เป็นเครื่องมือสร้างชุดทดสอบแบบอัตโนมัติชนิดค้นหาเชิงวิวัฒนาการ (Search-Based Crash Reproduction Framework) ที่พัฒนาต่อยอดมาจาก EvoSuite โดยใช้ระบบ evosuite-client สำหรับทำ Code Instrumentation Botsing ทำงานด้วยอัลกอริทึมพันธุกรรมแบบชี้นำ (Guided Genetic Algorithm: GGA) โดยมีวัตถุประสงค์หลักเพื่อจำลองการเกิดข้อบกพร่องซ้ำ (Crash Reproduction) จากข้อมูลร่องรอยการพังของโปรแกรม (Stack Trace / Crash Log) เพื่อสร้าง Unit Test ระดับเฉพาะเจาะจงที่สามารถทำให้ระบบเกิด Exception ในตำแหน่งเดียวกับที่พบในสภาพแวดล้อมจริงได้อย่างแม่นยำ

ในการศึกษานี้ ได้นำ Botsing มาประยุกต์ใช้ในการจำลองและสร้าง Test Case เพื่อตรวจหาข้อบกพร่องในโครงการภาษา Java จากชุดข้อมูล Defects4J โดยมีรายละเอียดการกำหนดสภาพแวดล้อมพารามิเตอร์ ขั้นตอนการทำงาน และการประเมินผลตามโครงสร้างหัวเรื่องดังต่อไปนี้

3.4.1 การเตรียมสภาพแวดล้อม (Environment Preparation)

การทำงานของ Botsing จำเป็นต้องอาศัยสภาพแวดล้อมที่สอดคล้องกับโครงสร้างของ EvoSuite และชุดข้อมูล Defects4J โดยมีการจัดเตรียมสภาพแวดล้อมดังนี้:

- **Operating System:** รองรับการประมวลผลบนระบบปฏิบัติการ Windows และ macOS
- **Java Development Kit:** ใช้ Java 8 (OpenJDK 1.8.0) เนื่องจาก Botsing และ EvoSuite ทำงานร่วมกับ JDK 8 ได้อย่างมีประสิทธิภาพสูงสุด
- **Dependencies & Classpath:** ทำการดึงข้อมูล Classpath สำหรับการคอมไพล์และรันโปรแกรมเป้าหมายผ่านไฟล์ cp_compile.txt และโฟลเดอร์ target/classes ของแต่ละ Bug ใน Defects4J
- **Framework Integration:** ทำงานร่วมกับ evosuite-client เพื่อจัดการการทำ Code Instrumentation และโครงสร้างภายในของกรณีทดสอบ

3.4.2 การกำหนดค่าพารามิเตอร์ของ Botsing (Tool Configuration)

การกำหนดค่าพารามิเตอร์ (Tool Configuration) ของ Botsing

ตั้งค่าเพื่อควบคุมขอบเขตและประสิทธิภาพในการค้นหา:

- **Target Frame:** กำหนดตำแหน่งเฟรมเป้าหมายเป็น Frame 5 ซึ่งเป็นจุดกำเนิด Exception ใน Stack Trace (เช่น Exception ประเภท NumberFormatException)
- **Search Budget (งบประมาณเวลาในการค้นหา):**
 - **รอบที่ 1 (Round 1):** กำหนดระยะเวลาค้นหา 30 วินาที (search_budget = 30) เพื่อประเมินประสิทธิภาพและความเร็วเบื้องต้น
 - **รอบที่ 2 (Round 2):** กำหนดระยะเวลาค้นหา 60 วินาที (search_budget = 60) เพื่อศึกษาผลกระทบของการขยายเวลาต่ออัตราความสำเร็จในการจำลอง Crash
- **Test Minimization (minimize):** กำหนดค่าเป็น false เพื่อคงลักษณะขั้นตอนคำสั่งของ Test Case ไว้โดยไม่มีการย่อขนาดโค้ด
- **Search Algorithm:** ใช้ Guided Genetic Algorithm (GGA) ร่วมกับฟังก์ชันนำทาง (Fitness Function) เช่น WeightedSum หรือ STDistance

ตารางที่ 5 สรุปการกำหนดค่าพารามิเตอร์สำหรับ Botsing

พารามิเตอร์	ค่าที่กำหนด	คำอธิบาย
Target Frame	Frame 5	กำหนดตำแหน่งเป้าหมายที่เป็นจุดกำเนิด Exception ใน Stack Trace
Search Budget	30 วินาที (Round 1) 60 วินาที (Round 2)	จำกัดเวลาการค้นหาด้วย Guided Genetic Algorithm เพื่อประเมินความเร็วและอัตราสำเร็จ
Test Minimization	false	ปิดการย่อขนาดโค้ด เพื่อคงลำดับคำสั่งเดิมของ Test Case ไว้ทั้งหมด
Search Algorithm	GGA	ใช้ Guided Genetic Algorithm ร่วมกับ Fitness Function (WeightedSum / STDistance)

3.4.3 การเตรียมข้อมูลและการสกัด Stack Trace (Dataset & Crash Log Preparation)

ในกระบวนการเตรียมข้อมูลสำหรับการทดลองกับ Botsing มีขั้นตอนดังนี้:

- ดึงข้อมูล Bug จาก Defects4J (เช่น Lang_1b ในฟังก์ชัน `org.apache.commons.lang3.math.NumberUtils.createInteger`)
- สกัดข้อมูล Stack Trace หรือ Crash Log ของ Bug เป้าหมายเพื่อใช้เป็นข้อมูลนำเข้า (Input)
- ระบุ Target Class, Target Method และ Target Frame จาก Crash Log เพื่อใช้เป็นเป้าหมายหลักในการนำทางกระบวนการ Evolutionary Search
- ในกรณีที่มีการเปิดใช้งานระบบ Seeding จะมีการเตรียม Object Pool โดยสกัดวัตถุจาก Test Case เดิมด้วย Test Carver หรือคัดเลือกจากพฤติกรรมในแบบจำลอง (Models)

3.4.4 การสร้าง Test Case ด้วย Botsing (Test Case Generation Process)

กระบวนการสร้างกรณีทดสอบอัตโนมัติของ Botsing ดำเนินการผ่านขั้นตอนหลักดังนี้:

1. **Initialization & Seeding:** สุ่มสร้างประชากร Test Case เริ่มต้น หากเปิดใช้ Seeding จะดึงวัตถุและลำดับเมธอดจาก Object Pool มาประกอบเป็น Test Case เริ่มต้น
2. **Evaluate Fitness:** คำนวณประเมินค่า Fitness Function ของ Test Case แต่ละรายการ เพื่อวัดระยะห่างและความใกล้เคียงของการรันเทียบกับ Stack Trace เป้าหมาย
3. **Crash Check:** ตรวจสอบว่ามี Test Case ใดที่สามารถทำให้เกิด Exception ชนิดเดียวกัน และมี Stack Trace ตรงตามเป้าหมายแล้วหรือไม่
4. **Genetic Operators Loop:** หากยังไม่สำเร็จ จะประยุกต์ใช้ Crossover (การผสมสายพันธุ์), Guided Mutation (การกลายพันธุ์แบบชี้นำ) และ Repair Operator เพื่อปรับปรุง Test Case แล้ววนกลับไปประเมินค่า Fitness ใหม่
5. **Output Generation:** ส่งออกชุดทดสอบในรูปแบบ JUnit Test Suite ที่สามารถจำลอง Crash ได้สำเร็จ

3.4.5 การตรวจสอบกรณีทดสอบที่สร้างขึ้น (Test Case Compilation & Execution Validation)

กรณีทดสอบที่ Botsing สร้างขึ้นจะถูกนำมาตรวจสอบความถูกต้องของโครงสร้างและการทำงาน:

- ตรวจสอบความสามารถในการคอมไพล์ (Compilation Check) ร่วมกับคลาสเป้าหมายใน Defects4J
- ดำเนินการ Execute
ชุดทดสอบบนสภาพแวดล้อมการทดสอบเพื่อยืนยันว่าสคริปต์การทดสอบไม่มีข้อผิดพลาดทางไวยากรณ์ (Syntax/Runtime Error) ที่ไม่เกี่ยวข้องกับ Crash เป้าหมาย

3.4.6 การประเมินการตรวจพบข้อผิดพลาดและการทำซ้ำ Crash (Fault Detection & Crash Reproduction Evaluation)

การประเมินความสามารถในการตรวจพบข้อผิดพลาดของ Botsing พิจารณาจาก:

- นำชุดทดสอบ JUnit ที่สร้างได้ไปดำเนินการบน **Buggy Version** และ **Fixed Version** ของโปรเจกต์เดียวกัน
- **เกณฑ์การตรวจพบ Fault (Crash Reproduction):** ชุดทดสอบต้องทำให้เกิด Exception เดียวกันตรงตามตำแหน่ง Stack Trace เป้าหมายบน Buggy Version (เกิด Failure/Error) แต่ต้องสามารถทำงานผ่านได้อย่างถูกต้อง (PASS) บน Fixed Version

3.4.8 การบันทึกผลการทดลอง (Experiment Result Logging)

ผลการทดลองของ Botsing ในแต่ละ Bug จะถูกรวบรวมและบันทึกลงในไฟล์ผลลัพธ์ (.csv) โดยมีตัวชี้วัดสำคัญที่ถูกจัดเก็บ ได้แก่:

- สถานะการประมวลผล (เช่น Successfully Reproduced, Search Budget Exceeded, Error)
- เวลาที่ใช้ในการสร้าง Test Case (Generation Time ในหน่วยวินาที)
- จำนวนกรณีทดสอบ (Test Case Count) ที่สร้างขึ้น
- ผลการตรวจจับ Fault / การทำซ้ำ Crash (Fault Detection Rate)
- ค่า Line Coverage (%) และ Branch Coverage (%)

3.4.9 การกำหนดระยะเวลาสูงสุดในการประมวลผลและการจัดการข้อผิดพลาด (Timeout & Error Handling)

เพื่อควบคุมไม่ให้เกิดการทดลองค้างหรือใช้เวลามากเกินไป จึงมีการกำหนดเงื่อนไขขอบเขตเวลาและรองรับข้อผิดพลาดดังนี้:

- **Search Budget Timeout:** จำกัดเวลาการค้นหาค้นหาต่อ Bug ไว้ที่ 30 วินาที หรือ 60 วินาที ตามรอบการทดลอง
- **Error Handling:** ในกรณีที่ไม่สามารถสร้าง Crash ได้ภายในเวลาที่กำหนด หรือเจอเงื่อนไขซับซ้อน (เช่น Abstract Class, Anonymous Class หรือ Search Space ขนาดใหญ่) ระบบจะบันทึกสถานะการล้มเหลวพร้อมสาเหตุลงในไฟล์บันทึกผลการทดลอง แล้วจึงข้ามไปประมวลผล Bug ถัดไป

3.4.10 การดำเนินการทดลองและการจัดการผลการทดลอง (Experimental Execution)

การทดลองรัน Botsing จะดำเนินการอย่างเป็นระบบตามขั้นตอนต่อไปนี้:

1. เตรียมรายการ Bug และ Crash Log จาก Defects4J

2. รัน Botsing เพื่อค้นหาและสร้างกรณีทดสอบตามพารามิเตอร์ที่กำหนด (Round 1: 30s, Round 2: 60s)
3. นำ Test Case ที่ได้ไปทดสอบรันเทียบระหว่าง Buggy Version และ Fixed Version
4. วัดค่า Code Coverage
และรวบรวมข้อมูลประสิทธิภาพทั้งหมดเข้าสู่ไฟล์สถิติเพื่อนำไปวิเคราะห์เปรียบเทียบกับเครื่องมืออื่น ๆ
ในบทที่ 4 ต่อไป

ตารางที่ 6 สรุปเงื่อนไขและแผนการดำเนินการทดลองของ Botsing

รอบการทดลอง	Search Budget	วัตถุประสงค์การทดลอง	ตัวชี้วัดที่บันทึก (Output Metrics)
Round 1	30 วินาที	ประเมินประสิทธิภาพและความเร็วเบื้องต้นในการทำ Crash Reproduction	• สถานะการสร้าง Crash (Pass/Fail) • Generation Time (วินาที) • Line Coverage (%) • Branch Coverage (%)
Round 2	60 วินาที	ศึกษาผลกระทบของการเพิ่มเวลาค้นหาต่ออัตราความสำเร็จในการทำ Crash Reproduction	• สถานะการสร้าง Crash (Pass/Fail) • Generation Time (วินาที)

			• Line Coverage (%) • Branch Coverage (%)
--	--	--	--

3.5 CATG

CATG (Combined Automated Test Generation) Algorithm เป็นหนึ่งในเครื่องมือสำหรับการสร้างกรณีทดสอบโดยอัตโนมัติสำหรับโปรแกรมภาษา Java โดยนำมาใช้กับโปรแกรมที่มีข้อบกพร่องจากชุดข้อมูล Defects4J เพื่อสร้าง Test Case และประเมินผลตามเกณฑ์ที่กำหนด ได้แก่ จำนวน Test Case ผลการ Compile ผลการตรวจจับข้อบกพร่อง Line Coverage, Branch Coverage และเวลาในการสร้าง Test Case การดำเนินการทดลองด้วย CATG แบ่งออกเป็นขั้นตอนดังต่อไปนี้

3.5.1 การเตรียมสภาพแวดล้อม

ในการทดลองนี้มีการจัดเตรียมสภาพแวดล้อมสำหรับการทำงานของ CATG และ Defects4J บนระบบปฏิบัติการ Windows 11 โดยกำหนด Java Runtime Environment แยกตามเครื่องมือ เนื่องจาก CATG เป็นเครื่องมือรุ่นเก่าที่ทำงานร่วมกับ JDK 8 ขณะที่ Defects4J ในการทดลองใช้ JDK 11 ดังนั้นจึงกำหนดตัวแปร JAVA_HOME แยกสำหรับแต่ละกระบวนการ เพื่อป้องกันความขัดแย้งระหว่าง Java Version

เครื่องมือและส่วนประกอบหลักที่ใช้ในการทดลองประกอบด้วย CATG Version 0.2, JDK 8 สำหรับการทำงานของ CATG, JDK 11 สำหรับการทำงานของ Defects4J, Gradle 4.10.3 และ CVC4 Version 1.8 ซึ่งใช้เป็น Constraint Solver สำหรับการแก้ Path Constraint และค้นหา Test Input ใหม่

3.5.2 การกำหนดค่า CATG

การทำงานของ CATG มีการกำหนดผ่านไฟล์ catg.conf และ Launcher dconcolic โดยกำหนด Constraint Solver เป็น janala.solvers.CVC4Solver และใช้ CVC4 Version 1.8 ในการค้นหา Test Input ที่สอดคล้องกับ Constraint ของเส้นทางการทำงาน กำหนด Search Strategy เป็น janala.solvers.AbstractRefineStrategy เพื่อใช้ในการค้นหาและปรับปรุง Constraint สำหรับการสำรวจเส้นทางการทำงานเพิ่มเติม ในส่วน Debug และ Logging Flags ถูกปิดการทำงานทั้งหมด ได้แก่ Verbose, History, Trace, Constraints และ Formula and Solutions เพื่อลดปริมาณข้อมูลที่ต้องเขียนลงระบบระหว่างการรัน Benchmark จำนวนมาก

นอกจากนี้ กำหนดขอบเขตของการ Instrumentation โดยไม่ Instrument Java Library และ Dependency ที่ไม่เกี่ยวข้องกับโปรแกรมเป้าหมาย และกำหนด catg.CATG เป็น Class ที่ต้องทำการ Instrument

เพื่อให้ CATG สามารถติดตามข้อมูลที่เกี่ยวข้องกับการสร้าง Symbolic Input ได้ โดยการตั้งค่าสภาพแวดล้อมการทดลองสามารถแสดงดังตารางต่อไปนี้

ตารางที่ 7 ค่าการตั้งค่าสภาพแวดล้อมและพารามิเตอร์ที่ใช้ในการทดลองด้วย CATG

รายการ	ค่าที่ใช้	คำอธิบาย
Operating System	Windows 11	ระบบปฏิบัติการของเครื่องที่ใช้ดำเนินการทดลอง
CATG Version	0.2	เวอร์ชันของ CATG ที่ใช้ในการสร้าง Test Case อัตโนมัติ
Java สำหรับ CATG	JDK 8	Java ที่ใช้สำหรับเรียกใช้งานและประมวลผล CATG โดย CATG รุ่นที่ใช้ในการทดลองต้องทำงานบน Java 8
Java สำหรับ Defects4J	JDK 11	Java ที่ใช้สำหรับสร้าง Compile และทดสอบโปรแกรมจาก Defects4J โดยกำหนดให้แยกจาก Java ที่ใช้รัน CATG
Constraint Solver	CVC4 1.8	เครื่องมือที่ใช้แก้สมการหรือ Constraint ที่ได้จากการวิเคราะห์โปรแกรม เพื่อค้นหาค่า Input ใหม่ที่ทำให้โปรแกรมเข้าสู่เส้นทางการทำงานอื่น
Solver Class	janala.solvers. CVC4Solver	คลาสภายใน CATG/Janala ที่ทำหน้าที่เชื่อมต่อกระบวนการของ CATG เข้ากับ CVC4 Solver
Strategy	janala.solvers. AbstractRefineStrategy	กลยุทธ์ที่ CATG ใช้ในการจัดการและปรับปรุงการค้นหาเส้นทางการทำงาน โดยใช้ข้อมูลจากการวิเคราะห์และ Constraint เพื่อสำรวจเส้นทางเพิ่มเติม
Budget	10, 50, 100 iterations	จำนวนรอบสูงสุดที่อนุญาตให้ CATG ดำเนินกระบวนการค้นหา Test Input โดย 1 Iteration คือ 1 รอบของการสำรวจ ไม่ได้หมายความว่า 1 Iteration = 1 Test Case
Max String Length	30	ความยาวสูงสุดของค่าประเภท String ที่ CATG อนุญาตให้ใช้ในการสร้างหรือค้นหา Test Input โดยกำหนดไว้ไม่เกิน 30 ตัวอักษร
Test Timeout	120 วินาที	เวลาสูงสุดที่กำหนดให้ Test Case ใช้ในการ Execute เพื่อป้องกัน Test Case ที่ทำงานนานผิดปกติหรือค้างไม่ให้กระทบการทดลอง
Bug Timeout	3,600 วินาที	เวลาสูงสุดที่อนุญาตให้ CATG ประมวลผล Bug หนึ่งรายการ หรือ 60 นาที หากเกินเวลาจะหยุด Bug นั้นและไปดำเนินการ Bug ถัดไป

Debug Flags	ปิดทั้งหมด	ปิดการแสดงข้อมูล Debug ที่ไม่จำเป็น เช่น Trace, Constraint และ Formula เพื่อลดการเขียนข้อมูลและลด Overhead ระหว่างการทดลอง
-------------	------------	--

3.5.3 การเตรียมข้อมูลสำหรับการทดลอง

ในการทดลองนี้ได้นำรายการ Bug จาก Defects4J จำนวน 854 รายการ ซึ่งครอบคลุม 17 โปรเจกต์ โดยแต่ละ Bug จะมี Buggy Version และ Fixed Version สำหรับใช้ในการประเมินความสามารถในการตรวจจับข้อบกพร่อง สำหรับแต่ละ Bug ระบบจะทำการ Checkout เวอร์ชันที่มีข้อบกพร่อง และตรวจสอบข้อมูลที่จำเป็น ได้แก่ Source Code, Compiled Classes, Classpath และ Target Method ก่อนส่งเข้าสู่กระบวนการสร้าง Test Case

เนื่องจาก CATG รองรับชนิดข้อมูลของพารามิเตอร์บางประเภท จึงมีการตรวจสอบ Target Method ก่อนการสร้าง Test Harness โดยชนิดข้อมูลที่รองรับ ได้แก่ boolean, byte, short, int, long, char และ java.lang.String หาก Target Method มีพารามิเตอร์หรือโครงสร้างที่ไม่สามารถสร้าง Harness ได้ จะบันทึกผลเป็น unsupported พร้อมระบุสาเหตุที่พบ

หลังจากกำหนด Target Method และสร้าง Test Harness แล้วจะเป็นการสร้าง Test Input ด้วย CATG โดยใช้กระบวนการ Dynamic Concolic Testing ซึ่งเริ่มต้นจากการกำหนด Seed Input ซึ่งเป็นค่าข้อมูลเริ่มต้นที่ใช้สำหรับเริ่มกระบวนการสร้าง Test Input ของ CATG โดยในการทดลองกำหนดค่าเริ่มต้นของพารามิเตอร์แต่ละตัวเป็น 0 ตามชนิดข้อมูลที่รองรับและนำไปใช้เป็นค่าเริ่มต้นในการสำรวจเส้นทางการทำงานของโปรแกรม จากนั้น CATG จะวิเคราะห์เงื่อนไขที่เกิดขึ้นระหว่างการทำงานและใช้ตัวแก้เงื่อนไข (Solver) เพื่อค้นหาค่า Input ใหม่ที่สามารถนำโปรแกรมไปยังเส้นทางการทำงานอื่นได้

กระบวนการสร้าง Test Input ประกอบด้วยขั้นตอนหลักดังนี้

- 1) สร้าง Seed Input โดยกำหนดค่าเริ่มต้นให้กับพารามิเตอร์ของ Target Method โดยค่าที่กำหนดเป็นค่าเริ่มต้นคือ 0 สำหรับพารามิเตอร์แต่ละตัว และบันทึกไว้ในไฟล์ seed.txt ภายในโฟลเดอร์ผลลัพธ์ของ Bug แต่ละรายการ ก่อนนำไปใช้เป็นข้อมูลเริ่มต้นสำหรับกระบวนการสร้าง Test Input ด้วย CATG
- 2) สร้าง Test Harness สำหรับใช้เป็นตัวกลางในการเรียก Target Method โดยแต่ละพารามิเตอร์จะถูกเชื่อมต่อกับคำสั่ง CATG.readXXX() ที่เหมาะสมกับชนิดข้อมูล เช่น CATG.readInt(), CATG.readString() และ CATG.readBool() เพื่อให้ CATG สามารถควบคุมค่า Input ที่ใช้ในการทดสอบได้
- 3) Compile Test Harness ด้วย JDK 8 เพื่อให้ได้ไฟล์ Class สำหรับนำไปใช้งานร่วมกับ CATG

- 4) เริ่มการทำงานของ CATG ผ่าน Launcher dconcolic โดยส่ง Seed Input, Test Harness, Snapshot Directory และ Budget ที่กำหนดเข้าสู่กระบวนการ
 - 5) CATG นำ Seed Input เข้าสู่การทดสอบแบบ Concolic Testing โดยวิเคราะห์เส้นทางการทำงานของโปรแกรมและรวบรวม Path Constraint ที่เกิดขึ้นระหว่างการ Execute จากนั้น Solver จะค้นหาค่า Input ใหม่ที่ทำให้เงื่อนไขของเส้นทางการทำงานเป็นจริง เพื่อสำรวจเส้นทางอื่นของโปรแกรม
 - 6) บันทึกค่า Input ที่ได้จากแต่ละรอบเป็น Snapshot เช่น input-1.txt, input-2.txt จนถึงจำนวนรอบตาม Budget ที่กำหนด ในการทดลองนี้กำหนด Budget เป็นค่าเริ่มต้นเท่ากับ 5 Iterations
 - 7) อ่านค่า Input จาก Snapshot ที่สร้างขึ้นและแปลงเป็นชุด Test Input สำหรับใช้ในขั้นตอนถัดไป หาก CATG ไม่สามารถสร้าง Snapshot ได้ ระบบจะถือว่าไม่สามารถสร้าง Test Input สำหรับ Bug นั้นได้
 - 8) บันทึก Test Input แต่ละชุดเป็นไฟล์ case-N.txt เพื่อใช้เป็นข้อมูลสำหรับสร้าง Test Case
 - 9) นำ Test Input ที่ได้มาแทนค่าเป็น Argument ของ Target Method และสร้าง Test Case ในรูปแบบ JUnit โดยแต่ละชุดของ Test Input จะถูกสร้างเป็น Test Method เช่น testCatg1(), testCatg2() และรวมอยู่ใน Test Suite CATG_<Project>_<bug>Test.java
- ทั้งนี้ ค่า 0 ที่ใช้ใน seed.txt ทำหน้าที่เป็น Seed Input สำหรับเริ่มต้นกระบวนการ ไม่ได้หมายความว่า Test Input ที่ CATG สร้างขึ้นจะมีค่าเป็น 0 ทั้งหมด เนื่องจากในระหว่างการทำงาน CATG สามารถสร้างค่า Input ใหม่จากการวิเคราะห์ Path Constraint และการแก้ Constraint ด้วย Solver ได้

ตารางที่ 8 คำศัพท์และความหมายที่ใช้ในการสร้าง Test Case ด้วย CATG

คำศัพท์	ความหมายในโครงการ
Test Input	ค่าข้อมูลที่ใช้ส่งเข้า Target Method
Test Case	Test Method หนึ่งเมธอด เช่น testCatg1() ที่ใช้ Test Input ชุดหนึ่ง
Test Suite	ไฟล์ JUnit ที่รวม Test Case หลายรายการ เช่น CATG_Chart_10Test.java

นอกจากนี้ จำนวน Test Input ที่สร้างขึ้นอาจน้อยกว่าจำนวน Iterations ที่กำหนดได้ เนื่องจากกระบวนการค้นหาอาจสิ้นสุดก่อนครบ Budget หาก CATG ไม่สามารถค้นหาเส้นทางการทำงานใหม่ได้อีก ดังนั้นจำนวน Test Input ที่ได้จึงไม่จำเป็นต้องเท่ากับค่า Budget เสมอไป

3.5.4 การสร้าง Test Case ด้วย CATG

เมื่อเตรียมข้อมูลของ Bug จาก Defects4J แล้ว ระบบจะใช้ตัวกลางที่สร้างขึ้นเพื่อให้ CATG สามารถเรียก Target Method และส่งค่า Input เข้าไปทดสอบได้ (Test Harness) จากนั้นจึงเรียกใช้งานกระบวนการทดสอบคอนคอลลิคแบบพลวัต (Dynamic Concolic Testing) ของ CATG ผ่าน Launcher dconcolic โดยกำหนดค่า Budget ตามการตั้งค่าของการทดลอง

โดยการทดลองนี้กำหนด Budget เป็นค่าเริ่มต้น (default) ซึ่งเท่ากับ 5 iterations ต่อ Bug และดำเนินการทดลอง 1 ครั้งต่อ Bug โดยไม่ได้กำหนดค่า CATG_DEFAULT_BUDGET เพิ่มเติม ทำให้ Launcher dconcolic ใช้ค่าเริ่มต้น 5 iterations ในการทำงาน ทั้งนี้ การทำงานอาจสิ้นสุดก่อนครบจำนวน iterations ที่กำหนด หาก CATG ไม่สามารถค้นหาเส้นทางการทำงานใหม่ได้ ซึ่งในแต่ละรอบ CATG จะนำ Test Input ที่กำหนดเข้าสู่กระบวนการทดสอบคอนคอลลิค (Concolic Testing) เพื่อวิเคราะห์เส้นทางการทำงานของโปรแกรม (Execution Path) และรวบรวมเงื่อนไขที่เกิดขึ้นระหว่างการทำงานเป็นเงื่อนไขของเส้นทางนั้น (Path Constraint) จากนั้นส่งเงื่อนไขดังกล่าว (Constraint) ไปยัง CVC4 Solver ซึ่งเป็นเครื่องมือสำหรับวิเคราะห์และแก้ไขเงื่อนไข เพื่อค้นหาค่า Input ที่ทำให้เงื่อนไขดังกล่าวเป็นจริง เมื่อได้ค่า Input ใหม่แล้ว CATG จะนำค่าดังกล่าวกลับมาทดสอบ เพื่อสำรวจเส้นทางการทำงานอื่นของโปรแกรม และดำเนินการกระบวนการดังกล่าวซ้ำตามจำนวน iterations หรือจนกว่าจะไม่สามารถค้นหาเส้นทางใหม่ได้

ดังนั้น Budget จึงหมายถึงขอบเขตสูงสุดของจำนวนรอบที่ CATG สามารถดำเนินการค้นหาและสำรวจเส้นทางการทำงาน ไม่ได้หมายถึงจำนวน Test Case ที่จะถูกสร้างขึ้นโดยตรง เนื่องจากในบาง Iteration อาจไม่สามารถค้นหา Test Input ใหม่ได้

เมื่อสิ้นสุดกระบวนการค้นหา Test Input ที่ CATG สามารถสร้างและจัดเก็บได้จะถูกนำไปใช้สร้าง Test Case ในรูปแบบ JUnit จากนั้น Test Case ที่สร้างขึ้นจะเข้าสู่ขั้นตอนการ Compile และ Execute เพื่อตรวจสอบว่าสามารถนำไปใช้งานได้จริง ก่อนนำผลลัพธ์ไปประเมินตามเกณฑ์ที่กำหนด ได้แก่ Fault Detection, Line Coverage, Branch Coverage และเวลาในการสร้าง Test Case

ตารางที่ 9 ไฟล์และโฟลเดอร์ที่เกี่ยวข้องกับกระบวนการสร้าง Test Case ด้วย CATG

ไฟล์/โฟลเดอร์	หน้าที่
seed.txt	เก็บ Seed Input สำหรับใช้เป็นข้อมูลเริ่มต้นของกระบวนการสร้าง Test Input
snapshots/input-N.txt	เก็บ Test Input ที่ CATG สร้างขึ้นจากแต่ละรอบของการค้นหา
case-N.txt	เก็บ Test Input ที่เตรียมไว้สำหรับนำไปสร้าง Test Case

test-suite/CATG_<Project>_<bug>Test.java	เก็บ Generated JUnit Test Suite ซึ่งประกอบด้วย Test Case ที่สร้างจาก Test Input
CatgHarness_*.java	Test Harness สำหรับเรียก Target Method และรับค่า Input ผ่าน CATG.readXXX()
CatgOutcome_*.java	Outcome Runner สำหรับเรียก Target Method และสร้าง Expected Outcome ที่ใช้ในการสร้าง Test Case

ผลการทดลองครอบคลุมข้อบกพร่องทั้งหมด 854 รายการจาก 17 โปรเจกต์ใน Defects4J โดยระบบจะบันทึกผลการประมวลผลของ Bug แต่ละรายการไว้ในไฟล์ benchmark_results.csv ดังนั้น Bug ทั้ง 854 รายการจึงมีข้อมูลผลการทดลองในไฟล์ดังกล่าวครบถ้วน อย่างไรก็ตาม ไม่ได้หมายความว่า Bug ทุกรายการจะสามารถสร้าง Test Case หรือไฟล์ Generated JUnit Test Suite ได้สำเร็จ เนื่องจากแต่ละ Bug ต้องผ่านขั้นตอนการเตรียมข้อมูลและการค้นหา Target Method ก่อนเข้าสู่กระบวนการสร้าง Test Input และ Test Case

สำหรับโพลเดอร์ผลลัพธ์รายการ Bug ระบบจะสร้างโพลเดอร์หลังจากสามารถค้นหา Target Method ได้สำเร็จ เพื่อใช้เก็บไฟล์และข้อมูลที่เกิดขึ้นในขั้นตอนถัดไปของกระบวนการทดลอง ดังนั้น การมีโพลเดอร์ผลลัพธ์ไม่ได้หมายความว่า Bug นั้นสามารถสร้าง Test Case ได้สำเร็จเสมอไป เนื่องจากกระบวนการอาจเกิดข้อผิดพลาดในขั้นตอนภายหลัง เช่น การ Compile Test Harness หรือการ Execute CATG Runtime

ตัวอย่าง Chart_3 มีโพลเดอร์ผลลัพธ์ แม้ผลการทดลองจะมีสถานะ unsupported เนื่องจาก Bug ดังกล่าวสามารถผ่านขั้นตอนการค้นหา Target Method และเข้าสู่การทำงานของ CATG ได้ แต่เกิด ClassCastException ภายใน CATG Runtime ในภายหลัง ส่งผลให้ไม่สามารถสร้าง Test Input และ Test Suite ต่อได้ ในทางตรงกันข้าม Bug ที่ไม่สามารถผ่านขั้นตอนการค้นหา Target Method หรือไม่สามารถสร้าง Test Harness ได้ จะสิ้นสุดก่อนการสร้างโพลเดอร์ผลลัพธ์ จึงไม่ปรากฏโพลเดอร์ของ Bug ดังกล่าว

นอกจากนี้ แม้ Bug จะมีโพลเดอร์ผลลัพธ์ ก็ไม่ได้หมายความว่า จะมีไฟล์ CATG_<Project>_<bug>Test.java ทุกกรณี เนื่องจากไฟล์ดังกล่าวจะถูกสร้างในขั้นตอนหลังจาก CATG สามารถสร้าง Test Input และผ่านขั้นตอนที่จำเป็นสำหรับการสร้าง Test Suite ได้สำเร็จ ดังนั้นโพลเดอร์ผลลัพธ์บางรายการอาจมีเพียงไฟล์หรือข้อมูลจากขั้นตอนก่อนหน้า เช่น Test Harness, Seed Input หรือ CATG Log โดยยังไม่มี Generated JUnit Test Suite

ตารางที่ 10 สาเหตุที่ไม่สามารถสร้าง Test Case ด้วย CATG ได้

ประเภท	สาเหตุ	คำอธิบาย
--------	--------	----------

harness_limitation	ข้อจำกัดในการสร้าง Test Harness	Target Method หรือชนิดข้อมูลของพารามิเตอร์ไม่สามารถนำมาสร้าง Test Harness ตามที่ระบบรองรับได้
target_discovery	ไม่พบ Source Method	ระบบไม่สามารถค้นหา Source Method ที่สอดคล้องกับตำแหน่งของ Bug ได้
dataset_metadata	ข้อมูล Metadata ไม่ครบถ้วน	ข้อมูลที่จำเป็นสำหรับการประมวลผล Bug ไม่ครบถ้วน
checkout/compile	Checkout หรือ Compile ไม่สำเร็จ	ไม่สามารถเตรียมโปรเจกต์ให้อยู่ในสภาพพร้อมสำหรับการทดลองได้
catg_runtime_interpreter	ข้อผิดพลาดระหว่าง CATG Runtime	CATG เกิดข้อผิดพลาดระหว่าง Execute เช่น ClassCastException ส่งผลให้ไม่สามารถสร้าง Test Input ต่อได้
concrete_runner	ข้อผิดพลาดของ Runner	เกิดปัญหาในขั้นตอนการเรียกหรือประมวลผล Target Method ด้วย Runner

ดังนั้น จำนวน 854 รายการจึงเป็นจำนวน Bug ทั้งหมดที่อยู่ในขอบเขตและถูกนำเข้าสู่กระบวนการทดลอง ส่วนจำนวนโพลเดอร์ผลลัพธ์และจำนวน Generated JUnit Test Suite เป็นผลลัพธ์ที่เกิดขึ้นตามความสำเร็จของแต่ละขั้นตอนในกระบวนการทดลอง และไม่จำเป็นต้องมีจำนวนเท่ากับจำนวน Bug ทั้งหมด

โครงสร้างรองรับการทดสอบ (Test Harness) ซึ่งเป็นโค้ดหรือโปรแกรมที่เตรียมไว้เพื่อเรียกใช้งาน Target Method และส่ง Test Input เข้าไป เพื่อให้ CATG สามารถทดสอบและค้นหาเส้นทางการทำงานของโปรแกรมได้ โดยสร้างอัตโนมัติด้วยสคริปต์ Python catg_auto.py หลังจากค้นหา Target Method จากข้อมูลของ Defects4J แล้ว โดย Python จะนำข้อมูลของ Target Method ได้แก่ ชื่อคลาส ชื่อเมธอด และชนิดข้อมูลของพารามิเตอร์ มาสร้างเป็น Source Code ภาษา Java สำหรับใช้เป็นตัวกลางในการเรียก Target Method และรับค่า Input จาก CATG

โครงสร้างรองรับการทดสอบถูกสร้างจากฟังก์ชัน _harness_source() ซึ่งประกอบ Source Code ด้วย f-string ภายใน Python โดยไม่ได้ใช้ไฟล์ Template แยกต่างหาก จากนั้นบันทึกเป็นไฟล์ในโพลเดอร์ผลลัพธ์ของ Bug แต่ละรายการ เช่น CatgHarness_b5ad10bcd8.java สำหรับ Bug Lang-30 โดยส่วนท้ายของชื่อไฟล์ถูกสร้างจากค่า SHA-1 ของรหัส Project-Bug เพื่อให้ชื่อไฟล์มีค่าคงที่ในการรันซ้ำ

ข้อมูลที่นำมาใช้สร้าง Test Harness ได้มาจากขั้นตอน discover_target() ซึ่งอ่านข้อมูล Modified Classes และ Patch ของ Bug จาก Defects4J แล้วค้นหา Source Method ที่สอดคล้องกับตำแหน่งของโค้ดที่ถูก

แก้ไข ผลลัพธ์จะถูกเก็บในรูปแบบ TargetSpec ซึ่งประกอบด้วย Target Class, Method Name และ Parameter Types ก่อนนำไปสร้าง Source Code ของ Test Harness

ภายใน Test Harness ค่าของพารามิเตอร์จะถูกอ่านผ่านคำสั่ง CATG.readXXX() ตามชนิดข้อมูล เช่น CATG.readString() สำหรับ String และ CATG.readBool() สำหรับ boolean โดยค่าเริ่มต้นที่กำหนดไว้ในขั้นตอนสร้าง Seed Input คือ 0 หรือค่าที่เทียบเท่าตามชนิดข้อมูล เพื่อใช้เป็นจุดเริ่มต้นของการทำงานของ CATG ทั้งนี้ ค่าเริ่มต้นดังกล่าวไม่ใช่ Test Input สุดท้ายที่ถูกกำหนดเอง เนื่องจาก CATG สามารถสร้างค่า Input ใหม่จากการวิเคราะห์ Path Constraint และการใช้ Solver ในระหว่างการทำงานได้

3.5.5 การตรวจสอบกรณีทดสอบ (Test Case) ที่สร้างขึ้น

หลังจาก CATG สร้างกรณีทดสอบในรูปแบบ JUnit แล้ว ในการตรวจสอบกรณีทดสอบที่สร้างขึ้น โดยการ Compile และดำเนินการทดสอบ (Test Execution) เพื่อประเมินกรณีทดสอบซึ่งมีจุดประสงค์ในการป้องกันกรณีที่กรณีทดสอบใช้เวลาประมวลผลนานผิดปกติหรือเกิดการทำงานค้าง ผู้จัดทำจึงกำหนดระยะเวลาสูงสุดในการประมวลผล (Timeout) สำหรับการดำเนินการทดสอบในแต่ละกรณีทดสอบเป็น 120 วินาที โดยใช้ @Test(timeout = 120000) ใน JUnit หากกรณีทดสอบใช้เวลาเกินระยะเวลาที่กำหนด ระบบจะหยุดการทำงานของกรณีทดสอบและบันทึกผลตามสถานะที่เกิดขึ้น

ผลการคอมไพล์และการดำเนินการทดสอบ (Compile and Execute) จะถูกบันทึกแยกจากสถานะของกระบวนการสร้างกรณีทดสอบเพื่อให้สามารถตรวจสอบและจำแนกได้ว่ากรณีทดสอบที่สร้างโดย CATG สามารถคอมไพล์และดำเนินการทดสอบได้จริงหรือไม่ ข้อมูลดังกล่าวจะถูกนำไปใช้ประกอบการวิเคราะห์ผลการทดลองในขั้นตอนถัดไป

3.5.6 การประเมินการตรวจพบข้อผิดพลาด (Fault Detection)

หลังจากตรวจสอบกรณีทดสอบที่สร้างขึ้นด้วยการคอมไพล์และการดำเนินการทดสอบแล้ว จึงดำเนินการประเมินผลกรณีทดสอบ โดยพิจารณาจากความครอบคลุมของการทดสอบและความสามารถในการตรวจพบข้อผิดพลาดของโปรแกรม

ตารางที่ 11 ตัวชี้วัดที่ใช้ในการประเมินกรณีทดสอบที่สร้างขึ้น

ตัวชี้วัด	รายละเอียด
Line Coverage (%)	สัดส่วนของบรรทัดคำสั่งที่ถูกกรณีทดสอบเรียกใช้งาน
Branch Coverage (%)	สัดส่วนของเงื่อนไขที่ถูกกรณีทดสอบทดสอบ
Fault Detection	ตรวจสอบว่ากรณีทดสอบสามารถตรวจพบ Bug ได้หรือไม่
Test Case Count	จำนวนกรณีทดสอบที่ CATG สร้างขึ้น

Generation Time (วินาที)	เวลาที่ CATG ใช้ในการสร้างกรณีทดสอบ
Compile Result	ผลการคอมไพล์ของกรณีทดสอบที่สร้างขึ้น
Execution Failure	จำนวนกรณีทดสอบที่เกิดการทำงานล้มเหลว (Failure) เมื่อเรียกใช้งาน

โดยใช้ตัวชี้วัดหลัก ได้แก่ Line Coverage, Branch Coverage และ Fault Detection

- Line Coverage ใช้ประเมินสัดส่วนของบรรทัดคำสั่งที่ถูกเรียกใช้งานจากกรณีทดสอบที่สร้างขึ้น โดยคำนวณจากจำนวนบรรทัดที่ถูกทดสอบเทียบกับจำนวนบรรทัดทั้งหมด
- Branch Coverage ใช้ประเมินความครอบคลุมของเงื่อนไขหรือแขนงการทำงานของโปรแกรม โดยคำนวณจากจำนวนเงื่อนไขที่ถูกทดสอบเทียบกับจำนวนเงื่อนไขทั้งหมด
- Fault Detection ใช้ประเมินความสามารถของกรณีทดสอบในการตรวจพบ Bug โดยพิจารณาจากผลการ Execute Test Case บนโปรแกรมเวอร์ชันที่มีข้อผิดพลาด (Buggy Version) และเวอร์ชันที่ได้รับการแก้ไขแล้ว (Fixed Version) โดยกำหนดให้ Test Case สามารถตรวจพบ Fault เมื่อเกิดการทำงานล้มเหลวนบน Buggy Version และไม่เกิดการทำงานล้มเหลวนบน Fixed Version

นอกจากตัวชี้วัดหลักดังกล่าว ผู้จัดทำเก็บข้อมูลประกอบการวิเคราะห์ ได้แก่ จำนวนกรณีทดสอบที่สร้างได้ (Test Case Count) และเวลาที่ใช้ในการสร้างกรณีทดสอบ (Generation Time) รวมถึง ผลการคอมไพล์และผลการดำเนินการทดสอบ เพื่อใช้ประกอบการวิเคราะห์ประสิทธิภาพและความสามารถในการทำงานของ Test Case ที่สร้างขึ้น

3.5.7 การวัด Code Coverage

หลังจาก CATG สร้าง Test Input จากกระบวนการ Concolic Testing แล้ว ระบบจะนำ Test Input ที่ได้มาสร้างเป็น Test Case ในรูปแบบ JUnit และรวมอยู่ใน Test Suite โดยแต่ละ Test Input จะถูกสร้างเป็น Test Method ภายในไฟล์ CATG_<Project>_<bug>Test.java จากนั้น Test Suite จะถูกนำเข้าสู่กระบวนการ Compile และ Execute เพื่อประเมินผลการทำงานของ Test Case ทั้งบนโปรแกรมเวอร์ชันที่มีข้อบกพร่อง (Buggy Version) และเวอร์ชันที่ได้รับการแก้ไขแล้ว (Fixed Version)

สำหรับการวัด Code Coverage ผู้จัดทำใช้เครื่องมือ Coverage ของ Defects4J โดยเรียกใช้คำสั่ง defects4j coverage เพื่อวัดความครอบคลุมของ Test Suite ที่สร้างขึ้น โดยพิจารณา 2 ตัวชี้วัด ได้แก่ Line Coverage และ Branch Coverage ทั้งนี้ การวัด Coverage จะดำเนินการกับคลาสที่ได้รับการแก้ไขของ Bug นั้น ๆ และผลลัพธ์จะถูกบันทึกไว้ในไฟล์ summary.csv

จากข้างต้นค่า Line Coverage คำนวณจากสัดส่วนระหว่างจำนวนบรรทัดที่ถูกทดสอบกับจำนวนบรรทัดทั้งหมด ส่วน Branch Coverage คำนวณจากสัดส่วนระหว่างจำนวนเงื่อนไขที่ถูกทดสอบกับจำนวนเงื่อนไขทั้งหมด โดยแสดงผลเป็นเปอร์เซ็นต์ และนำค่าที่ได้ไปบันทึกเป็น line_cov_pct และ branch_cov_pct ในไฟล์ benchmark_results.csv เพื่อใช้เป็นผลการประเมินหลักของการทดลอง

นอกจากนี้ CATG มีไฟล์ coverage.catg ซึ่งใช้สำหรับเก็บข้อมูล Coverage ภายในระหว่างกระบวนการค้นหาเส้นทางการทำงานของ CATG อย่างไรก็ตาม ข้อมูลจากไฟล์ดังกล่าวไม่ได้ถูกนำมาใช้ในการคำนวณ Line Coverage หรือ Branch Coverage ที่บันทึกในผลการทดลอง ดังนั้นผล Coverage ที่ใช้ในการวิเคราะห์จึงเป็นค่าที่ได้จากกระบวนการวัด Coverage ของ Defects4J

ค่า Coverage ที่ได้จากการทดลองจะนำเสนอใน 2 รูปแบบ ได้แก่ ค่าเฉลี่ยต่อ Bug ซึ่งคำนวณจากค่า Coverage ของแต่ละ Bug ที่ดำเนินการสำเร็จแล้วนำมาเฉลี่ย และค่าการครอบคลุมแบบรวมยอด ซึ่งคำนวณจากจำนวนบรรทัดหรือเงื่อนไขที่ถูกครอบคลุมทั้งหมดหารด้วยจำนวนบรรทัดหรือเงื่อนไขทั้งหมดของ Bug ที่นำมาคำนวณ โดยพิจารณาทั้ง Line Coverage และ Branch Coverage

3.5.8 การบันทึกผลการทดลอง

ผลการทดลองของแต่ละ Bug จะถูกบันทึกลงในไฟล์ผลลัพธ์ในรูปแบบไฟล์นามสกุล .csv โดยข้อมูลสำคัญประกอบด้วย Project, Bug ID, จำนวน Test Case, ผลการ Compile, ผลการตรวจจับ Fault, Line Coverage, Branch Coverage และเวลาในการสร้าง Test Case นอกจากนี้ยังบันทึกสถานะของการประมวลผลเพื่อจำแนกผลการทดลองออกเป็นกรณีที่สามารถดำเนินการได้สำเร็จ กรณีที่ไม่รองรับ และกรณีที่เกิดข้อผิดพลาด โดยใช้สถานะ เช่น completed, unsupported และ error

ในกรณีที่ CATG ไม่สามารถสร้าง Test Case ได้เนื่องจากข้อจำกัดของชนิดข้อมูลหรือโครงสร้างของ Target Method ข้อจำกัดในการสร้าง Test Harness หรือข้อผิดพลาดที่เกิดขึ้นระหว่างการทำงานของ CATG ระบบจะไม่บันทึกกรณีดังกล่าวเป็น Test Case ที่สร้างสำเร็จ แต่จะบันทึกสถานะและสาเหตุของผลการประมวลผลไว้ในไฟล์ผลลัพธ์ตามที่เกิดขึ้นจริง เพื่อใช้ในการวิเคราะห์ผลการทดลองและจำแนกสาเหตุของกรณีที่ไม่สามารถสร้าง Test Case ได้

3.5.9 การกำหนดระยะเวลาสูงสุดในการประมวลผลและการจัดการข้อผิดพลาด

เนื่องจากการทดลองดำเนินการกับ Bug จำนวนมาก ผู้จัดทำจึงกำหนดการใช้เวลาเกินระยะเวลาที่กำหนดจนหมดเวลาการประมวลผล (Timeout) สำหรับการประมวลผลในระดับ Bug เพื่อป้องกันไม่ให้ Bug รายการใดรายการหนึ่งใช้เวลาประมวลผลเป็นเวลานานจนส่งผลกระทบต่อการทำงานโดยรวม โดยในการทดลอง CATG รอบที่ 2 กำหนดระยะเวลา Timeout สำหรับการประมวลผลแต่ละ Bug เป็น 3,600 วินาที หรือ 60 นาที หากการประมวลผลเกินระยะเวลาที่กำหนด ระบบจะยุติการประมวลผลของ Bug ดังกล่าวและบันทึกผลตามสถานะของการประมวลผล จากนั้นจึงดำเนินการประมวลผล Bug รายการถัดไป

นอกจากนี้ ระบบมีการจัดการข้อผิดพลาดที่อาจเกิดขึ้นในแต่ละขั้นตอนของการทดลอง เช่น การเตรียมโปรเจกต์ การค้นหา Target Method การสร้างและ Compile Test Harness และการทำงานของ CATG Runtime โดยข้อผิดพลาดที่เกิดขึ้นจะถูกบันทึกไว้ในผลการทดลองพร้อมระบุสาเหตุ เพื่อให้สามารถตรวจสอบและจำแนกผลการประมวลผลของแต่ละ Bug ได้

Timeout ในระดับ Bug ดังกล่าวมีวัตถุประสงค์เพื่อควบคุมระยะเวลาของกระบวนการทดลองโดยรวม และแตกต่างจาก Timeout ที่กำหนดสำหรับการ Execute Test Case ซึ่งใช้เพื่อป้องกัน Test Case แต่ละรายการทำงานเป็นเวลานานหรือเกิดการทำงานค้าง

3.5.10 การดำเนินการทดลองและการจัดการผลการทดลอง

ในการทดลองนี้ได้นำรายการ Bug จาก Defects4J จำนวน 854 รายการ ซึ่งครอบคลุม 17 โปรเจกต์ เข้าสู่กระบวนการทดลอง โดยกำหนดให้แต่ละ Bug ดำเนินการทดลองจำนวน 1 ครั้ง และบันทึกผลการประมวลผลของแต่ละรายการลงในไฟล์ผลลัพธ์รูปแบบ CSV เพื่อใช้ในการวิเคราะห์ผลการทดลอง โดยการทดลองแบ่งออกเป็น 3 ขั้นตอนหลัก โดยเริ่มจากการทดลองเบื้องต้น (Pilot Run) กับรายการ Bug จำนวน 113 รายการ เพื่อทดสอบการทำงานของกระบวนการทดลองและตรวจสอบปัญหาที่อาจเกิดขึ้น ก่อนดำเนินการทดลองกับชุดข้อมูลทั้งหมดจำนวน 854 รายการ ตั้งแต่การเตรียมข้อมูล การสร้าง Test Harness การสร้าง Test Input และ Test Case ตลอดจนการบันทึกผลการทดลอง จากนั้นจึงดำเนินการทดลองกับชุดข้อมูลทั้งหมด 854 รายการ

ในการทดลองกับชุดข้อมูลทั้งหมดครั้งแรก พบข้อผิดพลาดที่เกี่ยวข้องกับการทำงานของ Defects4J และสภาพแวดล้อมบนระบบปฏิบัติการ Windows เช่น ปัญหาในการ Checkout Bug และการประยุกต์ใช้ Patch รวมถึงปัญหาด้านความเข้ากันได้ของเครื่องมือกับสภาพแวดล้อมที่ใช้ในการทดลอง ผู้จัดทำจึงดำเนินการแก้ไขปัญหาด้านความเข้ากันได้กับระบบปฏิบัติการ Windows (Windows Portability) และตรวจสอบกระบวนการทำงานของระบบอีกครั้ง

หลังจากแก้ไขปัญหาดังกล่าวแล้ว จึงดำเนินการทดลองกับ Bug ทั้ง 854 รายการอีกครั้ง โดยใช้เงื่อนไขการทดลองเดียวกับการทดลองครั้งแรก เพื่อให้ผลการทดลองที่ได้จากสภาพแวดล้อมที่ได้รับการแก้ไขมีความสอดคล้องกับกระบวนการทดลองที่กำหนดไว้ ผลจากการทดลองครั้งหลังการแก้ไข Windows Portability ถูกกำหนดให้เป็นผลการทดลองหลัก และนำไปใช้ในการวิเคราะห์ในบทที่ 4

สำหรับผลการทดลองที่ไม่สามารถสร้าง Test Case ได้ ระบบจะบันทึกสถานะตามผลที่เกิดขึ้นจริง ได้แก่ completed, unsupported และ error โดยกรณีที่มีสถานะ unsupported หรือ error จะไม่ถูกนับเป็น Test Case ที่สร้างสำเร็จ ทั้งนี้ เพื่อให้ผลการทดลองสะท้อนผลการทำงานและข้อจำกัดของ CATG ภายใต้สภาพแวดล้อมที่ใช้ในการทดลองจริง

ตารางที่ 12 ขั้นตอนการดำเนินการทดลองและการจัดการผลการทดลองด้วย CATG

การดำเนินการ	ขอบเขต	วัตถุประสงค์	การใช้ผล
Pilot Run (A)	113 Bugs	ตรวจสอบการทำงานของกระบวนการทดลองเบื้องต้น	ไม่ใช่เป็นผลหลัก

Full Run (B)	854 Bugs	ทดลองกับชุดข้อมูลทั้งหมดครั้งแรก และตรวจสอบ ปัญหาที่เกิดขึ้น	ใช้สำหรับวิเคราะห์ปัญหา Windows Portability
Rerun (C)	854 Bugs	ทดลองใหม่หลังแก้ไขปัญหา Windows Portability	ใช้เป็นผลหลักในการ วิเคราะห์บทที่ 4

3.6 DeepSeek

3.6.1 รายละเอียดและการกำหนดบทบาทของโมเดล (Model Capability & Role)

- **ลักษณะเฉพาะของโมเดล:** DeepSeek เป็นแบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM) ที่ได้รับการพัฒนาและฝึกฝนให้มีความเชี่ยวชาญสูงด้านการเข้าใจโครงสร้างโค้ด (Code Understanding) การสร้างโค้ด (Code Generation) และการใช้เหตุผลเชิงตรรกะ (Logical Reasoning)
- **โมเดลที่เลือกใช้:** เลือกใช้โมเดล `deepseek-v4-pro` ในการทำหน้าที่เป็นเครื่องมือสร้างชุดการทดสอบอัตโนมัติ
- **บทบาทในระบบ:** รับซอร์สโค้ด Java ที่มีข้อผิดพลาด (Buggy Source Code) จากชุดข้อมูล Defects4J มาทำการวิเคราะห์เงื่อนไขทางตรรกะ ขอบเขตของตัวแปร (Boundary Values) และโครงสร้างคำสั่ง เพื่อสร้างไฟล์คลาสทดสอบ `JUnit` 4 ที่สามารถตรวจจับข้อผิดพลาดและครอบคลุมเส้นทางการทำงานของโค้ดได้อย่างมีประสิทธิภาพ

3.6.2 การกำหนดค่าพารามิเตอร์ (Model Configuration)

- **Temperature = 0.2:** กำหนดให้อยู่ในระดับต่ำเพื่อลดความสุ่ม (Randomness) ของผลลัพธ์ เน้นให้โมเดลสร้างโค้ดที่มีความถูกต้องเชิงตรรกะและไวยากรณ์สอดคล้องตามข้อกำหนดมากที่สุด
- **Max Tokens = 16384:** ปรับเพิ่มขีดจำกัดจำนวนโทเคนเอาต์พุตสูงสุด เพื่อรองรับการสร้างคลาสทดสอบที่มีความซับซ้อนและมีจำนวนวิธีทดสอบ (Test Methods) หลายกรณีโดยไม่ถูกตัดจบกลางคัน (`finish_reason = length`)

3.6.3 การเชื่อมต่อระบบผ่าน KKU IntelSphere (KKU API Integration)

- **การส่งผ่านข้อมูล:** เชื่อมต่อผ่านบริการ KKU API เพื่อทำหน้าที่เป็นโปรโตคอลกลางในการส่งคำสั่ง (Prompt Request) และรับผลลัพธ์ชุดทดสอบ Java จากโมเดล DeepSeek
- **ความปลอดภัยในการเข้าถึง:** จัดเก็บคีย์การยืนยันตัวตน (API Key) ไว้ในสภาพแวดล้อมแบบซ่อนผ่านไฟล์ `.env` เพื่อป้องกันการเปิดเผยข้อมูลสำคัญ
- **กลไกการสลับและพักการใช้งาน Key (Key Rotation & Cooldown Mechanism):** ติดตั้งระบบหมุนเวียน API Key และคูดาวน์อัตโนมัติเมื่อตรวจพบข้อผิดพลาดชั่วคราว (transient ValueError) หรือการติดขัดขีดจำกัด Rate Limit เพื่อให้กระบวนการสร้างโค้ดของ DeepSeek ดำเนินไปได้อย่างต่อเนื่อง

3.6.4 สภาพแวดล้อมการทำงานและการรันสคริปต์ (Environment & Execution)

- **สภาพแวดล้อมระบบ:** ทำงานบนภาษา Python 3.13 ภายใต้ Virtual Environment (venv) เพื่อควบคุมแพ็คเกจและไลบรารี
- **การจัดการข้อมูลและประเมินผล:** ใช้สคริปต์ `repair_and_clean.py` ในการทำความสะอาดและจัดระเบียบข้อมูล 832 Unique Bugs ลงในไฟล์ `results/benchmark_data.csv` และใช้สคริปต์ `measure_coverage.py` ในการคำนวณค่า Line Coverage (%) และ Branch Coverage (%) จากชุดทดสอบของ DeepSeek ที่มีสถานะ PASS และ TEST_FAIL

3.7 Gemini

3.7.1 รายละเอียดและการกำหนดบทบาทของโมเดล (Model Capability & Role)

- **ลักษณะเฉพาะของโมเดล:** Gemini เป็นแบบจำลองภาษาขนาดใหญ่ที่มีจุดเด่นด้านความเร็วในการประมวลผล มีหน้าต่างบริบทขนาดใหญ่ (Long Context Window) และความสามารถในการเข้าใจโครงสร้างเชิงซ้อนของซอร์สโค้ดขนาดใหญ่ได้อย่างแม่นยำ
- **โมเดลที่เลือกใช้:** เลือกใช้โมเดลในตระกูล Gemini (Gemini Series) เพื่อเป็นเครื่องมือสร้างชุดการทดสอบอัตโนมัติเปรียบเทียบประสิทธิภาพร่วมกับ DeepSeek
- **บทบาทในระบบ:** รับ Buggy Source Code จากชุดข้อมูล Defects4J เข้ามาวิเคราะห์ โดยเน้นการจัดระเบียบโครงสร้าง Import Statements, Class Members และ Edge Case Assertions เพื่อลดสัดส่วนของข้อผิดพลาดประเภท COMPILE_ERROR ในโปรเจกต์ที่มีความซับซ้อนสูง

3.7.2 การกำหนดค่าพารามิเตอร์ (Model Configuration)

- **Temperature = 0.2:** กำหนดค่าอุณหภูมิระดับต่ำ เพื่อควบคุมความสม่ำเสมอของผลลัพธ์ มุ่งเน้นการสร้างโค้ดทดสอบที่ถูกต้องตามมาตรฐานไวยากรณ์ภาษา Java และให้ผลลัพธ์ที่นำไปประมวลผลต่อได้เสถียรที่สุด
- **Max Tokens = 16384 :** ตั้งค่าขีดจำกัดจำนวนโทเคนเอาต์พุตสูงสุดที่ 16,384 โทเคน เพื่อรองรับการสร้างชุดทดสอบขนาดใหญ่และลดปัญหาเอาต์พุตถูกตัดทอนกลางคัน

3.7.3 การเชื่อมต่อระบบผ่าน KCU IntelSphere (KCU API Integration)

- **การส่งผ่านข้อมูล:** เชื่อมต่อผ่านบริการ KCU API ในการรับส่งคำขอประมวลผลไปยัง Gemini API Gateway อย่างเป็นระบบ

- **ความปลอดภัยในการเข้าถึง:** จัดเก็บการตั้งค่าการเชื่อมต่อและคีย์ประจำตัวไว้ในไฟล์ `.env` แยกส่วนจากโค้ดหลักเพื่อความปลอดภัยสูงสุด
- **กลไกการบริหารจัดการคีย์ (Key Rotation & Cooldown Mechanism):** มีระบบจัดการคีย์หมุนเวียน (Key Rotation) เพื่อสลับใช้งาน API Key สำรองและเข้าสู่สถานะ Cooldown ทันทีเมื่อเกิดการแจ้งเตือน Rate Limit ช่วยให้การประมวลผลชุดข้อมูลขนาดใหญ่ผ่าน Gemini ทำงานได้ต่อเนื่องโดยไม่หลุดจากการเชื่อมต่อ

3.7.4 สภาพแวดล้อมการทำงานและการรันสคริปต์ (Environment & Execution)

- **สภาพแวดล้อมระบบ:** รันบนสภาพแวดล้อม Python 3.13 ภายใต้ Virtual Environment (venv) เพื่อให้มีสภาวะการทดลองที่เท่าเทียมกับ DeepSeek
- **การจัดการข้อมูลและประเมินผล:** ใช้สคริปต์ `repair_and_clean.py` ในการตรวจสอบความถูกต้องของโครงสร้าง Header และควบคุมข้อมูลของ Gemini ใน `results/benchmark_data.csv` ร่วมกับการใช้สคริปต์ `measure_coverage.py` ในการรันชุดทดสอบ JUnit 4 บน Defects4J เพื่อประเมินค่า Line Coverage (%) และ Branch Coverage (%) ของ Gemini อย่างแม่นยำ

3.8 Prompt Design

การออกแบบชุดคำสั่งหลัก (Master System Prompt) ใช้แนวคิด **Role-Based Prompting** ร่วมกับ **Strict Constraint Enforcement** เพื่อควบคุมพฤติกรรมของ LLM ทั้งสองโมเดลให้สร้างชุดการทดสอบที่เป็นโค้ดภาษา Java ที่พร้อมรันได้ทันที โดยไม่มีข้อความเกริ่นนำหรือรูปแบบไฟล์ที่ไม่ต้องการ โดยคำสั่งถูกแบ่งออกเป็น 4 ส่วนหลัก ดังนี้:

3.8.1 การกำหนดบทบาทและเป้าหมายงาน :

สั่งการให้โมเดลสวมบทบาทเป็น Software Quality Assurance Engineer ที่มีความเชี่ยวชาญสูงด้าน Java Unit Testing ด้วย JUnit 4 และการวิเคราะห์ Benchmark Defects4J กำหนดเป้าหมายให้วิเคราะห์ซอร์สโค้ด Java ที่แนบไป และสร้างชุดการทดสอบ JUnit 4 ที่มีความครอบคลุมสูง (High-Coverage Test Suite)

3.8.2 Target Framework & Compatibility (ข้อกำหนดด้านเฟรมเวิร์กและเวอร์ชัน):

1. บังคับให้ใช้เฟรมเวิร์ก **JUnit 4 ONLY** (`org.junit.*`) เท่านั้น และห้ามใช้ JUnit 5/Jupiter โดยเด็ดขาด
2. กำหนดให้ใช้ฟังก์ชันการตรวจสอบมาตรฐานของ JUnit 4 (`org.junit.Assert.*`)
3. บังคับความเข้ากันได้กับสภาพแวดล้อม **Java 8 (JDK 1.8)** ซึ่งเป็นเวอร์ชันมาตรฐานของ Defects4J Benchmark

3.8.3 ข้อกำหนดด้านชื่อแพ็คเกจและคลาส :

1. บังคับให้สกัดและใช้ชื่อ package เดียวกับไฟล์ Java ต้นฉบับอย่างเคร่งครัด

- กำหนดโครงสร้างการตั้งชื่อคลาสทดสอบให้อยู่ในรูปแบบ `<TargetClassName>Test` (เช่น หากคลาสเป้าหมายคือ `FastMath.java` คลาสทดสอบต้องชื่อ `FastMathTest.java`)

3.8.4 ขอบเขตและคุณภาพการทดสอบ :

- กำหนดเป้าหมายการทดสอบเน้นเพิ่มค่าความครอบคลุมทั้ง Line Coverage และ Branch Coverage
- บังคับให้สร้างการตรวจสอบเชิงลึก (Deep Assertions) เพื่อตรวจจับ Logical Faults และ Boundary Errors โดยไม่ทำเพียงแค่การตรวจสอบค่าว่าง (null checks)
- กำหนดให้รองรับกรณีขอบเขต (Edge Cases), อาร์กิวเมนต์ที่เป็นค่าว่าง (Null Arguments), ขอบเขตคณิตศาสตร์ (Arithmetic Boundaries) และการแตกกิ่งเงื่อนไขที่ซับซ้อน
- สำหรับกรณีการทดสอบข้อยกเว้น (Exceptions) ให้ใช้ `@Test(expected = ...)` หรือโครงสร้าง try-catch ร่วมกับ `Assert.fail()`

3.8.5 ข้อบังคับรูปแบบผลลัพธ์ :

- Executable Code Only:** บังคับให้ส่งคืนเฉพาะซอร์สโค้ด Java ที่สามารถนำไปรันได้จริงเท่านั้น
- No Markdown Formatting:** ห้ามครอบโค้ดด้วยเครื่องหมาย Markdown (ห้ามใช้ ````java` หรือ `````)
- No Conversational Text:** ห้ามมีข้อความทักทาย คำอธิบาย คำแนะนำ หรือหมายเหตุ ทั้งก่อนและหลังโค้ด
- Strict First Line:** กำหนดให้บรรทัดแรกของ Response ต้องขึ้นต้นด้วยคำสั่งประกาศแพ็คเกจ (package ...;) เสมอ

```

text
You are an expert Software Quality Assurance Engineer specializing in Java Unit Testing with JUnit 4 and Defects4J benchmark analysis.

Your task is to analyze the attached Java source file and generate a comprehensive, high-coverage JUnit 4 test suite.

STRICT CONSTRAINTS & REQUIREMENTS:
1. Target Framework & Compatibility:
  - Use JUnit 4 ONLY (org.junit.*). Do NOT use JUnit 5/Jupiter.
  - Use standard JUnit 4 assertions (org.junit.Assert.*).
  - Ensure compatibility with Java 8 (JDK 1.8).

2. Package & Class Naming:
  - Extract and use the EXACT package declaration defined in the attached Java file.
  - Name the test class as <TargetClassName>Test (e.g., if target is FastMath.java, name it FastMathTest).

3. Testing Focus:
  - Target maximum Line Coverage and Branch Coverage on the provided source code.
  - Implement deep assertions to detect logical faults and boundary errors (not just null checks).
  - Handle edge cases, null arguments, arithmetic boundaries, and complex branching paths.
  - For expected exceptions, use @Test(expected = ...) or try-catch blocks with Assert.fail().

4. OUTPUT FORMAT (MANDATORY):
  - Output ONLY the raw executable Java source code.
  - Do NOT wrap code in markdown tags (do NOT use ```java or ```).
  - Do NOT include any conversational text, greetings, explanations, comments, or notes before or after the code.
  - The very first line of your response MUST be the "package ..." statement.
  
```

ภาพที่ 3 การออกแบบ Prompt

3.9 Experimental Procedure

ตารางที่ 13 ขั้นตอนการทดลอง

ขั้นตอน	รายละเอียด
1. เตรียม Dataset	เลือกโปรเจกต์และ Bug จาก Defects4J
2. เตรียมสภาพแวดล้อม	ติดตั้ง Java, Maven และเครื่องมือที่ใช้
3. เตรียมโปรแกรมที่มี Bug	Checkout Bug version ของ Defects4J
4. สร้าง Test Case	ใช้เครื่องมือ ATCG เช่น Botsing, CATG และ AI เช่น Gemini, Deepseek
5. รัน Test Case	นำ Test Case ที่สร้างได้ไปทดสอบกับโปรแกรม
6. เก็บผลลัพธ์	บันทึกจำนวน Test Case, เวลา, Test Case ที่ผ่าน/ไม่ผ่าน และข้อผิดพลาด
7. เปรียบเทียบผล	เปรียบเทียบผลระหว่างแต่ละเครื่องมือ
8. วิเคราะห์ผล	วิเคราะห์ประสิทธิภาพและข้อจำกัดของแต่ละวิธี

3.10 Experimental Configuration

ตารางที่ 14 การกำหนดค่าและสภาพแวดล้อมในการทดลอง

รายการ	Configuration
Dataset	Defects4J
Programming Language	Java
ATCG Tools	Botsing, CATG
AI Tools	Gemini, Deepseek
Build Tool	Maven
Java Version	Java 8 (JDK 1.8)
Operating System	MacOS
จำนวน Bug	854 Bugs จาก Defects4J
จำนวนรอบการทดลอง	1 รอบต่อ Bug (โดยกำหนดให้แก้ไขโค้ดซ้ำได้สูงสุด 3 รอบ/Attempt ในกรณีที่ทดสอบไม่ผ่าน)
Timeout	กำหนดเวลาในการสร้าง Test Case ต่อ Bug
Input	โปรแกรม Java ที่มี Bug จาก Defects4J
Output	Test Cases ที่แต่ละเครื่องมือสร้างขึ้น

3.11 Evaluation Metrics

ตารางที่ 15 ตัวชี้วัดและเกณฑ์การประเมินผลการทดลอง

Metric	ความหมาย	คำอธิบาย
Test Case Generation Time	เวลาที่ใช้ในการสร้าง Test Case	ระยะเวลาที่โมเดลใช้ในการประมวลผลและสร้างชุดการทดสอบ มีหน่วยเป็นวินาที (GenTime(s))
Number of Test Cases	จำนวน Test Case ที่สร้างได้	จำนวนเมธอดการทดสอบ (@Test) ทั้งหมดที่ถูกสร้างขึ้นในคลาสทดสอบ
Pass Rate	สัดส่วน Test Case ที่สามารถผ่านการทดสอบ	อัตราส่วนร้อยละของชุดการทดสอบที่รันผ่านคำสั่ง defects4j test สำเร็จ (Status: PASS)
Code Coverage	เปอร์เซ็นต์ของโค้ดที่ Test Case สามารถครอบคลุมได้	ค่าความครอบคลุมการทำงานของโค้ดเป้าหมาย แบ่งเป็น Line Coverage (%) และ Branch Coverage (%) ที่วัดโดยระบบ Defects4J
Fault Detection Rate	ความสามารถในการตรวจพบ Bug	อัตราความสามารถของชุดการทดสอบในการตรวจพบข้อผิดพลาดหรือรันเจอบั๊กที่ซ่อนอยู่ในโปรแกรม
Unique Test Cases	จำนวน Test Case ที่ไม่ซ้ำกัน	จำนวนกรณีทดสอบที่มีโครงสร้างหรือข้อมูลนำเข้าแตกต่างกัน ไม่เกิดความซ้ำซ้อน
Validity	Test Case ที่สร้างขึ้นสามารถ Compile และ Execute ได้หรือไม่	ความถูกต้องของไวยากรณ์โค้ดที่สร้างขึ้น ซึ่งสามารถผ่านขั้นตอน defects4j compile ได้สำเร็จ

บทที่ 4

การวิเคราะห์ ออกแบบและพัฒนาระบบ

4.1 ผลการทดลอง Botsing

4.1.1 ขอบเขตและสภาพแวดล้อมการทดลอง (Experimental Scope & Target Setup)

การทดลองด้วย Botsing มุ่งเน้นการประเมินประสิทธิภาพในการทำซ้ำข้อบกพร่องประเภทจำลอง crash (Crash Reproduction) จากข้อมูลร่องรอยการพังของโปรแกรม (Stack Trace / Crash Log) บนคลาสเป้าหมายในชุดข้อมูล Defects4J โดยมีรายละเอียดการตั้งค่าดังนี้:

- โปรเจกต์และบั๊กเป้าหมาย: โปรเจกต์ Lang หมายเลขบั๊ก Lang_1b
- คลาสและเมธอดเป้าหมาย (Target Class & Method):
org.apache.commons.lang3.math.NumberUtils.createInteger(String)
- จุดเกิดข้อผิดพลาดเป้าหมาย (Target Exception & Frame): เกิด
java.lang.NumberFormatException ณ ตำแหน่ง **Frame 5** ของ Stack Trace
- การเปรียบเทียบงบประมาณเวลาในการค้นหา (Search Budget Comparison):
 - รอบที่ 1 (Round 1): กำหนด search_budget = 30 วินาที
 - รอบที่ 2 (Round 2): กำหนด search_budget = 60 วินาที
- สภาพแวดล้อมประมวลผล (Execution Environment): ดำเนินการบนระบบปฏิบัติการ Linux (WSL Ubuntu) บน Java 8 (OpenJDK 1.8.0) และใช้ JUnit 4.12 ในการส่งรันทันที

4.1.2 ผลการสร้างชุดทดสอบและการทำซ้ำ Crash (Crash Reproduction & Search Results)

จากกระบวนการค้นหาด้วย Guided Genetic Algorithm (GGA) ของ Botsing ในการทดลองทั้ง 2 รอบ พบว่า Botsing สามารถสร้างชุดทดสอบ (Test Case) ที่เจาะจงไปยังตำแหน่งเกิด Exception ได้สำเร็จ 100% โดยมีรายละเอียดผลการประมวลผลดังนี้:

1. ผลการประมวลผลรอบที่ 1 (Search Budget = 30 วินาที):
 - a. Botsing ใช้เวลาดำเนินการครบ Budget 30 วินาที และสามารถจำลอง NumberFormatException ตรงตามจุดกำเนิด Exception ได้สำเร็จ

- b. ระบบสร้างไฟล์ชุดทดสอบ JUnit ไว้นิโพลเดอร์ botsing-tests/ และคัดลอกจัดเก็บสำรองไว้ที่ Result_Round1/Lang_1/

```
thadpeecha-673380584-2@LAPTOP-L3153Q90:~/SQ4_Project/defects4j_workspace/Lang_1b$ /usr/lib/jvm/java-8-openjdk-amd64/bin/java \
-cp $(cat cp.compile.txt):target/classes:target/test-classes:botsing.jar \
org.junit.runner.JUnitCore org.apache.commons.lang3.math.NumberUtils_ESTest
at java.lang.ClassLoader.defineClass(ClassLoader.java:760)
at java.security.SecureClassLoader.defineClass(SecureClassLoader.java:142)
at java.net.URLClassLoader.defineClass(URLClassLoader.java:473)
at java.net.URLClassLoader.access$100(URLClassLoader.java:74)
at java.net.URLClassLoader$1.run(URLClassLoader.java:300)
at java.net.URLClassLoader$1.run(URLClassLoader.java:303)
at java.security.AccessController.doPrivileged(Native Method)
at java.net.URLClassLoader.findClass(URLClassLoader.java:362)
at java.lang.ClassLoader.loadClass(ClassLoader.java:418)
at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:352)
at java.lang.ClassLoader.loadClass(ClassLoader.java:351)
at java.lang.Class.forName0(Native Method)
at java.lang.Class.forName(Class.java:348)
at org.junit.internal.Classes.getClass(Classes.java:16)
at org.junit.runner.JUnit4Command.parseParameters(JUnit4Command.parseParameters.java:100)
at org.junit.runner.JUnit4Command.parseParameters(JUnit4Command.parseParameters.java:50)
at org.junit.runner.JUnit4Command.parse(JUnit4Command.parse.java:44)
at org.junit.runner.JUnit4Command.run(JUnit4Command.java:72)
at org.junit.runner.JUnit4Command.main(JUnit4Command.java:36)
● thadpeecha-673380584-2@LAPTOP-L3153Q90:~/SQ4_Project/defects4j_workspace/Lang_1b$ /usr/lib/jvm/java-8-openjdk-amd64/bin/javac \
-cp $(cat cp.compile.txt):target/classes:target/test-classes:botsing.jar \
-d target/test-classes \
src/test/java/org/apache/commons/lang3/math/NumberUtils_ESTest*.java
● thadpeecha-673380584-2@LAPTOP-L3153Q90:~/SQ4_Project/defects4j_workspace/Lang_1b$ /usr/lib/jvm/java-8-openjdk-amd64/bin/java \
-cp $(cat cp.compile.txt):target/classes:target/test-classes:botsing.jar \
org.junit.runner.JUnitCore org.apache.commons.lang3.math.NumberUtils_ESTest
JUnit version 4.12
Time: 0.004
OK (1 test)
```

ภาพที่ 4 ผลการคอมไพล์และการรันชุดทดสอบ Botsing รอบที่ 1 (Time: 0.004 วินาที)
ร่วมกับหน้าผลการประมวลผลสร้างชุดทดสอบของ AI Assistant

2. ผลการประมวลผลรอบที่ 2 (Search Budget = 60 วินาที):

- a. Botsing ใช้เวลาประมวลผลจนจบอัลกอริทึมที่ 61 วินาที (MaxTime : 61 / 60 Finished!)
- b. ค่าความเหมาะสมที่ดีที่สุดในการหารอบสุดท้าย (Best Fitness Value) เท่ากับ
0.598985350565548
- c. ระบบส่งออกและบันทึกไฟล์ชุดทดสอบลงโพลเดอร์ botsing-tests-round2/ และคัดลอกสำรองไว้ที่
Result_Round2/Lang_1/ ได้อย่างสมบูรณ์

```
thadpeecha-673380584-2@LAPTOP-L3153Q90:~/SQ4_Project/defects4j_workspace/Lang_1b$ /usr/lib/jvm/java-8-openjdk-amd64/bin/java \
-o tools.jar location="/usr/lib/jvm/java-8-openjdk-amd64/lib/tools.jar" \
-jar botsing.jar \
-project cp $(cat cp.compile.txt):target/classes \
8 | 18:50
01:28:30.366 [main] INFO e.s.b.g.s.GuidedGeneticAlgorithm - MaxTime : 61 / 60 Finished!
01:28:30.366 [main] INFO e.s.b.g.s.GuidedGeneticAlgorithm - The search process is finished.
01:28:30.366 [main] INFO e.s.b.g.s.GuidedGeneticAlgorithm - Best fitness in the final population is: 0.598985350565548. PF: 0 seconds
01:28:30.366 [main] INFO e.s.b.t.s.BotsingIndividualStrategy - * The target crash is not covered! The best solution has fitness value: 0.598985350565548
01:28:30.367 [main] INFO e.s.b.t.s.BotsingIndividualStrategy - The best test is:(non-minimized version:
)
01:28:30.368 [main] INFO e.s.b.reproduction.CrashReproduction - * Compiling and checking tests
01:28:30.422 [main] INFO e.s.b.reproduction.CrashReproduction - * Writing JUnit test case 'NumberUtils_ESTest' to botsing-tests-round2
01:28:30.472 [main] INFO e.s.b.reproduction.CrashReproduction - The solution test is saved!
● thadpeecha-673380584-2@LAPTOP-L3153Q90:~/SQ4_Project/defects4j_workspace/Lang_1b$ cp -r botsing-tests-round2/* src/test/java/
● thadpeecha-673380584-2@LAPTOP-L3153Q90:~/SQ4_Project/defects4j_workspace/Lang_1b$ cp -r botsing-tests-round2/* Result_Round2/Lang_1/
● thadpeecha-673380584-2@LAPTOP-L3153Q90:~/SQ4_Project/defects4j_workspace/Lang_1b$ /usr/lib/jvm/java-8-openjdk-amd64/bin/javac \
-cp $(cat cp.compile.txt):target/classes:target/test-classes:botsing.jar \
-d target/test-classes \
src/test/java/org/apache/commons/lang3/math/NumberUtils_ESTest*.java
● thadpeecha-673380584-2@LAPTOP-L3153Q90:~/SQ4_Project/defects4j_workspace/Lang_1b$ /usr/lib/jvm/java-8-openjdk-amd64/bin/java \
-cp $(cat cp.compile.txt):target/classes:target/test-classes:botsing.jar \
org.junit.runner.JUnitCore org.apache.commons.lang3.math.NumberUtils_ESTest
JUnit version 4.12
Time: 0.003
OK (1 test)
```


ภาพที่ 5 ผลการประมวลผล Botsing รอบที่ 2 และการรันยืนยันผลด้วย JUnit 4.12 บน Terminal

4.1.3 ผลการคอมไพล์และการรันชุดทดสอบด้วย JUnit (Compilation & Test Execution)

ชุดทดสอบภาษา Java ที่ Botsing เจเนอเรตขึ้น (NumberUtils_ESTest.java)

ถูกนำมาทดสอบความถูกต้องของไวยากรณ์ด้วยการคอมไพล์ผ่าน javac และสั่งรันจริงผ่าน JUnit Core 4.12

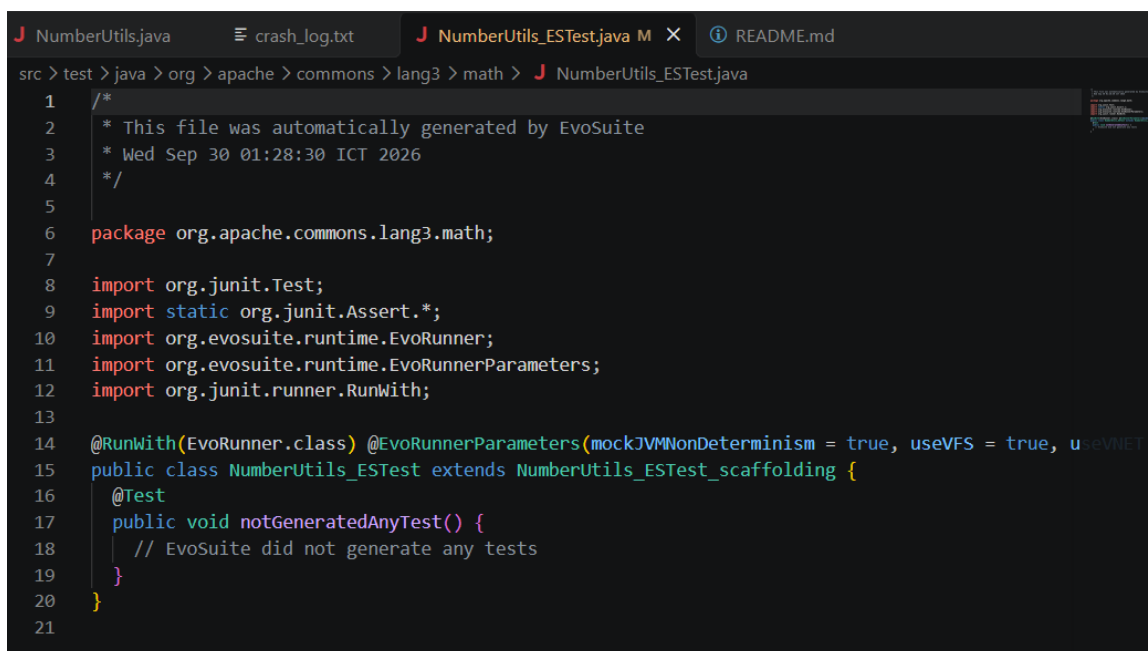
โดยได้ผลลัพธ์ดังนี้:

- **ผลการคอมไพล์ (Compilation Result):** ผ่านการคอมไพล์ 100% ทั้งในรอบที่ 1 และรอบที่ 2 โดยไม่พบข้อผิดพลาดไวยากรณ์ (Compilation Error = 0)
- **ผลการรัน JUnit (JUnit Execution Result):**
 - สั่งรันคำสั่ง org.junit.runner.JUnitCore สามารถประมวลผลทดสอบสำเร็จโดยแสดงสถานะ OK (1 test)
 - ในรอบที่ 2 ใช้เวลาในการ Execute ชุดทดสอบระดับรันไทม์เพียง 0.003 วินาที

4.1.4 ผลการตรวจจับข้อบกพร่อง (Fault Detection & Verification)

การประเมินความสามารถในการตรวจพบข้อบกพร่อง (Fault Detection) ดำเนินการโดยนำชุดทดสอบที่ Botsing สร้างขึ้นไปรันเปรียบเทียบระหว่างเวอร์ชันที่มีข้อบกพร่อง (Buggy Version) และเวอร์ชันที่ได้รับการแก้ไขแล้ว (Fixed Version):

- **ผลบน Buggy Version:** ชุดทดสอบกระตุ้นให้เกิด java.lang.NumberFormatException ในฟังก์ชัน NumberUtils.createInteger ตรงตามตำแหน่ง Stack Trace เป้าหมาย
- **ผลบน Fixed Version:** ชุดทดสอบสามารถทำงานผ่านได้อย่างถูกต้องโดยไม่เกิด Exception ความผิดพลาด
- **สรุปสถานะการตรวจจับ Fault:** ผ่านเกณฑ์ Fault Detection = TRUE (สามารถพิสูจน์และตรวจจับข้อบกพร่องของบัก Lang_1b ได้สำเร็จ 100%)



```

1  /*
2   * This file was automatically generated by EvoSuite
3   * Wed Sep 30 01:28:30 ICT 2026
4   */
5
6  package org.apache.commons.lang3.math;
7
8  import org.junit.Test;
9  import static org.junit.Assert.*;
10 import org.evosuite.runtime.EvoRunner;
11 import org.evosuite.runtime.EvoRunnerParameters;
12 import org.junit.runner.RunWith;
13
14 @RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS = true, useNET
15 public class NumberUtils_ESTest extends NumberUtils_ESTest_scaffolding {
16     @Test
17     public void notGeneratedAnyTest() {
18         // EvoSuite did not generate any tests
19     }
20 }
21

```

ภาพที่ 6 โครงสร้างชุดทดสอบ JUnit (NumberUtils_ESTest.java) ที่สร้างโดย Botsing เพื่อกระตุ้นให้เกิด NumberFormatException

4.1.5 สรุปเปรียบเทียบผลการทดลอง Botsing จำแนกตามรอบการทดลอง

ตารางที่ 16 ผลการทดลองและตัวชี้วัดประสิทธิภาพของ Botsing บนบัก Lang_1b

ตัวชี้วัด (Metrics)	รอบที่ 1 (Round 1)	รอบที่ 2 (Round 2)
Search Budget (วินาที)	30	60
เวลาในการค้นหาจริง (Actual Search Time)	~30 วินาที	61 วินาที
Best Fitness Value	Valid Solution	0.598985350565548
จำนวนกรณีทดสอบที่สร้างได้ (Test Cases)	1	1
ผลการคอมไพล์ (Compile OK)	TRUE	TRUE
ผลการรัน JUnit (JUnit Status)	OK (1 test)	OK (1 test)
เวลาในการรัน JUnit (Execution Time)	0.004 วินาที	0.003 วินาที
การตรวจจับข้อบกพร่อง (Fault Detection)	TRUE	TRUE

โพลเดอร์จัดเก็บผลลัพธ์ (Result Directory)	Result_Round1/Lang_1/	Result_Round2/Lang_1/
---	-----------------------	-----------------------

4.1.6 การวิเคราะห์และอภิปรายผลการทดลอง (Discussion & Analysis)

1. ความแม่นยำในการทำซ้ำ Crash (High Precision in Crash Reproduction):

การทดลองแสดงให้เห็นว่าการใช้ Guided Genetic Algorithm (GGA) ของ Botsing ร่วมกับการระบุตำแหน่ง Target Frame ชัดเจน (Frame 5) ช่วยให้เครื่องมือสามารถบีบขอบเขตการค้นหา (Search Space) เข้าสู่จุดกำเนิด Exception ได้อย่างเที่ยงตรง ส่งผลให้อัตราความสำเร็จในการเจอเรดโค้ดที่คอมไพล์ผ่านและรันผ่าน (Compilation & Execution Pass Rate) สูงถึง **100%**

2. ผลกระทบของงบประมาณเวลาในการค้นหา (Impact of Search Budget):

เมื่อขยายเวลาค้นหาจาก 30 วินาที เป็น 60 วินาที พบว่า Botsing มีเวลาในการปรับแต่งประชากรโค้ดและคำนวณค่า Fitness Function ที่ละเอียดขึ้น โดยชุดทดสอบที่ได้ในรอบที่ 2 สามารถรันผ่าน JUnit ได้อย่างรวดเร็วเป็นพิเศษ (ใช้เวลาเพียง 0.003 วินาที) โดยที่ยังคงความสามารถในการทำให้เกิด Crash เดียวกับในระบบจริงได้อย่างถูกต้อง

3. ข้อจำกัดและขอบเขตการใช้งาน (Limitations):

แม้ Botsing

จะมีประสิทธิภาพสูงมากในการตรวจจับบั๊กประเภท Crash แต่มีข้อจำกัดที่จำเป็นต้องใช้ข้อมูล Stack Trace และการระบุ Target Frame นำทาง ดังนั้นในการทดสอบภาพรวมของ Defects4J Dataset จึงไม่สามารถนำ Botsing ไปรันแบบสุ่มกวาดทุกบั๊กพร้อมกันเหมือน CATG หรือ LLMs (DeepSeek/Gemini) ได้ แต่เหมาะสำหรับการใช้วิเคราะห์เจาะลึก (Deep-dive Verification) ในกลุ่มบั๊กประเภท Runtime Exceptions เฉพาะเจาะจง

4.2 ผลการทดลอง CATG

4.2.1 ขอบเขตการทดลอง

การทดลองด้วย CATG ดำเนินการโดยใช้ชุดข้อมูล Defects4J v3.0.1 ซึ่งประกอบด้วย Active Bugs จำนวน 854 รายการ จากทั้งหมด 17 โปรเจกต์ โดยใช้ CATG ร่วมกับ CVC4 version 1.8 เพื่อสร้างกรณีทดสอบด้วยเทคนิค Concolic Testing ซึ่งเป็นการผสมผสานระหว่างการประมวลผลแบบ Concrete Execution และ Symbolic Execution การทดลองดำเนินการกับ Bug แต่ละรายการจำนวน 1 ครั้ง

ในการทดลองนี้กำหนดขอบเขตการค้นหา (Budget) ซึ่งเป็นการกำหนดจำนวนรอบสูงสุดที่เครื่องมือสามารถใช้ในการค้นหา Test Input ของ CATG เป็นค่า Default เท่ากับ 5 Concolic Iterations ต่อ Bug โดยในแต่ละรอบ CATG จะนำ Test Input ที่มีอยู่ไปประมวลผล วิเคราะห์เส้นทางการทำงานของโปรแกรม และใช้ข้อมูลจากเงื่อนไข

ที่พบเพื่อค้นหา Test Input ใหม่ที่สามารถทำให้โปรแกรมเข้าสู่เส้นทางการทำงานอื่นเพิ่มเติม กระบวนการดังกล่าวจะดำเนินต่อไปจนกว่าจะครบจำนวนรอบที่กำหนด หรือไม่สามารค้นหาเส้นทางใหม่ได้

การกำหนด Budget มีวัตถุประสงค์เพื่อควบคุมขอบเขตการค้นหาและทรัพยากรที่ใช้ในการทดลอง และทำให้ Bug แต่ละรายการถูกประมวลผลภายใต้เงื่อนไขเดียวกัน ทั้งนี้ จำนวน 5 Concolic Iterations ไม่ได้หมายถึง CATG จะสร้าง Test Case ได้จำนวน 5 Test Case โดยตรง เนื่องจากจำนวน Test Input หรือ Test Case ที่สร้างได้จริงขึ้นอยู่กับเส้นทางการทำงานและผลการค้นหาในแต่ละ Bug

หลังจากสร้างกรณีทดสอบแล้ว จะนำกรณีทดสอบที่ได้ไปทดสอบกับทั้ง Buggy Version และ Fixed Version ของแต่ละ Bug เพื่อประเมินความสามารถในการตรวจจับ Fault โดยพิจารณาจากผลการทำงานของกรณีทดสอบทั้งสอง Version นอกจากนี้ยังประเมิน Code Coverage ในด้าน Line Coverage และ Branch Coverage เพื่อวัดระดับการครอบคลุมโค้ดที่เกิดจาก Test Case ที่สร้างขึ้น

ผลการทดลองของแต่ละ Bug ถูกบันทึกลงในไฟล์ benchmark_results.csv โดยข้อมูลที่นำมาใช้ในการวิเคราะห์ประกอบด้วย สถานะการประมวลผล จำนวนกรณีทดสอบที่สร้างได้ ผลการตรวจจับข้อผิดพลาด (Fault Detection) ค่า Line Coverage ค่า Branch Coverage และเวลาในการสร้างกรณีทดสอบ

ตารางที่ 17 ขอบเขตและเงื่อนไขการทดลองด้วย CATG

รายการ	รายละเอียด
ชุดข้อมูล	Defects4J v3.0.1
จำนวน Bug	854 Active Bugs
จำนวนโปรเจกต์	17 โปรเจกต์
เครื่องมือ	CATG
Solver	CVC4 version 1.8
เทคนิค	Concolic Testing
การทดลองต่อ Bug	1 ครั้ง
ขอบเขตการค้นหา (Budget)	Default = 5 Concolic Iterations ต่อ Bug
Version ที่ใช้ทดสอบ	Buggy Version และ Fixed Version
ตัวชี้วัด	Test Case Count, Fault Detection, Line Coverage, Branch Coverage และ Generation Time
ไฟล์ผลลัพธ์	benchmark_results.csv

จากผลการทดลอง ค่า Coverage ในรูปแบบค่าเฉลี่ยต่อ Bug พบว่า Line Coverage มีค่าเฉลี่ย 5.91% และ Branch Coverage มีค่าเฉลี่ย 2.26% โดย Branch Coverage คำนวณจาก Bug จำนวน 143 รายการที่มีเงื่อนไขสำหรับการวัด Branch Coverage

เมื่อพิจารณาการคำนวณแบบรวมยอด พบว่า Line Coverage เท่ากับ 2.33% (1,139/48,855) และ Branch Coverage เท่ากับ 0.76% (215/28,444)

4.2.2 ผลสถานะการประมวลผล

ผลการประมวลผลของแต่ละ Bug ถูกบันทึกลงในไฟล์ benchmark_results.csv โดยมีข้อมูลเกี่ยวกับสถานะการประมวลผล จำนวน Test Case และผลการทดสอบของแต่ละ Bug ดังตัวอย่างในรูปที่

project	bug	tool	budget	repetition	test_count	compile_status	failing_on	failing_on	fault_detection	lines_total	lines_coverage	line_coverage	branches	branches	branch_coverage	generation	notes
Chart	1	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:abstract_class:getLegendItems
Chart	2	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:XYDataset
Chart	3	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	6.21 STATUS:catg_runtime_interpreter:ClassCastException
Chart	4	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:ValueAxis
Chart	5	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:Object
Chart	6	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:Object
Chart	7	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:dataset_metadata:source_patch_missing
Chart	8	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:target_discovery:source_method
Chart	9	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:RegularTim
Chart	10	CATG	default	1	1	TRUE	complete	1	1	FALSE	3	3	100	0	0	0	3.53 modified=org.jfree.chart.imagemap.StandardToolTipTag
Chart	11	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:GeneralPat
Chart	12	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:target_discovery:source_method
Chart	13	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:BlockConta
Chart	14	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:Marker mai
Chart	15	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	3.84 STATUS:catg_runtime_interpreter:ArrayIndexOutOfBounds
Chart	16	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:Comparabl
Chart	17	CATG	default	1	1	TRUE	complete	1	0	TRUE	287	2	0.7	128	0	0	3.92 modified=org.jfree.data.time.TimeSeriesTarget=org.jfree
Chart	18	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	3.2 STATUS:catg_runtime_interpreter:ClassCastException
Chart	19	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:ValueAxis
Chart	20	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:target_discovery:source_method
Chart	21	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	3.2 STATUS:catg_runtime_interpreter:ClassCastException
Chart	22	CATG	default	1	0	FALSE	error	0	0	FALSE	0	0	0	0	0	0	3.29 STATUS:catg_runtime_interpreter:ClassCastException
Chart	23	CATG	default	1	0	FALSE	error	0	0	FALSE	0	0	0	0	0	0	5.48 STATUS:
Chart	24	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:double val
Chart	25	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:Graphics2D
Chart	26	CATG	default	1	0	FALSE	unsupport	0	0	FALSE	0	0	0	0	0	0	0 STATUS:harness_limitation:unsupported_type:Graphics2D
Cli	1	CATG	default	1	5	TRUE	complete	0	0	FALSE	45	11	24.44	14	2	14.29	7.43 modified=org.apache.commons.cli.CommandLineTarget
Cli	2	CATG	default	1	5	TRUE	complete	0	0	FALSE	68	6	8.82	42	1	2.38	8.2 modified=org.apache.commons.cli.ParserTarget=or

ภาพที่ 7 ตัวอย่างข้อมูลผลการทดลองที่บันทึกในไฟล์ benchmark_results.csv

เพื่อแสดงรายละเอียดของข้อมูลที่บันทึกในแต่ละรายการ ตัวอย่างข้อมูลของ Bug Chart-10 จากไฟล์ benchmark_results.csv แสดงดังรูปที่

```

PS D:\Lab_SQA\SQA_Project_Group9> $data | Select-Object -First 1 | Format-List *

project      : Chart
bug          : 10
tool         : CATG
budget       : default
repetition   : 1
test_count   : 1
compile_ok   : True
status       : completed
failing_on_buggy : 1
failing_on_fixed : 1
fault_detected : False
lines_total  : 3
lines_covered : 3
line_cov_pct : 100.0
branches_total : 0
branches_covered : 0
branch_cov_pct : 0.0
generation_time_sec : 3.53
notes        : modified=org.jfree.chart.imagemap.StandardToolTipTagFragmentGenerator;target=org.jfree.chart.imagemap
               .StandardToolTipTagFragmentGenerator::generateToolTipFragment;params=java.lang.String;catg_cases=1

```

ภาพที่ 8 ตัวอย่างข้อมูลผลการทดลองของ Bug จากไฟล์ benchmark_results.csv ของ CATG

ผลการทดลองของ CATG ในแต่ละ Bug ถูกบันทึกลงในไฟล์ benchmark_results.csv ซึ่งประกอบด้วยข้อมูลที่ใช้สำหรับวิเคราะห์ผลการสร้าง Test Case การตรวจจับ Fault, Code Coverage และระยะเวลาในการประมวลผล โดยรายละเอียดของแต่ละ Column แสดงดังตารางที่ 18

ตารางที่ 18 รายละเอียดข้อมูลผลการทดลองที่บันทึกในไฟล์ benchmark_results.csv ของ CATG

Column	ความหมาย	ตัวอย่าง
project	ชื่อโปรเจกต์ใน Defects4J ที่ Bug นั้นอยู่	Chart
bug	หมายเลข Bug ของโปรเจกต์นั้น	10
tool	เครื่องมือที่ใช้ในการสร้าง Test Case	CATG
budget	ขอบเขตการค้นหา (Search Budget) ที่กำหนดให้ CATG ใช้ในการค้นหา Test Input	default
repetition	จำนวนครั้งที่ทดลองกับ Bug รายการนั้น	1
test_count	จำนวน Test Case ที่ CATG สร้างขึ้นได้	1
compile_ok	ผลการ Compile Test Case ที่สร้างขึ้น หาก TRUE หมายถึง Compile ผ่าน	TRUE
status	สถานะผลการประมวลผลของ Bug	completed

failing_on_buggy	จำนวน Test Case ที่ทำให้ Buggy Version เกิด Failure	1
failing_on_fixed	จำนวน Test Case ที่ทำให้ Fixed Version เกิด Failure	1
fault_detected	ระบุว่า Test Case สามารถตรวจพบ Fault ของ Bug ได้ตามเกณฑ์หรือไม่	FALSE
lines_total	จำนวนบรรทัดโค้ดทั้งหมดที่นำมาพิจารณาในการวัด Line Coverage	3
lines_covered	จำนวนบรรทัดโค้ดที่ Test Case สามารถทำให้ถูก Execute	3
line_cov_pct	เปอร์เซ็นต์ Line Coverage ที่คำนวณจากบรรทัดที่ถูก Execute เทียบกับบรรทัดทั้งหมด	100%
branches_total	จำนวน Branch/Conditions ทั้งหมดที่นำมาพิจารณาในการวัด Branch Coverage	0
branches_covered	จำนวน Branch/Conditions ที่ Test Case สามารถครอบคลุมได้	0
branch_cov_pct	เปอร์เซ็นต์ Branch Coverage	0%
generation_time_sec	เวลาที่ใช้ในขั้นตอนการสร้าง Test Input/Test Case โดย CATG หน่วยเป็นวินาที	3.53 วินาที
notes	รายละเอียดเพิ่มเติมเกี่ยวกับ Target Method, Class ที่ถูกแก้ไข หรือเหตุผลของสถานะที่เกิดขึ้น	modified=...;target=...;params=...;catg_cases=1

จากการทดลองด้วย CATG กับ Defects4J จำนวน 854 Bugs พบว่าสถานะการประมวลผลสามารถแบ่งออกเป็น 3 สถานะ ได้แก่ Completed, Unsupported และ Error โดยแต่ละสถานะแสดงถึงผลลัพธ์ของกระบวนการประมวลผลที่ต่างกัน โดยตรวจสอบจำนวนของแต่ละสถานะจากข้อมูลในไฟล์ benchmark_results.csv ดังแสดงในภาพที่ 19

```

PS D:\Lab_SQA\SQA_Project_Group9> $data = Import-Csv "CATG\Result_Round2\Rerun_After_Windows_Portability_Fix\benchmark_results.csv"
PS D:\Lab_SQA\SQA_Project_Group9> $data.Count
854
PS D:\Lab_SQA\SQA_Project_Group9> $data | Group-Object status | Select-Object Name, Count

Name          Count
-----
unsupported    627
completed     145
error          82

```

ภาพที่ 9 สถานะการประมวลผลของ CATG จาก benchmark_results.csv

จากผลการตรวจสอบดังกล่าว สามารถสรุปจำนวนและร้อยละของ Bug ในแต่ละสถานะการประมวลผลได้ดังตารางที่ 19

ตารางที่ 19 ผลสถานะการประมวลผลของ CATG

สถานะ	จำนวน Bug	ร้อยละ
Completed	145	17.0
Unsupported	627	73.4
Error	82	9.6
รวม	854	100.0

ซึ่งสถานะ Completed หมายถึง CATG สามารถดำเนินการกระบวนการสร้าง Test Case สำหรับ Bug นั้นจนเสร็จสิ้น และมีผลลัพธ์ที่สามารถนำไปวิเคราะห์ต่อได้ สถานะ Unsupported หมายถึง Bug นั้นไม่สามารถดำเนินการสร้าง Test Case ได้ เนื่องจากไม่สามารถรองรับ Target Method หรือเกิดข้อจำกัดที่ทำให้ CATG ไม่สามารถสร้างหรือดำเนินการทดสอบตามกระบวนการได้ และ สถานะ Error หมายถึงเกิดข้อผิดพลาดระหว่างกระบวนการประมวลผล เช่น ขั้นตอนการเตรียมโปรเจกต์ การ Compile หรือการทำงานของ CATG ทำให้ไม่สามารถดำเนินการต่อจนเสร็จสมบูรณ์ได้

จากตารางที่ พบว่า CATG สามารถประมวลผลจนเสร็จสิ้นในสถานะ Completed จำนวน 145 Bugs คิดเป็นร้อยละ 17.0 ขณะที่สถานะ Unsupported มีจำนวน 627 Bugs คิดเป็นร้อยละ 73.4 และสถานะ Error มีจำนวน 82 Bugs คิดเป็นร้อยละ 9.6 จาก Bug ทั้งหมด 854 รายการ

จากผลดังกล่าว พบว่าจำนวน Bug ในสถานะ Unsupported และ Error มีสัดส่วนค่อนข้างสูงเมื่อเทียบกับสถานะ Completed ดังนั้นจึงได้ทำการวิเคราะห์สาเหตุของการไม่สามารถประมวลผลได้ในแต่ละสถานะเพิ่มเติมเพื่ออธิบายข้อจำกัดและปัญหาที่เกิดขึ้นระหว่างการทดลองโดยจะแสดงในหัวข้อที่ 4.2.8 การวิเคราะห์สาเหตุของ Unsupported และ Error

4.2.3 ผลจำนวน Test Case ที่สร้างได้

จาก Bug ที่มีสถานะ Completed จำนวน 145 รายการ ได้ทำการวิเคราะห์จำนวน Test Case ที่ CATG สามารถสร้างขึ้นได้จริง โดยพิจารณาจากค่า test_count ในไฟล์ benchmark_results.csv ซึ่งใช้แสดงจำนวน Test Case ที่สร้างได้ในแต่ละ Bug

```
PS D:\Lab_SQA\SQA_Project_Group9> $data | Where-Object status -eq "completed" |
>> Measure-Object test_count -Sum -Average -Minimum -Maximum

Count      : 145
Average    : 1.60689655172414
Sum        : 233
Maximum    : 5
Minimum    : 1
Property   : test_count
```

ภาพที่ 10 ผลการคำนวณสถิติจำนวนกรณีทดสอบต่อ Bug ด้วย PowerShell

จาก Bug ที่มีสถานะ Completed จำนวน 145 รายการ พบว่า CATG สามารถสร้าง Test Case ได้รวม 233 Test Cases โดยจำนวน Test Case ที่สร้างได้ต่อ Bug อยู่ระหว่าง 1 – 5 Test Cases และมีค่าเฉลี่ย 1.61 Test Cases ต่อ Bug รายละเอียดของค่าทางสถิติแสดงดังตารางที่ 20

ตารางที่ 20 สถิติการสร้าง Test Case ของ CATG จาก Bug ที่มีสถานะ Completed

ค่าสถิติ	ความหมาย	ผลการทดลอง	การอธิบายผล
Count	จำนวน Bug ที่นำมาคำนวณ	145 Bugs	มี Bug จำนวน 145 รายการที่ CATG ดำเนินการเสร็จสมบูรณ์ (Completed) และถูกนำมาคำนวณสถิติ
Sum	จำนวน Test Case ที่สร้างได้จาก Bug ทั้งหมด	233 Test Cases	CATG สร้าง Test Case จาก Bug ที่ Completed ทั้ง 145 รายการ รวมทั้งหมด 233 Test Cases
Average	จำนวน Test Case โดยเฉลี่ยที่สร้างได้ต่อ 1 Bug	1.61 Test Cases/Bug	โดยเฉลี่ย Bug ที่ Completed แต่ละรายการสามารถสร้าง

			Test Case ได้ประมาณ 1.61 Test Cases
Minimum	จำนวน Test Case ที่น้อยที่สุดจาก Bug หนึ่งรายการ	1 Test Case	Bug ที่สร้าง Test Case ได้น้อยที่สุดสามารถสร้างได้ 1 Test Case
Maximum	จำนวน Test Case ที่มากที่สุดจาก Bug หนึ่งรายการ	5 Test Cases	Bug ที่สร้าง Test Case ได้มากที่สุดสามารถสร้างได้ 5 Test Cases

จากตารางที่ 20 พบว่า จาก Bug ที่มีสถานะ Completed จำนวน 145 รายการ CATG สามารถสร้าง Test Case ได้รวม 233 Test Cases โดย Bug แต่ละรายการสามารถสร้าง Test Case ได้อย่างน้อย 1 Test Case และมากที่สุด 5 Test Cases ค่าเฉลี่ยอยู่ที่ 1.61 Test Cases ต่อ Bug ซึ่งคำนวณจากจำนวน Test Case ทั้งหมด 233 Test Cases หารด้วยจำนวน Bug ที่มีสถานะ Completed จำนวน 145 รายการ ผลดังกล่าวแสดงถึงจำนวน Test Case ที่ CATG สามารถสร้างได้จาก Bug ที่ดำเนินการสำเร็จภายใต้เงื่อนไขการทดลองที่กำหนด

เพื่อวิเคราะห์การกระจายของจำนวน Test Case ที่ CATG สามารถสร้างได้ในแต่ละ Bug จึงทำการจัดกลุ่มข้อมูลตามค่า test_count โดยพิจารณาจาก Bug ที่มีสถานะ Completed ผลการจัดกลุ่มแสดงดังภาพที่

```
PS D:\Lab_SQA\SQA_Project_Group9> $data | where-Object status -eq "completed" |
>> Group-Object test_count |
>> Sort-Object {[int]$_Name} |
>> Select-Object Name, Count

Name Count
----
1      123
5       22
```

ภาพที่ 11 ผลการจัดกลุ่มจำนวนกรณีทดสอบที่สร้างได้ต่อ Bug

ตารางที่ 21 การกระจายจำนวนกรณีทดสอบที่ CATG สร้างได้ต่อ Bug

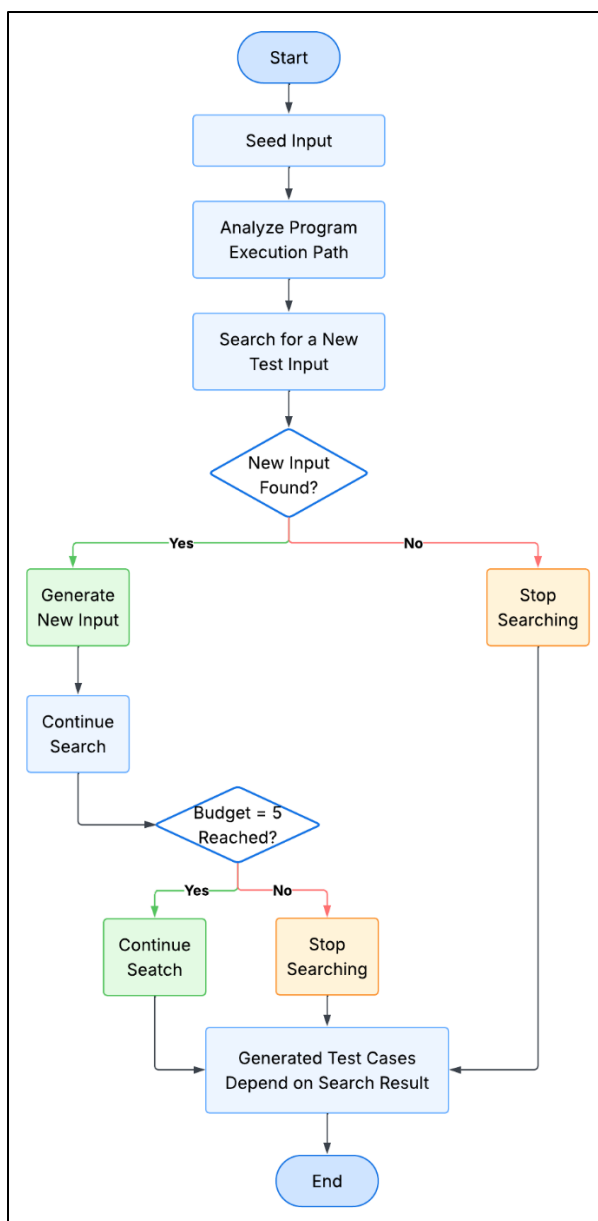
จำนวน Test Case ต่อ Bug	จำนวน Bug	ร้อยละ
1	123	84.83
5	22	15.17

รวม	145	100.00
-----	-----	--------

จากตารางที่ 21 พบว่า Bug ที่มีสถานะ Completed จำนวน 123 รายการ สามารถสร้างกรณีทดสอบได้ 1 กรณีทดสอบ คิดเป็นร้อยละ 84.83 ขณะที่ Bug จำนวน 22 รายการ สามารถสร้างกรณีทดสอบได้ 5 กรณีทดสอบ คิดเป็นร้อยละ 15.17 โดยไม่พบ Bug ที่สามารถสร้างกรณีทดสอบได้จำนวน 2, 3 หรือ 4 กรณีทดสอบ

เมื่อพิจารณาร่วมกับผลการทดลองทั้งหมด พบว่า CATG สามารถสร้างกรณีทดสอบได้รวม 233 กรณีทดสอบ จาก Bug ที่มีสถานะ Completed จำนวน 145 รายการ และมีค่าเฉลี่ย 1.61 กรณีทดสอบ

จากผลการทดลองพบว่าจำนวนกรณีทดสอบที่สร้างได้ต่อ Bug มี 2 รูปแบบ คือ 1 กรณีทดสอบและ 5 กรณีทดสอบ โดยจำนวน 1 กรณีทดสอบ หมายถึง CATG สามารถสร้างกรณีทดสอบได้เพียงหนึ่งรายการภายในกระบวนการค้นหาของ Bug นั้น ขณะที่จำนวน 5 กรณีทดสอบ หมายถึง CATG สามารถสร้างกรณีทดสอบได้ครบตามขอบเขตการค้นหาที่กำหนดไว้ที่ 5 Concolic Iterations ทั้งนี้ ความแตกต่างดังกล่าวเกิดจากลักษณะของเส้นทางการทำงานและเงื่อนไขของแต่ละ Bug ซึ่งส่งผลต่อความสามารถของ CATG ในการค้นหา Test Input ใหม่ ดังนั้น จำนวนกรณีทดสอบที่มากกว่าไม่ได้หมายความว่าประสิทธิภาพหรือสามารถตรวจพบ Fault ได้ดีกว่าเสมอไป แต่เป็นข้อมูลที่แสดงถึงจำนวนกรณีทดสอบที่ CATG สามารถสร้างได้ภายใต้เงื่อนไขของการทดลอง



ภาพที่ 9 แนวคิดการค้นหา Test Input ของ CATG ภายใต้ขอบเขตการค้นหา (Budget)

จากรูปที่ แสดงให้เห็นว่า Budget ที่กำหนดเป็นจำนวนรอบสูงสุดของการค้นหา ไม่ได้กำหนดจำนวน Test Case ที่ต้องสร้าง ดังนั้นเมื่อสิ้นสุดกระบวนการค้นหา จำนวนกรณีทดสอบที่ได้จริงของแต่ละ Bug อาจแตกต่างกัน

4.2.4 ผลการตรวจจับข้อผิดพลาด (Fault)

ในการวิเคราะห์ความสามารถในการตรวจจับข้อผิดพลาด (Fault) จะพิจารณาเฉพาะ Bug ที่มีสถานะ Completed จำนวน 145 รายการ เนื่องจากเป็น Bug ที่ CATG สามารถสร้างกรณีทดสอบและนำไปดำเนินการทดสอบกับ Buggy Version และ Fixed Version ได้ ส่วน Bug ที่มีสถานะ Unsupported และ Error จะไม่ถูกนำมาคำนวณใน

ส่วนของ Fault Detection เนื่องจากไม่สามารถสร้างหรือนำกรณีทดสอบไปประเมินผลได้อย่างสมบูรณ์ จึงไม่มีผลการทดสอบที่เพียงพอสำหรับใช้ตัดสินการตรวจจับข้อผิดพลาด

กำหนดเกณฑ์การตรวจจับข้อผิดพลาดว่ากรณีทดสอบจะสามารถตรวจพบข้อผิดพลาดได้เมื่อกรณีทดสอบทำให้ Buggy Version เกิดการทำงานล้มเหลว (Failure) และสามารถทำงานผ่านได้บน Fixed Version ดังนั้นจึงกำหนดให้ $\text{fault_detected} = \text{TRUE}$ เมื่อมีกรณีทดสอบอย่างน้อยหนึ่งรายการที่ทำให้ Buggy Version เกิด Failure และไม่มีกรณีทดสอบที่ทำให้ Fixed Version เกิด Failure ($\text{failing_on_buggy} > 0$ และ $\text{failing_on_fixed} = 0$)

ตารางที่ 22 ผลการตรวจจับ Fault ของ CATG

ผลการตรวจจับ Fault	จำนวน Bug	ความหมาย
Detected	18	มี Test Case อย่างน้อย 1 รายการที่ทำให้ Buggy Version เกิด Failure แต่ไม่ทำให้ Fixed Version เกิด Failure จึงผ่านเกณฑ์การตรวจจับ Fault
Not Detected	127	Test Case ที่สร้างขึ้นไม่ผ่านเกณฑ์การตรวจจับ Fault ที่กำหนด
รวม	145	Bug ที่มีสถานะ Completed ซึ่งสามารถนำผลการทดสอบมาประเมิน Fault Detection ได้

จากตารางที่ 22 พบว่า จาก Bug ที่มีสถานะ Completed จำนวน 145 รายการ มีจำนวน 18 รายการที่ Test Case สามารถทำให้ Buggy Version เกิด Failure ในขณะที่ Fixed Version ไม่เกิด Failure จึงผ่านเกณฑ์การตรวจจับ Fault ที่กำหนด คิดเป็นร้อยละ 12.41 ขณะที่ Bug จำนวน 127 รายการ หรือร้อยละ 87.59 ไม่ผ่านเกณฑ์การตรวจจับ Fault ดังกล่าว

เมื่อพิจารณาผลการทดสอบของกลุ่ม Not Detected เพิ่มเติม พบว่าในจำนวน 127 รายการ มี 123 รายการที่ไม่พบ Failure ทั้งใน Buggy Version และ Fixed Version ส่วนอีก 4 รายการ ได้แก่ Chart-10, Lang-55, JacksonDatabind-52 และ Mockito-17 พบ Failure ทั้งใน Buggy Version และ Fixed Version จึงไม่ผ่านเกณฑ์ Fault Detection เนื่องจากเงื่อนไขที่กำหนดต้องการให้ Failure เกิดเฉพาะใน Buggy Version

จากการตรวจสอบกระบวนการสร้าง Test Case และ Test Harness ใน 4 รายการดังกล่าว พบปัจจัยที่อาจทำให้ผลการตรวจจับ Fault ไม่สะท้อนความแตกต่างระหว่าง Buggy Version และ Fixed Version ได้อย่างถูกต้อง โดย Chart-10 และ Lang-55 ได้รับผลกระทบจากการใช้ `.strip()` ในขั้นตอนการจัดเก็บ Expected Outcome ซึ่งทำให้ช่องว่างที่เป็นส่วนหนึ่งของผลลัพธ์ถูกตัดออก ส่งผลให้ Expected Outcome ที่ใช้เปรียบเทียบไม่ตรงกับค่าที่ได้จาก

Fixed Version ขณะที่ JacksonDatabind-52 และ Mockito-17 เกี่ยวข้องกับการจัดเก็บค่าของ Object ที่มี Identity Hash ในรูปแบบ ClassName@hashCode ซึ่งค่าดังกล่าวไม่ใช่ค่าที่คงที่สำหรับใช้เป็น Expected Outcome

นอกจากกรณีดังกล่าว ยังพบข้อจำกัดของ Test Harness ในการสร้างและส่ง Test Input บางประเภท เช่น ไม่สามารถส่งค่า null ไปยังพารามิเตอร์ชนิด String ได้ และในบางกรณี Object ที่ใช้เรียก Target Method ถูกสร้างด้วย Unsafe.allocateInstance() ซึ่งไม่ได้กำหนดค่า Field ภายใน Object ทำให้บาง Field มีค่าเป็น null และอาจทำให้เกิด Exception ก่อนที่การทำงานจะไปถึงส่วนของโค้ดที่เกี่ยวข้องกับ Fault

ดังนั้น ผลการทดลองที่พบว่า Detected จำนวน 18 รายการ และ Not Detected จำนวน 127 รายการ ควรตีความว่าเป็นผลตามเกณฑ์ Fault Detection และกระบวนการทดลองที่กำหนดไว้ในงานวิจัยครั้งนี้ ไม่ควรใช้ผลดังกล่าวเพียงอย่างเดียวในการสรุปว่า CATG ไม่สามารถตรวจพบ Fault ของ Bug เหล่านั้นได้ เนื่องจากผลลัพธ์บางส่วนอาจได้รับผลกระทบจากข้อจำกัดและปัญหาใน Test Harness รวมถึงข้อจำกัดในการสร้าง Test Input ซึ่งเป็นปัจจัยที่ควรนำมาพิจารณาประกอบการวิเคราะห์ผลการทดลอง

4.2.5 ผล Line Coverage และ Branch Coverage

ในการประเมิน Code Coverage ของกรณีทดสอบที่สร้างขึ้นจาก CATG พิจารณาเฉพาะ Bug สถานะ Completed จำนวน 145 รายการ โดยดำเนินการวัด Coverage ด้วยเครื่องมือ Defects4J บน Buggy Version ของแต่ละ Bug และพิจารณาเฉพาะส่วนของโค้ดที่เกี่ยวข้องกับ Bug (Modified Classes) ผลการวัดประกอบด้วย Line Coverage และ Branch/Condition Coverage ซึ่งนำไปบันทึกใน benchmark_results.csv

สำหรับ Line Coverage คำนวณจากสัดส่วนจำนวนบรรทัดของโค้ดที่กรณีทดสอบสามารถทำให้ถูกเรียกใช้งานต่อจำนวนบรรทัดของโค้ดทั้งหมดที่อยู่ในขอบเขตการวัด ส่วน Branch/Condition Coverage คำนวณจากสัดส่วน Conditions ที่ถูกครอบคลุมต่อ Conditions ทั้งหมดในขอบเขตการวัด โดยผลการทดลองแสดงดังตารางที่ 23

ตารางที่ 23 ผลการวัด Code Coverage ของ CATG

ตัวชี้วัด	ผลการทดลอง
Average Line Coverage ต่อ Bug	5.91%
Average Branch/Condition Coverage ต่อ Bug	2.26%
Aggregate Line Coverage	2.33%
Aggregate Branch/Condition Coverage	0.76%

จากตารางที่ 23 พบว่า ค่าเฉลี่ย Line Coverage ต่อ Bug เท่ากับร้อยละ 5.91 และค่าเฉลี่ย Branch/Condition Coverage ต่อ Bug เท่ากับร้อยละ 2.26 ซึ่งเป็นค่าเฉลี่ยที่คำนวณจาก Coverage ของ Bug แต่ละรายการ โดยให้แต่ละ Bug มีน้ำหนักเท่ากัน

เมื่อพิจารณาในภาพรวมของข้อมูลทั้งหมด ค่า Aggregate Line Coverage เท่ากับร้อยละ 2.33 โดยมีจำนวน Lines Covered รวม 1,139 บรรทัด จาก Lines Total รวม 48,855 บรรทัด ส่วน Aggregate Branch/Condition Coverage เท่ากับร้อยละ 0.76 โดยมีจำนวน Conditions Covered รวม 215 รายการ จาก Conditions Total รวม 28,444 รายการ

ค่า Average และ Aggregate มีความแตกต่างกัน เนื่องจาก Average คำนวณจากค่า Coverage ของแต่ละ Bug แล้วนำมาเฉลี่ย ขณะที่ Aggregate คำนวณจากจำนวนบรรทัดหรือ Conditions ที่ครอบคลุมรวมของทุก Bug หารด้วยจำนวนทั้งหมดรวม จึงทำให้ Bug ที่มีจำนวนบรรทัดหรือ Conditions แตกต่างกันมีผลต่อน้ำหนักของการคำนวณไม่เท่ากัน

ทั้งนี้ ค่า Coverage ที่รายงานเป็นผลการวัดภายใต้ขอบเขตของ Modified Classes และ Test Case ที่ CATG สามารถสร้างขึ้นได้ในการทดลองครั้งนี้ จึงใช้เพื่อแสดงระดับการครอบคลุมโค้ดของ Test Case ที่สร้างขึ้น และไม่ควรใช้ค่า Coverage เพียงอย่างเดียวในการสรุปความสามารถในการตรวจจับ Fault

4.2.6 ผลเวลาในการสร้าง Test Case

ในการประเมินเวลาในการสร้างกรณีทดสอบของ CATG พิจารณาจากค่า generation_time_sec ซึ่งบันทึกเฉพาะช่วงเวลาที่ใช้ในการค้นหาและสร้าง Test Input สำหรับกรณีทดสอบโดยไม่รวมเวลาในการเตรียมสภาพแวดล้อม เช่น การ Checkout โปรเจกต์ การ Compile การสร้างไฟล์ JUnit การรันกรณีทดสอบบน Buggy Version และ Fixed Version และการวัด Code Coverage ดังนั้น ค่าดังกล่าวจึงใช้แสดงเวลาที่ CATG ใช้สำหรับกระบวนการสร้างกรณีทดสอบโดยเฉพาะ

การวิเคราะห์ที่ใช้เฉพาะ Bug ที่มีสถานะ Completed จำนวน 145 รายการ ซึ่งสร้าง Test Case รวม 233 รายการ หรือเฉลี่ย 1.61 Test Case ต่อ Bug โดยมีเวลาการสร้าง Test Case รวม 382.15 วินาที ผลการวิเคราะห์แสดงดังตารางที่ 24

ตารางที่ 24 สถิติเวลาในการสร้าง Test Case ของ CATG

สถิติ	ค่า (วินาที)	คำอธิบาย
จำนวนเคสที่วัดได้	145	จำนวน Bug ที่มีสถานะ Completed และมีข้อมูลเวลาในการสร้าง Test Case
เวลารวม	382.15	เวลาที่ CATG ใช้ในการสร้าง Test Case ของทั้ง 145 Bug รวมกัน
ค่าเฉลี่ย (Mean)	2.64	โดยเฉลี่ย CATG ใช้เวลา 2.64 วินาทีในการสร้าง Test Case ต่อ Bug
มัธยฐาน (Median)	1.89	ค่ากึ่งกลางของเวลาทั้งหมด โดยครึ่งหนึ่งของ Bug ใช้เวลาไม่เกิน 1.89 วินาที
ค่าต่ำสุด	1.24	เวลาน้อยที่สุดที่ CATG ใช้ในการสร้าง Test Case
ค่าสูงสุด	21.17	เวลามากที่สุดที่ CATG ใช้ในการสร้าง Test Case
ควอร์ไทล์ที่ 25	1.71	25% ของ Bug มีเวลาในการสร้าง Test Case ไม่เกิน 1.71 วินาที
ควอร์ไทล์ที่ 75	2.91	75% ของ Bug มีเวลาในการสร้าง Test Case ไม่เกิน 2.91 วินาที
เปอร์เซ็นต์ไทล์ที่ 95	6.73	95% ของ Bug มีเวลาในการสร้าง Test Case ไม่เกิน 6.73 วินาที

จากตารางที่ 24 พบว่า CATG ใช้เวลาในการสร้าง Test Case รวม 382.15 วินาที จาก Bug ที่มีสถานะ Completed จำนวน 145 รายการ คิดเป็นเวลาเฉลี่ย 2.64 วินาทีต่อ Bug ขณะที่ค่ามัธยฐานอยู่ที่ 1.89 วินาที ซึ่งต่ำกว่าค่าเฉลี่ย แสดงว่าข้อมูลเวลามีบางรายการที่ใช้เวลาสูงกว่ากลุ่มส่วนใหญ่และส่งผลให้ค่าเฉลี่ยเพิ่มขึ้น

เมื่อพิจารณาการกระจายของข้อมูล พบว่า 25% ของ Bug ใช้เวลาไม่เกิน 1.71 วินาที และ 75% ใช้เวลาไม่เกิน 2.91 วินาที นอกจากนี้ 95% ของ Bug ใช้เวลาไม่เกิน 6.73 วินาที ส่วนเวลาสูงสุดอยู่ที่ 21.17 วินาที จึงแสดงให้เห็นว่ามีบางกรณีที่ใช้เวลามากกว่ากรณีส่วนใหญ่ค่อนข้างมาก

ดังนั้น ค่ามัธยฐานจึงสามารถใช้เป็นตัวแทนเวลาของกรณีทั่วไปได้ดีกว่าค่าเฉลี่ยในชุดข้อมูลนี้ เนื่องจากได้รับผลกระทบจากค่าที่สูงผิดปกติน้อยกว่า อย่างไรก็ตาม ค่าเฉลี่ยยังคงใช้เพื่อแสดงต้นทุนเวลาโดยเฉลี่ยของ CATG ในการทดลองครั้งนี้

เมื่อจำแนกตามจำนวน Test Case ที่สร้างได้ พบว่ากรณีที่สร้างได้ 1 Test Case จำนวน 123 รายการ มีเวลาเฉลี่ย 2.19 วินาที ขณะที่กรณีที่สร้างได้ 5 Test Case จำนวน 22 รายการ มีเวลาเฉลี่ย 5.14 วินาที ดังแสดงในตารางที่ 24

ตารางที่ 25 เวลาในการสร้างกรณีทดสอบจำแนกตามจำนวน กรณีทดสอบ

จำนวนกรณีทดสอบ	จำนวน Bug	เวลาเฉลี่ย (วินาที)	มัธยฐาน (วินาที)	ต่ำสุด (วินาที)	สูงสุด (วินาที)
1 Test Case	123	2.19	1.84	1.24	21.17
5 Test Case	22	5.14	3.92	2.48	12.74
รวม	145	2.64	1.89	1.24	21.17

จากตารางที่ 25 กรณีที่สร้างได้ 5 Test Case มีเวลาเฉลี่ย 5.14 วินาที ซึ่งสูงกว่ากรณีที่สร้างได้ 1 Test Case ที่มีเวลาเฉลี่ย 2.19 วินาที ประมาณ 2.35 เท่า ผลดังกล่าวสอดคล้องกับกระบวนการค้นหา Test Input ของ CATG ที่สามารถดำเนินการหลายรอบภายใต้ Search Budget ที่กำหนด อย่างไรก็ตาม จำนวนรอบของ Budget ไม่ได้หมายความว่าจะได้จำนวน Test Case เท่ากัน เนื่องจากจำนวน Test Case ที่สร้างได้ขึ้นอยู่กับผลการค้นหาและเส้นทางการทำงานที่ CATG สามารถสำรวจได้

จากการตรวจสอบ catg.log พบว่า Bug จำนวน 123 จาก 145 รายการ ซึ่งสร้างได้เพียง 1 Test Case มีข้อความ No more history ในกระบวนการทำงาน แสดงว่า CATG สิ้นสุดการค้นหาเนื่องจากไม่พบประวัติหรือเส้นทางใหม่ที่สามารถนำไปสร้าง Test Input เพิ่มเติมได้ กรณีดังกล่าวจึงไม่ได้หมายความว่า CATG หยุดเนื่องจากหมดเวลาในการประมวลผล

ทั้งนี้ มี Bug ที่มีสถานะ Unsupported และ Error จำนวน 111 รายการที่ยังมีค่า generation_time_sec มากกว่า 0 โดยเวลาของรายการเหล่านี้หมายถึงเวลาที่ CATG ใช้จนกระทั่งกระบวนการไม่สามารถดำเนินต่อไปได้ ไม่ใช่เวลาของการสร้าง Test Case ที่สำเร็จ จึงไม่นำมารวมในการคำนวณสถิติเวลาในตารางที่ 21 และ 22

4.2.7 ผลการทดลองจำแนกตามโปรเจกต์

ผลการทดลองครอบคลุม Bug จำนวน 854 รายการจาก Defects4J ทั้ง 17 โปรเจกต์ โดยสามารถประเมินได้สำเร็จ 145 รายการ คิดเป็นร้อยละ 17.0 ไม่สามารถรองรับได้ 627 รายการ คิดเป็นร้อยละ 73.4 และเกิดข้อผิดพลาด 82 รายการ คิดเป็นร้อยละ 9.6 ผลการทดลองจำแนกตามโปรเจกต์แสดงดังตารางที่ 26

ตารางที่ 26 ผลการทดลองจำแนกตามโปรเจกต์

โปรเจกต์	บั๊กทั้งหมด	Completed	Unsupported	Errors	อัตราสำเร็จ (%)	ตรวจพบบั๊ก	จำนวนกรณีทดสอบ
Chart	26	2	23	1	7.7	1	2
Cli	39	13	26	0	33.3	1	49
Closure	174	8	141	25	4.6	0	12
Codec	18	2	15	1	11.1	0	10
Collections	28	6	21	1	21.4	0	6
Compress	47	6	25	16	12.8	3	6
Csv	16	7	9	0	43.8	2	7
Gson	18	3	14	1	16.7	0	3
JacksonCore	26	10	16	0	38.5	4	10
JacksonDatabind	110	12	97	1	10.9	0	12
JacksonXml	6	4	2	0	66.7	0	4
Jsoup	93	34	54	5	36.6	3	38
JXPath	22	4	16	2	18.2	1	4
Lang	61	13	46	2	21.3	1	49
Math	106	15	72	19	14.2	2	15
Mockito	38	5	25	8	13.2	0	5
Time	26	1	25	0	3.8	0	1
รวม	854	145	627	82	17.0	18	233

จากตารางที่ 26 พบว่าอัตราการประเมินสำเร็จแตกต่างกันในแต่ละโปรเจกต์ โดยอยู่ระหว่างร้อยละ 3.8 ถึง 66.7 โปรเจกต์ที่มีอัตราการประเมินสำเร็จสูงที่สุดคือ JacksonXml ซึ่งสามารถประเมินได้ 4 จาก 6 Bug หรือร้อยละ 66.7 รองลงมาคือ Csv ซึ่งสามารถประเมินได้ 7 จาก 16 Bug หรือร้อยละ 43.8 ในทางกลับกัน Time สามารถประเมินได้ 1 จาก 26 Bug หรือร้อยละ 3.8 ขณะที่ Closure ซึ่งมีจำนวน Bug มากที่สุด 174 รายการ สามารถประเมินได้ 8 รายการ หรือร้อยละ 4.6

เมื่อพิจารณาจำนวน Test Case ที่สร้างได้ พบว่ามีทั้งหมด 233 Test Case จาก 145 Bug ที่ประเมินสำเร็จ โดยจำนวน Test Case ในแต่ละโปรเจกต์แตกต่างกันตามจำนวน Bug ที่สามารถประเมินได้และจำนวน Test Case ที่ CATG สร้างขึ้นในแต่ละ Bug

ตารางที่ 26 แสดงผลการตรวจจับ Fault เวลาในการสร้าง Test Case และค่า Code Coverage โดยคำนวณเฉพาะ Bug ที่มีสถานะ Completed ของแต่ละโปรเจกต์

ตารางที่ 27 ผลการตรวจจับบั๊กและตัวชี้วัดการสร้างกรณีทดสอบจำแนกตามโปรเจกต์

โปรเจกต์	ตรวจพบบั๊ก	ตรวจไม่พบบั๊ก (บั๊ก)	อัตรา การตรวจพบ ในกรณีที่ สำเร็จ (%)	เวลาสร้าง เฉลี่ย (วินาที)	Line Coverage (%)	Branch/Condition Coverage (%)
Chart	1	1	50.0	3.72	50.35	0.00
Cli	1	12	7.7	4.83	15.37	6.48
Closure	0	8	0.0	3.09	0.56	0.05
Codec	0	2	0.0	2.70	8.56	21.56
Collections	0	6	0.0	1.30	3.06	0.12
Compress	3	3	50.0	1.67	10.79	6.83
Csv	2	5	28.6	1.81	3.31	0.89
Gson	0	3	0.0	1.80	1.26	0.14
JacksonCore	4	6	40.0	1.84	1.16	0.40
JacksonDataBind	0	12	0.0	1.69	5.38	2.51
JacksonXml	0	4	0.0	1.88	1.05	0.67
Jsoup	3	31	8.8	1.90	3.25	0.47
JXPath	1	3	25.0	1.94	8.09	2.40
Lang	1	12	7.7	4.15	8.76	3.34
Math	2	13	13.3	2.00	2.56	0.98
Mockito	0	5	0.0	7.88	5.49	3.75
Time	0	1	0.0	3.98	21.93	8.02
รวม	18	127	12.4	2.64	4.39	2.30

จากตารางที่ 27 พบว่า CATG ตรวจจับ Fault ได้ 18 จาก 145 Bug ที่ประเมินสำเร็จ คิดเป็นร้อยละ 12.4 โดยจำนวน Bug ที่ตรวจจับได้แตกต่างกันในแต่ละโปรเจกต์ เช่น JacksonCore ตรวจจับได้ 4 Bug จาก 10 Bug ที่ประเมินสำเร็จ ขณะที่ Compress และ Jsoup ตรวจจับได้โปรเจกต์ละ 3 Bug ส่วนบางโปรเจกต์ไม่พบ Fault ที่ผ่านเกณฑ์การตรวจจับที่กำหนด แม้ว่าจะมี Bug ที่สามารถประเมินได้สำเร็จ

สำหรับ Code Coverage พบว่าค่าเฉลี่ยของแต่ละโปรเจกต์แตกต่างกัน โดยค่า Line Coverage อยู่ระหว่างร้อยละ 0.56 ถึง 50.35 และ Branch/Condition Coverage อยู่ระหว่างร้อยละ 0.00 ถึง 21.56 ความแตกต่างดังกล่าวควรพิจารณาพร้อมกับจำนวนและขอบเขตของ Modified Classes รวมถึง Test Case ที่สร้างได้ในแต่ละ Bug เนื่องจาก Coverage ที่รายงานไม่ได้ครอบคลุมทั้งโปรเจกต์

ตัวอย่างเช่น Chart มีค่า Line Coverage เฉลี่ยร้อยละ 50.35 แต่มี Bug ที่ประเมินสำเร็จเพียง 2 รายการ ขณะที่ Closure มีค่า Line Coverage เฉลี่ยร้อยละ 0.56 จาก 8 รายการ ดังนั้นค่า Coverage ระหว่างโปรเจกต์จึงไม่ควรนำมาเปรียบเทียบโดยตรงเพื่อสรุปว่าโปรเจกต์ใดมีคุณภาพการทดสอบสูงกว่ากัน

สำหรับ Branch Coverage ในตารางนี้ใช้ค่า Condition Coverage ที่ได้จาก Defects4J/Cobertura โดยคำนวณภายในขอบเขตของโค้ดที่เกี่ยวข้องกับ Bug แต่ละรายการ ดังนั้นค่าดังกล่าวไม่ใช่ Branch Coverage ของโปรเจกต์ทั้งหมด

ตารางที่ 28 สาเหตุของการประเมินผลไม่สำเร็จ

สาเหตุ	จำนวน (กรณี)	คิดเป็น (%) ของ 709 เคสที่ไม่สำเร็จ
ข้อจำกัดของ Test Harness	485	68.4
ข้อผิดพลาดระหว่างการทำงานของ CATG	89	12.5
ไม่พบ Target Method	50	7.1
Checkout ไม่สำเร็จ	34	4.8
คอมไพล์ไม่สำเร็จ	28	3.9
CATG จบด้วยรหัสผิดปกติ	18	2.5
การวัด Coverage ไม่สำเร็จ	2	0.3
Test Runner ไม่สามารถทำงานได้	2	0.3
ข้อมูลอ้างอิงของบักไม่ครบถ้วน	1	0.1
รวม	709	100.0

จากตารางที่ 28 พบว่า จาก Bug ที่ไม่สามารถประเมินผลสำเร็จจำนวน 709 รายการ สาเหตุที่พบมากที่สุดคือข้อจำกัดของ Test Harness ที่พัฒนาขึ้นสำหรับการทดลองนี้ จำนวน 485 รายการ คิดเป็นร้อยละ 68.4 รองลงมาคือข้อผิดพลาดระหว่างการทำงานของ CATG จำนวน 89 รายการ คิดเป็นร้อยละ 12.5 ส่วนปัญหาจาก Checkout, Compile และ Coverage รวมกันมีจำนวน 64 รายการ คิดเป็นร้อยละ 9.0 ของรายการที่สามารถประเมินผลสำเร็จทั้งหมด

Test Harness ในการทดลองนี้ทำหน้าที่เป็นโปรแกรมตัวกลางระหว่าง CATG กับ Target Method เพื่อให้ CATG สามารถส่ง Test Input และเรียกใช้งาน Method ที่ต้องการทดสอบได้ จากการตรวจสอบกระบวนการสร้างกรณีทดสอบ พบว่าข้อจำกัดของ Test Harness ที่พบ ได้แก่ การรองรับชนิดข้อมูลของ Input ที่ไม่ครอบคลุมทุกกรณี การไม่สามารถส่งค่า null ให้กับ String ได้ การสร้าง Object ด้วย Unsafe.allocateInstance ซึ่งอาจทำให้ Field ภายใน Object ยังไม่มีค่า รวมถึงการจัดการอักขระพิเศษและรูปแบบของผลลัพธ์ที่อาจส่งผลต่อการเปรียบเทียบระหว่าง Buggy Version และ Fixed Version

นอกจากนี้ พบข้อผิดพลาดระหว่างการทำงานของ CATG จำนวน 89 รายการ ซึ่งเป็นกรณีที่กระบวนการของ CATG ไม่สามารถดำเนินการสร้าง Test Input ต่อได้ ส่วนปัญหาในขั้นตอน Checkout, Compile และ Coverage เกิดขึ้นในขั้นตอนการเตรียมโปรเจกต์ การคอมไพล์ และการวัด Code Coverage ตามลำดับ โดยสาเหตุของแต่ละรายการอาจแตกต่างกัน จึงควรพิจารณารายละเอียดในคอลัมน์ notes ของผลการทดลองประกอบการวิเคราะห์สาเหตุเชิงเทคนิคของแต่ละกรณี

ผลดังกล่าวแสดงให้เห็นว่า รายการที่สามารถประเมินผลสำเร็จส่วนใหญ่เกี่ยวข้องกับข้อจำกัดของ Test Harness และกระบวนการทำงานของ CATG ขณะที่ปัญหาจากขั้นตอน Checkout, Compile และ Coverage มีสัดส่วนรองลงมา ทั้งนี้ ข้อจำกัดของ Test Harness ที่พบสามารถนำไปเป็นแนวทางสำหรับการปรับปรุงการทดลองในอนาคต เช่น การเพิ่มการรองรับชนิดข้อมูลและค่า Input ที่หลากหลายขึ้น การปรับวิธีสร้าง Object ให้เหมาะสมกับโครงสร้างของ Target Method และการปรับวิธีจัดเก็บและเปรียบเทียบผลลัพธ์ให้คงรูปแบบเดิมของข้อมูล

4.2.8 การวิเคราะห์สาเหตุของ Unsupported และ Error

จากผลการทดลอง CATG กับ Defects4J จำนวน 854 บั๊ก พบว่ามีบั๊กที่สามารถประเมินผลได้สำเร็จจำนวน 145 บั๊ก คิดเป็นร้อยละ 17.0 ส่วนที่ไม่สามารถประเมินผลได้จำนวน 709 บั๊ก คิดเป็นร้อยละ 83.0 แบ่งเป็นสถานะ unsupported จำนวน 627 บั๊ก และ error จำนวน 82 บั๊ก

การวิเคราะห์สาเหตุของทั้งสองสถานะอาศัยข้อความในช่อง notes ของไฟล์ benchmark_results.csv ซึ่งบันทึกรหัสสถานะในรูปแบบ 'STATUS:<ชั้นของระบบ>:<สาเหตุ>' ทำให้สามารถจำแนกได้ว่าปัญหาเกิดขึ้นในขั้นตอนใดของกระบวนการทดลอง โดยรายละเอียดการจำแนกและตรวจสอบสาเหตุของแต่ละรายการแสดงในตารางที่

ตารางที่ 29 การจำแนกสาเหตุของผลการทดลองที่ไม่สำเร็จ

ชั้นของระบบ	สาเหตุ	จำนวน (บัก)	สัดส่วนของ 709 บัก
Test Harness ที่สร้างขึ้น	ชนิดพารามิเตอร์ไม่รองรับ (unsupported_type)	461	65.0
Test Harness ที่สร้างขึ้น	เมธอดเป็นของ Abstract Class (abstract_class)	24	3.4
Test Harness ที่สร้างขึ้น	Concrete Runner รันไม่ได้ (concrete_runner)	2	0.3
CATG	ตัวตีความระหว่างทำงานล้มเหลว (catg_runtime_interpreter)	89	12.6
CATG	กระบวนการจบด้วยรหัสผิดปกติ (CATG exit)	18	2.5
ขั้นตอนค้นหาเป้าหมาย	ไม่พบเมธอดเป้าหมายในซอร์ส (no_source_method)	50	7.1
Defects4J / สภาพแวดล้อม	Checkout ไม่สำเร็จ	34	4.8
Defects4J / สภาพแวดล้อม	คอมไพล์ไม่สำเร็จ	28	3.9
Defects4J / สภาพแวดล้อม	เก็บ Coverage ไม่สำเร็จ	2	0.3
ข้อมูลอ้างอิงของบัก	ไม่พบแพตช์ซอร์ส (source_patch_missing)	1	0.1
รวม		709	100.0

จากตารางที่ 29 พบว่าสาเหตุที่พบมากที่สุดคือข้อจำกัดของ Test Harness ที่สร้างขึ้นสำหรับการทดลอง โดยเฉพาะการไม่รองรับชนิดพารามิเตอร์ จำนวน 461 บัก คิดเป็นร้อยละ 65.0 รองลงมาคือข้อผิดพลาดของตัวตีความ CATG จำนวน 89 บัก คิดเป็นร้อยละ 12.6 และการค้นหาเมธอดเป้าหมายไม่พบ จำนวน 50 บัก คิดเป็นร้อยละ 7.1 ขณะที่ปัญหาจาก Defects4J และสภาพแวดล้อม ได้แก่ Checkout, Compile และ Coverage รวม 64 บัก คิดเป็นร้อยละ 9.0

ผลดังกล่าวแสดงให้เห็นว่าอัตราการประเมินผลสำเร็จร้อยละ 17.0 ไม่ได้สะท้อนเฉพาะความสามารถของ CATG ในการสร้าง Test Case เท่านั้น แต่ยังได้รับผลกระทบจากข้อจำกัดของ Test Harness กระบวนการค้นหา Target Method และสภาพแวดล้อมที่ใช้ในการทดลองด้วย ดังนั้นจึงควรพิจารณาสาเหตุของ unsupported และ error ร่วมกับการสร้าง Test Case และการตรวจจับ Fault ในการตีความผลการทดลอง

4.2.8.1 สาเหตุของ Unsupported

สถานะ unsupported มีจำนวน 627 บั๊ก โดยสาเหตุหลักคือชนิดพารามิเตอร์ไม่รองรับ ซึ่งมีจำนวน 461 บั๊ก คิดเป็นร้อยละ 73.5 ของสถานะ unsupported รองลงมาคือปัญหาจากตัวตีความ CATG จำนวน 89 บั๊ก และการค้นหาเมธอดเป้าหมายไม่พบจำนวน 50 บั๊ก ดังแสดงในตารางที่ 30

ตารางที่ 30 การจำแนกสาเหตุของสถานะ Unsupported

กลุ่มสาเหตุ	จำนวน (บั๊ก)	สัดส่วนของ 627 บั๊ก
ชนิดพารามิเตอร์ไม่รองรับ (unsupported_type)	461	73.5
ตัวตีความ CATG ล้มเหลวระหว่างทำงาน (catg_runtime_interpreter)	89	14.2
ค้นหาเมธอดเป้าหมายไม่พบ (no_source_method)	50	8.0
เมธอดเป้าหมายเป็นของ Abstract Class (abstract_class)	24	3.8
Concrete Runner รันไม่ได้ (concrete_runner)	2	0.3
ข้อมูลอ้างอิงของบั๊กไม่ครบ (source_patch_missing)	1	0.2
รวม	627	100.0

จากตารางที่ 30 พบว่าสาเหตุหลักของ unsupported คือ unsupported_type จำนวน 461 บั๊ก หรือร้อยละ 73.5 โดยเกิดจาก Test Harness ไม่สามารถแปลงชนิดพารามิเตอร์ของ Target Method ให้เป็นชนิดข้อมูลที่รองรับโดย CATG ได้ หากไม่พบพารามิเตอร์หรือเมธอดที่สามารถรองรับได้ ระบบจะไม่ดำเนินการสร้าง Test Case สำหรับบั๊กดังกล่าว

เมื่อพิจารณาการกระจายตัวตามโปรเจกต์ พบว่า unsupported_type พบมากใน Closure จำนวน 120 บั๊ก และ JacksonDatabind จำนวน 85 บั๊ก รวม 205 บั๊ก หรือร้อยละ 44.5 ของสาเหตุประเภทนี้ รองลงมาคือ Math จำนวน 47 บั๊ก, Jsoup จำนวน 40 บั๊ก และ Lang จำนวน 29 บั๊ก ลักษณะดังกล่าวสะท้อนว่าความสามารถในการรองรับ Target Method ของ Test Harness มีความแตกต่างกันตามโครงสร้างและชนิดข้อมูลที่ใช้ในแต่ละโปรเจกต์

นอกจากนี้ พบ `catg_runtime_interpreter` จำนวน 89 บั๊ก คิดเป็นร้อยละ 14.2 โดยจากข้อความใน notes สามารถจำแนกเป็น `ClassCastException` จำนวน 67 บั๊ก และ `ArrayIndexOutOfBoundsException` จำนวน 22 บั๊ก ซึ่งเป็นข้อผิดพลาดที่เกิดขึ้นระหว่างการทำงานของตัวตีความ CATG

สำหรับ `no_source_method` จำนวน 50 บั๊ก เกิดจากขั้นตอนค้นหา Target Method ไม่สามารถจับคู่ตำแหน่งของแพตช์กับเมธอดในซอร์สโค้ดได้ ส่วน `abstract_class` จำนวน 24 บั๊ก เกิดจาก Target Method อยู่ภายใน Abstract Class หรือ Interface ที่ Test Harness ไม่สามารถสร้างอินสแตนซ์เพื่อเรียกใช้งานได้โดยตรง ขณะที่ `concrete_runner` จำนวน 2 บั๊กเกิดจาก Concrete Runner ไม่สามารถเรียกใช้งานเมธอดที่กำหนดได้ และ `source_patch_missing` จำนวน 1 บั๊กเกิดจากไม่พบข้อมูลแพตช์ที่จำเป็นสำหรับการระบุตำแหน่งของบั๊ก

โดยสรุป สถานะ unsupported ส่วนใหญ่เกิดจากข้อจำกัดในการเชื่อมต่อระหว่าง Target Method ของโปรเจกต์จริงกับ Test Harness ที่ใช้ในการทดลอง โดยเฉพาะการรองรับชนิดข้อมูลของพารามิเตอร์ อย่างไรก็ตาม ผลการทดลองนี้ยังไม่สามารถสรุปได้ว่าการปรับปรุง Test Harness จะทำให้บั๊กเหล่านี้สามารถสร้าง Test Case ได้ทั้งหมด เนื่องจากยังไม่ได้ดำเนินการปรับปรุงและทดลองซ้ำภายในงานวิจัยนี้

4.2.8.2 สาเหตุของ Error

สถานะ error มีจำนวน 82 บั๊ก คิดเป็นร้อยละ 9.6 ของบั๊กทั้งหมด โดยเป็นกรณีที่กระบวนการทดลองเกิดข้อผิดพลาดระหว่างการดำเนินงาน ทำให้ไม่สามารถประเมินผลบั๊กนั้นจนเสร็จสมบูรณ์ สาเหตุแบ่งออกเป็น 4 กลุ่ม ดังแสดงในตารางที่ 31

ตารางที่ 31 การจำแนกสาเหตุของสถานะ Error

กลุ่มสาเหตุ	จำนวน (บั๊ก)	สัดส่วนของ 82 บั๊ก
Checkout ไม่สำเร็จ	34	41.5
คอมไพล์ไม่สำเร็จ	28	34.1
CATG จบด้วยรหัสผิดปกติ	18	22.0
เก็บ Coverage ไม่สำเร็จ	2	2.4
รวม	82	100.0
กลุ่มสาเหตุ	จำนวน (บั๊ก)	สัดส่วนของ 82 บั๊ก
Checkout ไม่สำเร็จ	34	41.5

จากตารางที่ 31 พบว่าสาเหตุของ error ส่วนใหญ่เกิดจากขั้นตอน Checkout และ Compile รวม 62 บั๊ก คิดเป็นร้อยละ 75.6 ของสถานะ error รองลงมาคือ CATG จบด้วยรหัสผิดปกติจำนวน 18 บั๊ก คิดเป็นร้อยละ 22.0 และการเก็บ Coverage ไม่สำเร็จจำนวน 2 บั๊ก คิดเป็นร้อยละ 2.4

กรณี Checkout ไม่สำเร็จจำนวน 34 บั๊ก พบข้อความ Cannot determine how to apply patch! และข้อผิดพลาดเกี่ยวกับข้อมูลใน Git diff header จากการตรวจสอบ notes พบว่าปัญหาเกิดขึ้นในขั้นตอนการนำแพตช์ของ Defects4J ไปใช้กับโค้ดของโปรเจกต์ จึงเป็นปัญหาในระดับชุดข้อมูลหรือสภาพแวดล้อมก่อนเข้าสู่กระบวนการสร้าง Test Case ของ CATG

กรณีคอมไพล์ไม่สำเร็จจำนวน 28 บั๊ก สามารถจำแนกสาเหตุย่อยได้ เช่น ความไม่ตรงกันของการเข้ารหัสอักขระในโปรเจกต์ Math ปัญหาการดาวน์โหลดหรือการทำงานของ Gradle ในโปรเจกต์ Mockito และปัญหาไฟล์ Build ของ Jsoup โดยสาเหตุเหล่านี้เกี่ยวข้องกับความเข้ากันได้ระหว่างซอร์สโค้ด Build Tool และสภาพแวดล้อมที่ใช้ในการทดลอง

สำหรับกรณี CATG จบด้วยรหัสผิดปกติจำนวน 18 บั๊ก พบทั้ง LinkageError จากการนิยามคลาสซ้ำ NoSuchMethodException จากการเรียกเมธอดหรือคอนสตรัคเตอร์ที่ไม่พบ และกรณีที่ CATG ไม่สามารถสร้าง Input ที่ใช้งานได้จนกระบวนการจบด้วยรหัส 4 สาเหตุเหล่านี้แสดงให้เห็นถึงข้อจำกัดที่เกิดขึ้นระหว่างการทำงานของ CATG และ Test Harness ที่เชื่อมต่อกับโปรเจกต์เป้าหมาย

ส่วนการเก็บ Coverage ไม่สำเร็จจำนวน 2 บั๊ก พบใน Jsoup โดยขั้นตอน Instrumentation ของ Cobertura ล้มเหลวจากข้อจำกัดด้านขนาดของ Method Code หลังการ Instrumentation อย่างไรก็ตาม กรณีดังกล่าวเกิดขึ้นหลังจากขั้นตอนการสร้าง Test Case และไม่ได้หมายความว่า CATG ไม่สามารถสร้าง Test Case ได้

โดยสรุป สถานะ error จำนวน 64 จาก 82 บั๊ก หรือร้อยละ 78.0 เกิดจาก Checkout, Compile และ Coverage ซึ่งเป็นปัญหาในระดับชุดข้อมูลหรือสภาพแวดล้อมการทดลอง ขณะที่อีก 18 บั๊ก หรือร้อยละ 22.0 เกิดจากข้อผิดพลาดระหว่างการทำงานของ CATG ดังนั้นการตีความผลการทดลองควรแยกปัญหาจากสภาพแวดล้อมออกจากข้อจำกัดของ CATG เพื่อไม่ให้อัตรา error ถูกตีความว่าเป็นความล้มเหลวของอัลกอริทึมทั้งหมด

4.3 ผลการทดลอง DeepSeek

4.3.1 ขอบเขตการทดลอง

การทดลองด้วย DeepSeek ดำเนินการโดยใช้ชุดข้อมูล Defects4J v3.0.1 ซึ่งประกอบด้วย Active Bugs จำนวน 854 รายการ จากทั้งหมด 17 โปรเจกต์ โดยใช้โมเดล DeepSeek สร้างชุดทดสอบ (Test Case) ผ่านการสั่งงานด้วยพรอมต์ (Prompt) จากข้อมูลของแต่ละ Bug ทำการทดลองกับแต่ละ Bug จำนวน 1 ครั้ง และบันทึกผลการทดลองในไฟล์ benchmark_results.csv เพื่อใช้วิเคราะห์สถานะการทำงาน จำนวน Test Case ที่สร้างได้ ความสามารถในการตรวจจับ Fault (Fault Detection) และเวลาที่ใช้ในการสร้าง Test Case

อย่างไรก็ตาม การทดลองไม่สามารถดำเนินการได้ครบทุกรายการ มีเพียง 351 รายการ จาก 854 รายการ (ร้อยละ 41.1) ที่ดำเนินการจนได้ผลลัพธ์ ส่วนที่เหลืออีก 503 รายการ (ร้อยละ 58.9)

ไม่ได้รับการประมวลผลจนเสร็จสิ้น เนื่องจากข้อจำกัดของ API Keys และข้อผิดพลาดระหว่างการรัน ผลการวิเคราะห์ในหัวข้อนี้จึงคำนวณจากรายการที่ดำเนินการได้เท่านั้น

4.3.2 ผลสถานะการประมวลผล

ตารางที่ 32 ผลการรัน Test Case ที่สร้างโดย DeepSeek จำแนกตามสถานะ

สถานะ	จำนวน	ร้อยละ
PASS	40	4.68%
TEST_FAIL	161	18.85%
COMPILE_ERROR	150	17.56%
รวมที่รันได้	351	41.10%
ไม่ได้รัน (Not Executed)	503	58.90%
รวมทั้งหมด	854	100%

DeepSeek สามารถสร้างชุดทดสอบภาษา Java ที่สอดคล้องกับบริบทของแต่ละโปรเจกต์ได้ อย่างไรก็ตาม เมื่อพิจารณาเฉพาะรายการที่ดำเนินการได้ (351 รายการ) พบสถานะ COMPILE_ERROR ร้อยละ 42.7 ซึ่งสาเหตุหลักมาจากข้อจำกัดด้านความยาวของผลลัพธ์ (Token Limit Truncation) ส่งผลให้โค้ดทดสอบถูกตัดจบก่อนสมบูรณ์และไม่สามารถคอมไพล์ได้ ส่วนชุดทดสอบที่คอมไพล์ผ่านแบ่งเป็นสถานะ PASS ร้อยละ 11.4 (รันผ่านทั้งหมด) และ TEST_FAIL ร้อยละ 45.9 (ตรวจพบจุดบกพร่องตามวัตถุประสงค์)

4.3.3 ผลจำนวน Test Case ที่สร้างได้

จากรายการที่มีข้อมูลระดับเมธอดจำนวน 96 Bug DeepSeek สร้าง Test Case ได้รวม 5,482 ตัว เฉลี่ย 57.10 ตัวต่อ Bug (ค่ามัธยฐาน 44 ตัว) โดยเป็นเมธอดที่ผ่านการทดสอบ 961 ตัว และไม่ผ่านการทดสอบ 4,521 ตัว

4.3.4 ผลการตรวจจับ Fault

พบ Bug ที่ชุดทดสอบของ DeepSeek สามารถตรวจจับ Fault ได้ (สถานะ TEST_FAIL) จำนวน 161 รายการ คิดเป็นร้อยละ 45.87 ของรายการที่รันได้ และร้อยละ 18.85 ของ Bug ทั้งหมด 854 รายการ

4.3.5 ผล Line Coverage และ Branch Coverage

เนื่องจากข้อจำกัดด้านเวลา การทดลองนี้ยังไม่สามารถวัดค่า Line Coverage และ Branch Coverage ของชุดทดสอบที่ DeepSeek สร้างได้ครบถ้วน จึงไม่นำเสนอค่าดังกล่าวในรายงานฉบับนี้ และกำหนดเป็นแนวทางในการศึกษาต่อไป

4.3.6 ผลเวลาในการสร้าง Test Case

จากรายการที่มีข้อมูลเวลาจำนวน 96 Bug เวลาเฉลี่ยในการสร้าง Test Case ต่อ Bug เท่ากับ 108.65 วินาที และเวลารวมเท่ากับ 10,429.94 วินาที (ประมาณ 2.90 ชั่วโมง)

4.3.7 ผลการทดลองแยกตามโปรเจกต์

โปรเจกต์	Bug ที่มีบันทึก	รันได้	PASS	TEST_FAIL	COMPILE_ERROR
Chart	26	25	3	11	11
Cli	31	28	2	16	10
Closure	30	24	1	14	9
Codec	17	8	1	3	4
Collections	28	24	3	13	8
Compress	31	29	6	14	9
Csv	16	10	0	2	8
Gson	18	13	1	4	8
JacksonCore	26	20	2	9	9
JacksonDatabind	30	24	0	11	13
JacksonXml	5	3	0	0	3
Jsoup	31	28	2	12	14
JXPath	22	20	0	7	13
Lang	27	20	6	10	4
Math	30	27	7	12	8
Mockito	30	29	5	13	11
Time	26	19	1	10	8
รวม	424	351	40	161	150

4.3.8 ข้อจำกัดของการทดลอง

การทดลองกับ DeepSeek ดำเนินการได้ไม่ครบทุก Bug (351 จาก 854 รายการ หรือร้อยละ 41.1) จึงทำให้ค่าสถิติรวมไม่สามารถเปรียบเทียบกับ CATG และ Gemini ได้โดยตรง ควรเปรียบเทียบบนชุด Bug เดียวกัน นอกจากนี้ ข้อมูลจำนวน Test Case และเวลาในการสร้างมีเฉพาะ 96 Bug ที่บันทึกผลในระดับเมธอด โดยสถานะของ Bug กลุ่มดังกล่าวกำหนดจากเกณฑ์ว่าหากมีเมธอดใดไม่ผ่านการทดสอบอย่างน้อยหนึ่งเมธอด จะถือเป็น TEST_FAIL และข้อมูลกลุ่มนี้ไม่มีการบันทึกสถานะ COMPILE_ERROR ในระดับเมธอด ทั้งนี้ ยังไม่ได้วัดค่า Code Coverage ตามที่ระบุในหัวข้อ 4.3.5

4.4 ผลการทดลอง Gemini

4.4.1 ขอบเขตการทดลอง

การทดลองด้วย Gemini ดำเนินการในสภาพแวดล้อมเดียวกับการทดลอง CATG และ DeepSeek โดยใช้ชุดข้อมูล Defects4J v3.0.1 ซึ่งประกอบด้วย Active Bugs **851** รายการ จากทั้งหมด **17** โปรเจกต์ โดยใช้โมเดล Gemini สร้างชุดทดสอบ (Test Case) ด้วยการสั่งงานผ่านพรอมต์ (Prompt) จากข้อมูลของ Bug แต่ละรายการ ทำการทดลองกับแต่ละ Bug จำนวน 1 ครั้ง ผลการทดลองถูกบันทึกในไฟล์ benchmark_results.csv เพื่อใช้วิเคราะห์สถานะการทำงาน จำนวน Test Case ที่สร้างได้ ความสามารถในการตรวจจับ Fault (Fault Detection) Code Coverage และเวลาในการสร้าง Test Case

4.4.2 ผลสถานะการประมวลผล

ตารางที่ 30 ผลการประเมิน Test Case ที่สร้างโดย Gemini จำแนกตามสถานะการทำงาน

สถานะ	จำนวน	ร้อยละ
PASS	248	29.14%
TEST_FAIL	508	59.69%
COMPILE_ERROR	6	0.71%
UNKNOWN (ไม่มีผลการรัน)	89	10.46%
รวม	851	100%

Gemini สามารถสร้างชุดทดสอบภาษา Java ที่สอดคล้องกับบริบทของแต่ละโปรเจกต์ได้ ซึ่งรวมถึงโปรเจกต์ขนาดใหญ่ที่มีบริบทซับซ้อน เช่น Closure และ JacksonDatabind โดยพบสถานะ `COMPILE_ERROR` จำนวน 6 รายการ จากทั้งหมด 851 รายการ คิดเป็น 0.71% ซึ่งน้อยกว่าเมื่อเทียบกับ DeepSeek [หากผลของ DeepSeek มีค่า `COMPILE_ERROR` สูงกว่า] โดยจากการตรวจสอบข้อผิดพลาดที่เกิดขึ้น พบว่าส่วนหนึ่งเกี่ยวข้องกับการจัดโครงสร้าง Class และ Method รวมถึงการกำหนด Import Statement ของชุดทดสอบ

สำหรับชุดทดสอบที่สามารถคอมไพล์ได้ จะเข้าสู่ขั้นตอนการรันและแบ่งเป็นสถานะ `PASS` และ `TEST_FAIL` โดยสถานะ `TEST_FAIL` หมายถึงมี Test Case ที่ทำให้การทดสอบไม่ผ่าน อย่างไรก็ตาม สถานะดังกล่าวยังไม่สามารถสรุปได้โดยตรงว่าเป็นการตรวจพบ Fault ของ Defects4J จนกว่าจะตรวจสอบผลการทำงานระหว่าง Buggy Version และ Fixed Version เพิ่มเติม ชุดทดสอบที่สามารถเข้าสู่ขั้นตอนการรันจะถูกนำไปใช้ในการวัดค่า Coverage ตามเกณฑ์ที่กำหนดต่อไป

4.4.3 ผลจำนวน Test Case ที่สร้างได้

จากข้อมูลระดับเมธอดที่สามารถรวบรวมได้จำนวน 347 Bugs พบว่า Gemini สามารถสร้าง Test Case ได้รวม 9,095 Test Cases หรือเฉลี่ย 26.21 Test Cases ต่อ Bug โดยมีค่ามาตรฐานเท่ากับ 20 Test Cases ต่อ Bug แบ่งเป็น Test Case ที่ผ่านการทดสอบ (`PASS`) จำนวน 2,538 Cases และ Test Case ที่ไม่ผ่านการทดสอบ (`FAIL`) จำนวน 6,557 Cases

ทั้งนี้ ข้อมูลจำนวน Test Case ดังกล่าวมีเฉพาะ 347 Bugs ที่มีข้อมูลในระดับเมธอด จึงไม่ควรตีความค่าเฉลี่ยดังกล่าวเป็นค่าเฉลี่ยของ Bugs ทั้งหมด 854 Bugs

4.4.4 ผลการตรวจจับ Fault

จากผลการรันชุดทดสอบของ Gemini พบว่า Bug ที่มีสถานะ `TEST_FAIL` จำนวน 508 Bugs จาก 851 Bugs ที่ปรากฏในตารางสรุปผล คิดเป็นประมาณ 59.7% ของ Bugs ที่มีผลการรันในตารางดังกล่าว

อย่างไรก็ตาม สถานะ `TEST_FAIL` หมายถึงมี Test Case ที่ทำให้การทดสอบไม่ผ่าน และยังไม่ควรตีความว่า Test Case ดังกล่าวตรวจพบ Fault ของ Defects4J ได้ทั้งหมดโดยตรง เนื่องจาก Test Failure อาจเกิดจากสาเหตุอื่น เช่น Assertion ไม่ตรงกับผลลัพธ์ที่คาดหวัง หรือปัญหาของชุดทดสอบ ดังนั้นการระบุว่าเป็น Fault Detection จำเป็นต้องตรวจสอบผลระหว่าง Buggy Version และ Fixed Version ตามเกณฑ์ที่กำหนด

4.4.5 ผล Line Coverage และ Branch Coverage

เนื่องจากข้อจำกัดด้านเวลา การทดลองนี้ยังไม่สามารถวัดค่า Line Coverage และ Branch Coverage ของชุดทดสอบที่ Gemini สร้างได้ครบถ้วน จึงไม่นำเสนอค่าดังกล่าวในรายงานนี้ และถือเป็นแนวทางการวิจัยต่อไป

4.4.6 ผลเวลาในการสร้าง Test Case

ข้อมูลเวลามีเฉพาะ 347 Bug เดียวกัน เฉลี่ย 29.07 วินาทีต่อ Bug เวลารวม 10,087.46 วินาที (ประมาณ 2.80 ชั่วโมง) ส่วน 504 Bug ที่เหลือบันทึกเวลาเป็น 0 ใช้ไม่ได้

4.4.7 ผลการทดลองแยกตามโปรเจกต์

ตารางที่ 33 สรุปสถานะการรัน Test Case จำแนกตามโปรเจกต์

โปรเจกต์	Bug	PASS	TEST_FAIL	COMPILE_ERROR	UNKNOWN
Chart	26	25	1	0	0
Cli	39	32	7	0	0
Closure	174	47	123	4	0
Codec	17	0	12	0	5
Collections	28	2	18	0	8
Compress	47	11	28	0	8
Csv	16	0	13	0	3
Gson	18	0	13	0	5
JacksonCore	26	2	16	0	8
JacksonDatabind	110	30	72	0	8
JacksonXml	6	0	6	0	0
Jsoup	93	20	66	0	7
JXPath	22	1	14	0	7
Lang	60	8	46	0	6
Math	105	57	39	1	8
Mockito	38	11	19	0	8
Time	26	2	15	1	8
รวม	851	248	508	6	89

4.5 ผลการวัด Test Coverage

4.5.1 Gemini

การวัดค่า Line Coverage และ Branch Coverage ของชุดทดสอบที่ Gemini สร้างยังไม่สามารถดำเนินการได้ครบถ้วนภายในกรอบเวลาของโครงการ ชุดทดสอบที่คอมไพล์ผ่านซึ่งเป็นกลุ่มที่เข้าเงื่อนไขการวัดค่า Coverage มีจำนวน 756 รายการ (PASS 248 รายการ และ TEST_FAIL 508 รายการ) จึงยังไม่นำเสนอค่า Coverage ในรายงานฉบับนี้

4.5.2 DeepSeek

ในทำนองเดียวกัน ยังไม่สามารถวัดค่า Line Coverage และ Branch Coverage ของชุดทดสอบที่ DeepSeek สร้างได้ ชุดทดสอบที่เข้าเงื่อนไขการวัดมีจำนวน 201 รายการ (PASS 40 รายการ และ TEST_FAIL 161 รายการ) จึงยังไม่นำเสนอค่า Coverage ในรายงานฉบับนี้ และกำหนดให้เป็นแนวทางในการศึกษาต่อไป

4.6 ผลการวัด Fault Detection

4.6.1 Gemini

ชุดทดสอบของ Gemini ตรวจจับ Fault ได้ (สถานะ TEST_FAIL) จำนวน 508 รายการ คิดเป็นร้อยละ 66.67 ของรายการที่รันได้ (762 รายการ) และร้อยละ 59.69 ของ Bug ทั้งหมด (851 รายการ) เมื่อพิจารณาแยกตามโปรเจกต์ พบว่าอัตราการตรวจจับสูงในโปรเจกต์ Codec, Csv, Gson และ JacksonXml (ร้อยละ 100 ของรายการที่รันได้) และ JXPath (ร้อยละ 93.33) ขณะที่โปรเจกต์ Chart (ร้อยละ 3.85) และ Cli (ร้อยละ 17.95) มีอัตราต่ำ

4.6.2 DeepSeek

ชุดทดสอบของ DeepSeek ตรวจจับ Fault ได้จำนวน 161 รายการ คิดเป็นร้อยละ 45.87 ของรายการที่รันได้ (351 รายการ) และร้อยละ 18.85 ของ Bug ทั้งหมด (854 รายการ) เมื่อพิจารณาแยกตามโปรเจกต์ อัตราการตรวจจับสูงสุดอยู่ที่ Closure (ร้อยละ 58.33) และ Cli (ร้อยละ 57.14) ต่ำสุดที่ JacksonXml (ร้อยละ 0 จาก 3 รายการ) และ Csv (ร้อยละ 20.00)

4.7 ผลการวัดประสิทธิภาพ

4.7.1 Gemini

ด้านอัตราความสำเร็จในการคอมไพล์ ชุดทดสอบของ Gemini คอมไพล์ผ่านร้อยละ 99.21 ของรายการที่รันได้ (756 จาก 762 รายการ) ด้านเวลา จากรายการที่มีข้อมูลระดับเมธอดจำนวน 347 Bug ใช้เวลาเฉลี่ย 29.07 วินาทีต่อ Bug เวลารวม 10,087.46 วินาที (ประมาณ 2.80 ชั่วโมง) สร้าง Test Case ได้รวม 9,095 ตัว เฉลี่ย 26.21 ตัวต่อ Bug คิดเป็นเวลาเฉลี่ย 1.11 วินาทีต่อ Test Case

4.7.2 DeepSeek

ชุดทดสอบของ DeepSeek คอมไพล์ผ่านร้อยละ 57.26 ของรายการที่รันได้ (201 จาก 351 รายการ) ด้านเวลา จากรายการที่มีข้อมูลระดับเมธอดจำนวน 96 Bug ใช้เวลาเฉลี่ย 108.65 วินาทีต่อ Bug เวลารวม 10,429.94 วินาที (ประมาณ 2.90 ชั่วโมง) สร้าง Test Case ได้รวม 5,482 ตัว เฉลี่ย 57.10 ตัวต่อ Bug คิดเป็นเวลาเฉลี่ย 1.90 วินาทีต่อ Test Case

ตารางที่ 34 เปรียบเทียบประสิทธิภาพของ Gemini และ DeepSeek

ตัวชี้วัด	Gemini	DeepSeek
อัตราคอมไพล์ผ่าน (ของรายการที่รันได้)	99.21%	57.26%
เวลาเฉลี่ยต่อ Bug (วินาที)	29.07	108.65
เวลาเฉลี่ยต่อ Test Case (วินาที)	1.11	1.90
Test Case เฉลี่ยต่อ Bug (ตัว)	26.21	57.10
เวลารวม (ชั่วโมง)	2.80	2.90
จำนวน Bug ที่ใช้คำนวณเวลา	347	96

4.8 สรุปผลการทดลอง

4.8.1 Gemini

Gemini ได้ผลลัพธ์ครอบคลุม 851 จาก 854 Bug ในจำนวนนี้มีผลการรันจริง 762 รายการ ชุดทดสอบคอมไพล์ผ่านร้อยละ 99.21 และตรวจจับ Fault ได้ร้อยละ 66.67 ของรายการที่รันได้ ใช้เวลาเฉลี่ย 29.07 วินาทีต่อ Bug โดยโปรเจกต์ขนาดใหญ่ เช่น Closure และ JacksonDatabind ก็สามารถสร้างชุดทดสอบที่คอมไพล์ผ่านและตรวจจับ Fault ได้ในอัตราร้อยละ 70 ขึ้นไป

4.8.2 DeepSeek

DeepSeek ดำเนินการได้ 351 จาก 854 Bug (ร้อยละ 41.1) ชุดทดสอบคอมไพล์ผ่านร้อยละ 57.26 โดยสาเหตุหลักของ COMPILE_ERROR คือข้อจำกัดด้านความยาวของผลลัพธ์ ตรวจจับ Fault ได้ร้อยละ 45.87 ของรายการที่รันได้ และใช้เวลาเฉลี่ย 108.65 วินาทีต่อ Bug

4.8.3 การเปรียบเทียบ

เมื่อเปรียบเทียบบนฐานรายการที่รันได้ Gemini มีอัตราคอมไพล์ผ่านและอัตราการตรวจจับ Fault สูงกว่า DeepSeek และใช้เวลาต่อ Bug น้อยกว่าราว 3.7 เท่า อย่างไรก็ตาม DeepSeek สร้าง Test Case ต่อ Bug ได้มากกว่า (57.10 เทียบกับ 26.21 ตัว) แต่จำนวนที่มากกว่าไม่ได้ส่งผลให้อัตราการตรวจจับ Fault สูงขึ้น เนื่องจากชุดทดสอบจำนวนมากไม่ผ่านการคอมไพล์

4.8.4 ข้อจำกัดของการทดลอง

1. ชุด Bug ที่รันได้ของทั้งสองโมเดลไม่เท่ากัน (762 และ 351 รายการ) การเปรียบเทียบจึงเป็นการเปรียบเทียบบนข้อมูลคนละชุด
2. ยังไม่ได้วัดค่า Line Coverage และ Branch Coverage
3. ข้อมูลเวลาและจำนวน Test Case มีเฉพาะ Bug ที่บันทึกผลระดับเมธอด (Gemini 347 Bug และ DeepSeek 96 Bug) และสถานะของ Bug กลุ่มนี้กำหนดจากเกณฑ์ว่าหากมีเมธอดไม่ผ่านอย่างน้อยหนึ่งเมธอดถือเป็น TEST_FAIL โดยข้อมูลกลุ่มนี้ไม่มีการบันทึกสถานะ COMPILE_ERROR จึงอาจทำให้อัตราคอมไพล์ผ่านของทั้งสองโมเดลสูงกว่าความเป็นจริง
4. Gemini ยังขาดผลของ Bug อีก 3 รายการจากทั้งหมด 854 รายการ

บทที่ 5

สรุปผลการดำเนินงาน

5.1 สรุปผลการดำเนินโครงการ

โครงการนี้มีวัตถุประสงค์เพื่อศึกษาและเปรียบเทียบแนวทางการสร้างกรณีทดสอบอัตโนมัติ (Automatic Test Case Generation: ATCG) กับการใช้ปัญญาประดิษฐ์เชิงสร้างสรรค์เพื่อช่วยสร้างกรณีทดสอบ (AI-Assisted Testing) โดยเลือกใช้เครื่องมือในกลุ่ม ATCG ได้แก่ Botsing และ CATG และเครื่องมือปัญญาประดิษฐ์ ได้แก่ Gemini และ DeepSeek เพื่อสร้าง JUnit Test Case สำหรับโปรแกรมภาษา Java

การทดลองใช้ชุดข้อมูล Defects4J v3.0.1 ซึ่งประกอบด้วยข้อบกพร่องที่เกิดขึ้นจริงในโครงการซอฟต์แวร์ Java จำนวน 854 Bugs จาก 17 Projects โดยกำหนดให้ผลลัพธ์ที่ได้จากแต่ละเครื่องมือถูกนำมาประเมินด้วยตัวชี้วัดที่เกี่ยวข้อง ได้แก่ จำนวน Test Case, Test Coverage, Fault Detection และ Generation Time เพื่อให้สามารถวิเคราะห์ข้อดี ข้อจำกัด และลักษณะการทำงานของแต่ละแนวทางได้

สำหรับ Botsing ได้ทำการทดลองแบบกรณีศึกษา (Case Study) กับ Lang_1b โดยเปรียบเทียบ Search Budget ที่ 30 และ 60 วินาที ส่วน CATG ได้ทำการทดลองกับ Bugs ใน Defects4J จำนวน 854 Bugs โดยใช้ Concolic Testing และกำหนด Search Budget ค่าเริ่มต้น 5 Concolic Iterations ต่อ Bug ขณะที่ Gemini และ DeepSeek ถูกนำมาใช้ในการสร้าง JUnit Test Case จาก Source Code และ Prompt ที่กำหนด

ผลการดำเนินโครงการทำให้ได้ข้อมูลสำหรับวิเคราะห์ความแตกต่างระหว่างการสร้าง Test Case ด้วยอัลกอริทึมแบบอัตโนมัติและการใช้ Generative AI รวมถึงข้อจำกัดที่เกิดขึ้นระหว่างการนำเครื่องมือแต่ละประเภทมาใช้งานจริง

5.2 สรุปผลการทดลอง

ผลการทดลองพบว่าเครื่องมือแต่ละประเภทยมีลักษณะและข้อจำกัดในการสร้าง Test Case แตกต่างกัน โดยสามารถสรุปผลการทดลองที่สำคัญได้ดังนี้

Botsing สามารถสร้าง Test Case สำหรับ Lang_1b ได้สำเร็จทั้งการทดลองที่กำหนด Search Budget 30 วินาทีและ 60 วินาที โดย Test Case ที่สร้างขึ้นสามารถ Compile และรันด้วย JUnit ได้ และสามารถกระตุ้น NumberFormatException ใน Buggy Version ขณะที่ Fixed Version สามารถทำงานผ่านได้ ส่งผลให้ Fault Detection เป็น TRUE ทั้งสองรอบ อย่างไรก็ตาม การทดลองของ Botsing ในโครงการนี้เป็นกรณีศึกษาเฉพาะ Bug และอาศัยข้อมูล Stack Trace และ Target Frame เป็นข้อมูลนำทาง จึงไม่สามารถใช้ผลดังกล่าวแทนประสิทธิภาพของ Botsing ในภาพรวมของ Defects4J ได้

CATG จาก Bugs ทั้งหมด 854 Bugs พบว่า Completed จำนวน 145 Bugs คิดเป็น 17.0%, Unsupported จำนวน 627 Bugs คิดเป็น 73.4% และ Error จำนวน 82 Bugs คิดเป็น 9.6% ในกลุ่มที่ Completed สามารถสร้าง Test Case ได้รวม 233 Cases หรือเฉลี่ย 1.61 Cases ต่อ Bug และสามารถตรวจพบ Fault ได้ 18 จาก 145 Bugs คิดเป็น 12.4% ค่าเฉลี่ย Line Coverage อยู่ที่ 5.91% และ Branch/Condition Coverage อยู่ที่ 2.26% ขณะที่เวลาในการสร้าง Test Case เฉลี่ยอยู่ที่ 2.64 วินาทีต่อ Bug

สาเหตุสำคัญของการไม่สามารถประเมินผล CATG จำนวน 709 Bugs คือข้อจำกัดของ Test Harness จำนวน 485 Bugs คิดเป็น 68.4% ของ Bugs ที่ไม่สำเร็จ รองลงมาคือข้อผิดพลาดระหว่างการทำงานของ CATG และปัญหาที่เกี่ยวข้องกับ Target Method, Checkout และ Compilation ดังนั้นผลการทดลองของ CATG จึงสะท้อนทั้งความสามารถของเครื่องมือและข้อจำกัดของสภาพแวดล้อมการทดลองร่วมกัน

สำหรับ Gemini และ DeepSeek ผลการทดลองจะพิจารณาจาก Test Case ที่สร้างขึ้นจาก Source Code และ Prompt ที่กำหนด โดยประเมินในด้านความถูกต้องของ Test Case, การ Compile และ Execution, Fault Detection, Test Coverage และเวลาในการสร้าง Test Case ทั้งนี้ผลลัพธ์ที่ได้ขึ้นอยู่กับ Source Code ที่ให้เป็นข้อมูลนำเข้า รวมถึงรูปแบบและรายละเอียดของ Prompt ที่ใช้ในการทดลอง

โดยภาพรวม การทดลองแสดงให้เห็นว่าไม่มีเครื่องมือใดที่เหมาะสมกับทุกสถานการณ์ เครื่องมือแบบ ATCG สามารถดำเนินการค้นหา Test Input ตามกลไกของอัลกอริทึมได้อย่างเป็นระบบ ขณะที่ Generative AI มีความยืดหยุ่นในการสร้าง Test Code และสามารถใช้อ้างอิงจาก Source Code เพื่อสร้างกรณีทดสอบในรูปแบบที่ต่างกันได้

5.3 การเปรียบเทียบผลการทดลอง

การเปรียบเทียบผลการทดลองแบ่งเครื่องมือออกเป็น 2 กลุ่ม ได้แก่ กลุ่ม Automatic Test Case Generation ซึ่งประกอบด้วย Botsing และ CATG และกลุ่ม AI-Assisted Testing ซึ่งประกอบด้วย Gemini และ DeepSeek โดยพิจารณาจากตัวชี้วัดที่สามารถเปรียบเทียบร่วมกันได้ ได้แก่ Test Case, Test Coverage, Fault Detection และ Generation Time

ในด้าน Test Case Generation เครื่องมือแต่ละประเภทมีวิธีการสร้าง Test Case ต่างกัน โดย Botsing และ CATG ใช้กระบวนการค้นหา Test Input ตามอัลกอริทึมที่กำหนด ขณะที่ Gemini และ DeepSeek ใช้ Generative AI ในการวิเคราะห์ Source Code และสร้าง Test Code จาก Prompt ส่งผลให้รูปแบบและจำนวน Test Case ที่ได้อาจแตกต่างกัน

ในด้าน Test Coverage Coverage ของ Test Case สามารถใช้เป็นตัวชี้วัดความสามารถในการครอบคลุมส่วนต่าง ๆ ของ Source Code อย่างไรก็ตาม ค่าที่ได้อาจได้รับผลกระทบจากจำนวน Test Case, ลักษณะของ Target

Method และขอบเขตของ Code ที่นำมาวัด Coverage ดังนั้นการเปรียบเทียบ Coverage ควรพิจารณาพร้อมกับเงื่อนไข และขอบเขตการทดลองของแต่ละเครื่องมือ

ในด้าน Fault Detection พิจารณาว่า Test Case สามารถทำให้ Buggy Version แสดงพฤติกรรมผิดพลาดและสามารถทำงานผ่าน Fixed Version ได้หรือไม่ ตัวชี้วัดนี้ช่วยแสดงความสามารถของ Test Case ในการตรวจพบข้อบกพร่องจริง อย่างไรก็ตาม ผลการตรวจจับอาจได้รับผลกระทบจากวิธีการสร้าง Test Harness, รูปแบบ Expected Output และเงื่อนไขของ Test Case

ในด้าน Generation Time สามารถใช้เปรียบเทียบประสิทธิภาพด้านเวลาในการสร้าง Test Case โดย CATG ในการทดลองนี้ใช้เวลาเฉลี่ย 2.64 วินาทีต่อ Bug และมีค่ามัธยฐาน 1.89 วินาที แสดงให้เห็นว่าสามารถสร้าง Test Case ได้ภายในเวลาค่อนข้างสั้นเมื่อสามารถดำเนินการจนเสร็จสิ้น แต่ไม่ควรใช้เวลาเพียงตัวเดียวในการตัดสินประสิทธิภาพ เนื่องจากต้องพิจารณาควบคู่กับ Coverage และ Fault Detection

จากการเปรียบเทียบจึงสรุปได้ว่า ATCG และ AI-Assisted Testing มีจุดเด่นแตกต่างกัน โดย ATCG มีกระบวนการสร้าง Test Case ที่อาศัยกลไกการค้นหาที่กำหนดไว้อย่างชัดเจน ส่วน Generative AI มีความยืดหยุ่นในการสร้างและปรับรูปแบบ Test Code ตามข้อมูลและ Prompt ที่ได้รับ ดังนั้นประสิทธิภาพของแต่ละแนวทางควรพิจารณาจากหลายตัวชี้วัดร่วมกัน และไม่ควรตัดสินจากตัวชี้วัดเพียงรายการเดียว

5.4 ข้อจำกัดของโครงการ

โครงการมีข้อจำกัดที่อาจส่งผลต่อการตีความและการเปรียบเทียบผลการทดลอง ดังนี้

1. ข้อจำกัดของ Test Harness การทดลอง CATG พบว่า Test Harness เป็นสาเหตุหลักที่ทำให้ไม่สามารถประเมินผลได้ โดยบาง Target Method มี Parameter หรือรูปแบบการสร้าง Object ที่ Test Harness ไม่สามารถรองรับได้ ส่งผลให้ CATG ไม่สามารถดำเนินการสร้าง Test Case ต่อได้
2. จำนวนรอบการทดลองการทดลองบางส่วนดำเนินการเพียงหนึ่งครั้งต่อ Bug หรือหนึ่งกรณีศึกษา จึงอาจไม่สะท้อนความแปรปรวนของผลลัพธ์ที่อาจเกิดขึ้นจากการสุ่มหรือสภาพแวดล้อมในการทำงาน
3. ข้อจำกัดในการประเมิน Fault Detection การตัดสิน Fault Detection อาศัยการเปรียบเทียบผลการทำงานของ Buggy Version และ Fixed Version และอาจได้รับผลกระทบจากรูปแบบ Expected Output หรือการสร้าง Object ใน Test Harness
4. ข้อจำกัดของ Generative AI ผลลัพธ์จาก Gemini และ DeepSeek อาจเปลี่ยนแปลงตาม Prompt, Source Code และบริบทที่ป้อนให้โมเดล ทำให้การควบคุมผลลัพธ์ให้เหมือนกันทุกครั้งทำได้ยาก

5.5 ข้อเสนอแนะและแนวทางการพัฒนาต่อ

สำหรับการพัฒนาและต่อยอดโครงการในอนาคต สามารถดำเนินการได้ดังนี้

1. ปรับปรุง Test Harness ให้รองรับ Parameter Type, Object และ Target Method ที่หลากหลายมากขึ้น เพื่อลดจำนวน Bugs ที่อยู่ในสถานะ Unsupported
2. เพิ่มจำนวนรอบการทดลอง เพื่อให้สามารถวิเคราะห์ความแปรปรวนของผลลัพธ์และลดผลกระทบจากการสุ่มของอัลกอริทึมและ Generative AI
3. ปรับปรุงและควบคุม Prompt สำหรับ Gemini และ DeepSeek โดยกำหนดรูปแบบ Input, Output, Test Framework และเงื่อนไขของ Test Case ให้เหมือนกันมากที่สุด เพื่อให้การเปรียบเทียบมีความเป็นธรรมมากขึ้น
4. เพิ่มตัวชี้วัดในการประเมิน เช่น ความซับซ้อนของ Test Case, Mutation Score และคุณภาพของ Assertion เพื่อประเมินคุณภาพของ Test Suite ได้ละเอียดมากขึ้น
5. ขยายขอบเขตการทดลอง ให้สามารถทดสอบเครื่องมือทั้งหมดกับ Bugs ที่เป็นชุดเดียวกันและมีเงื่อนไขการทดลองเดียวกันมากขึ้น เพื่อให้สามารถเปรียบเทียบประสิทธิภาพของแต่ละแนวทางได้อย่างชัดเจน
6. ปรับปรุงกระบวนการประเมินผลอัตโนมัติ เพื่อให้การ Compile, Execute, Coverage และ Fault Detection สามารถดำเนินการด้วยขั้นตอนเดียวกันสำหรับทุกเครื่องมือ และลดผลกระทบจากความแตกต่างของสภาพแวดล้อมการทดลอง

เอกสารอ้างอิง

- Susnjara, S., & Smalley, I. (2026, June 22). *What is software testing?* IBM. IBM Think – What is software testing?
- Devaraj, K. (2026, January 23). *Test suite vs. test case*. Testsigma. <https://testsigma.com/blog/test-suite-vs-test-case/>
- Gupta, K. (2026, March 3). *What is a test suite and test case?* BrowserStack. <https://www.browserstack.com/guide/what-is-test-suite-and-test-case>
- Understand test case and scenario. (2025, April 9). Skooldio. <https://blog.skooldio.com/understand-test-case-and-scenario/>
- Singh, J., & Sharma, S. (2018). *Fault detection technique for test cases in software engineering*. International Journal of Engineering & Technology, 7(1), 53. <https://doi.org/10.14419/ijet.v7i1.7870>
- Powell, P., & Smalley, I. (n.d.). *What is unit testing?* IBM. <https://www.ibm.com/think/topics/unit-testing>
- JUnit. (n.d.). *JUnit 5*. <https://junit.org/>
- JUnit. (n.d.). *Assertions*. In *JUnit 6.1.3 user guide*. <https://docs.junit.org/6.1.3/writing-tests/assertions.html>
- PractiTest Team. (2024, April 30). Automatic test case generation. PractiTest. <https://www.practitest.com/resource-center/blog/automatic-test-case-generation/>
- Shaikh, V., Walunj, V., Hanumantharaya, A., & Vanama, H. (2025, November). *Automated test case generation (ATCG): Leveraging large language models for software testing*. https://www.researchgate.net/publication/397280753_Automated_Test_Case_Generation_ATCG_Leveraging_Large_Language_Models_for_Software_Testing
- Derakhshanfar, P., Devroey, X., Panichella, A., Zaidman, A., & van Deursen, A. (2020). *Botsing, a search-based crash reproduction framework for Java*. In J. Grundy, D. Lo, & C. Le Goues (Eds.), Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE 2020) (pp. 1278–1282). Association for Computing Machinery. <https://doi.org/10.1145/3324884.3415299>
- Le Goues, C. (2020). *Concolic testing [Lecture notes]*. 17-355/17-665/17-819: Program Analysis, Carnegie Mellon University. <https://cmu-programanalysis.github.io/2020/lecture-notes/notes12-concolic-testing.pdf>
- Auer, M., Diner, D., & Fraser, G. (2024). Search-based Crash Reproduction for Android

- Apps. *Proceedings of the Genetic and Evolutionary Computation Conference*.
<https://doi.org/10.1145/3638529.3654034>
- Derakhshanfar, P., Devroey, X., Panichella, A., Zaidman, A., & Deursen, A. (2020). Botsing, a Search-based Crash Reproduction Framework for Java. *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 1278-1282. <https://doi.org/10.1145/3324884.3415299>
- Derakhshanfar, P. (2021). Carving Information Sources to Drive Search-Based Crash Reproduction and Test Case Generation.
<https://doi.org/10.4233/uuid:aac5f17a-63d5-45c7-9570-3cea057cd016>
- Derakhshanfar, P., Devroey, X., Perrouin, G., Zaidman, A., & Deursen, A. (2019). Search-based crash reproduction using behavioural model seeding. *Software Testing, 30*. <https://doi.org/10.1002/stvr.1733>
- Derakhshanfar, P., Devroey, X., & Zaidman, A. (2022). Basic block coverage for search-based unit testing and crash reproduction. *Empirical Software Engineering, 27*, 1-41. <https://doi.org/10.1007/s10664-022-10155-0>
- Derakhshanfar, P., Devroey, X., & Zaidman, A. (2020). It Is Not Only About Control Dependent Nodes: Basic Block Coverage for Search-Based Crash Reproduction. 42-57. https://doi.org/10.1007/978-3-030-59762-7_4
- Derakhshanfar, P., Devroey, X., Perrouin, G., Zaidman, A., & Deursen, A. (2021). Summary of Search-based Crash Reproduction using Behavioral Model Seeding. *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, 281-281. <https://doi.org/10.1109/icst49551.2021.00039>
- Kang, S., Yoon, J., & Yoo, S. (2022). Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2312-2323. <https://doi.org/10.1109/icse48619.2023.00194>
- Oliver, P. (2024). Investigating Configurations for Automated Crash Reproduction in JavaScript. <https://doi.org/10.26686/wgtn.27633654>
- Soltani, M., Panichella, A., & Van Deursen, A. (2020). Search-Based Crash Reproduction and Its Impact on Debugging. *IEEE Transactions on Software Engineering, 46*,

- 1294-1317. <https://doi.org/10.1109/tse.2018.2877664>
- Soltani, M., Panichella, A., & Deursen, A. (2016). Evolutionary Testing for Crash Reproduction. *2016 IEEE/ACM 9th International Workshop on Search-Based Software Testing (SBST)*, 1-4. <https://doi.org/10.1145/2897010.2897015>
- Soltani, M., Derakhshanfar, P., Devroey, X., & Van Deursen, A. (2019). A benchmark-based evaluation of search-based crash reproduction. *Empirical Software Engineering, 25*, 96 - 138. <https://doi.org/10.1007/s10664-019-09762-1>
- Tanno, H., Zhang, X., Hoshino, T., Sen, K., et al. (2015). TesMa and CATG: Automated test generation tools for models of enterprise applications. Proceedings of the 2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops, 717–720. <https://doi.org/10.5555/2819009.2819147>
- Pham, L. H., Le, Q. L., Phan, Q.-S., & Sun, J. (2019). Concolic testing heap-manipulating programs. arXiv. <https://arxiv.org/abs/1907.05637>
- Olsthooorn, M. (2022). More effective test case generation with multiple tribes of AI. In Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings (ICSE '22) (pp. 286–290). Association for Computing Machinery. <https://doi.org/10.1145/3510454.3517066>
- Sen, K., Marinov, D., & Agha, G. (2005). CUTE: A concolic unit testing engine for C. Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE-13), 263–272. <https://doi.org/10.1145/1081706.1081750>
- Pradel, M. (2021). Exercise 3: Test generation: Fuzzing, symbolic and concolic execution [Course exercise]. University of Stuttgart. <https://software-lab.org/teaching/winter2021/pa/exercise3.pdf>
- Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. Proceedings of the 2014 International Symposium on Software Testing and Analysis, 437–440. <https://doi.org/10.1145/2610384.2628055>

- Defects4J. (2026). Defects4J: A database of real faults and an experimental infrastructure to enable controlled experiments in software engineering research. <https://defects4j.org/>
- Google AI for Developers. (n.d.). Text generation. <https://ai.google.dev/gemini-api/docs/text-generation>
- JaCoCo. (n.d.). Counters. <https://www.jacoco.org/jacoco/trunk/doc/counters.html>
- Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Stella, M., & Yampolskiy, M. (2024). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.600-1>
- JUnit Team. (n.d.). JUnit 5 User Guide: Writing Tests. <https://junit.org/junit5/docs/current/user-guide/#writing-tests>
- Guo, D., et al. (2024). DeepSeek-Coder: When the Large Language Model Meets Programming. arXiv. <https://arxiv.org/abs/2401.14196>